



Giáo trình CT174 Phân tích thiết kế thuật toán (04 chương)

Phân tích và thiết kế thuật toán (Trường Đại học Cần Thơ)



Scan to open on Studocu

Th.s. NGUYỄN VĂN LINH

GIẢI THUẬT

**Được biên soạn trong khuôn khổ dự án ASVIET002CNTT
"Tăng cường hiệu quả đào tạo và năng lực tự đào tạo của sinh viên
khoa Công nghệ Thông tin - Đại học Cần thơ"**

ĐẠI HỌC CẦN THƠ - 12/2003

LỜI NÓI ĐẦU

N. Wirth, một nhà khoa học máy tính nổi tiếng, tác giả của ngôn ngữ lập trình Pascal, đã đặt tên cho một cuốn sách của ông là

“Cấu trúc dữ liệu + Giải thuật = Chương trình”.

Điều đó nói lên tầm quan trọng của giải thuật trong lập trình nói riêng và trong khoa học máy tính nói chung. Vì lẽ đó giải thuật, với tư cách là một môn học, cần phải được sinh viên chuyên ngành tin học nghiên cứu một cách có hệ thống.

Môn học “Giải thuật” được bố trí sau môn “Cấu trúc dữ liệu” trong chương trình đào tạo kỹ sư tin học nhằm giới thiệu cho sinh viên những kiến thức cơ bản nhất, những kỹ thuật chủ yếu nhất của việc PHÂN TÍCH và THIẾT KẾ giải thuật. Các kỹ thuật được trình bày ở đây đã được các nhà khoa học tin học tổng kết và vận dụng trong cài đặt các chương trình. Việc nắm vững các kỹ thuật đó sẽ rất bổ ích cho sinh viên khi phải giải quyết một vấn đề thực tế.

Giáo trình này được hình thành trên cơ sở tham khảo cuốn sách “Data Structure and Algorithms” của A.V Aho, những kinh nghiệm giảng dạy của bản thân và các bạn đồng nghiệp.

Mặc dù đã có nhiều cố gắng trong quá trình biên soạn nhưng chắc chắn còn nhiều thiếu sót, rất mong nhận được sự đóng góp của quý bạn đọc.

Cần thơ, ngày 8 tháng 12 năm 2003

Nguyễn Văn Linh

MỤC LỤC

PHẦN TỔNG QUAN	i
Chương 1: KỸ THUẬT PHÂN TÍCH GIẢI THUẬT	1
1.1 TỔNG QUAN	1
1.2 SỰ CẦN THIẾT PHẢI PHÂN TÍCH GIẢI THUẬT	2
1.3 THỜI GIAN THỰC HIỆN CỦA GIẢI THUẬT	2
1.4 TỶ SUẤT TĂNG VÀ ĐỘ PHỨC TẠP CỦA GIẢI THUẬT	3
1.5 CÁCH TÍNH ĐỘ PHỨC TẠP	4
1.6 PHÂN TÍCH CÁC CHƯƠNG TRÌNH ĐỆ QUY	7
1.7 TỔNG KẾT CHƯƠNG 1	16
BÀI TẬP CHƯƠNG 1	16
Chương 2: SẮP XẾP	18
2.1 TỔNG QUAN	18
2.2 BÀI TOÁN SẮP XẾP	19
2.3 CÁC PHƯƠNG PHÁP SẮP XẾP ĐƠN GIẢN	20
2.4 QUICKSORT	25
2.5 HEAPSORT	31
2.6 BINSORT	39
2.7 TỔNG KẾT CHƯƠNG 2	44
BÀI TẬP CHƯƠNG 2	44
Chương 3: KỸ THUẬT THIẾT KẾ GIẢI THUẬT	45
3.1 TỔNG QUAN	45
3.2 KỸ THUẬT CHIA ĐỀ TRỊ	45
3.3 KỸ THUẬT “THAM ĂN”	50
3.4 QUY HOẠCH ĐỘNG	56
3.5 KỸ THUẬT QUAY LUI	63
3.6 KỸ THUẬT TÌM KIẾM ĐỊA PHƯƠNG	78
3.7 TỔNG KẾT CHƯƠNG 3	82
BÀI TẬP CHƯƠNG 3	82
Chương 4: CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT LƯU TRỮ NGOÀI	85
4.1 TỔNG QUAN	85
4.2 MÔ HÌNH XỬ LÝ NGOÀI	85
4.3 ĐÁNH GIÁ CÁC GIẢI THUẬT XỬ LÝ NGOÀI	86
4.4 SẮP XẾP NGOÀI	87
4.5 LƯU TRỮ THÔNG TIN TRONG TẬP TIN	93
4.6 TỔNG KẾT CHƯƠNG 4	103
BÀI TẬP CHƯƠNG 4	104

PHẦN TỔNG QUAN

1. Mục đích yêu cầu

Môn học giải thuật cung cấp cho sinh viên một khối lượng kiến thức tương đối hoàn chỉnh về phân tích và thiết kế các giải thuật lập trình cho máy tính. Sau khi học xong môn học này, sinh viên cần:

- Hiểu được khái niệm thời gian thực hiện của chương trình, độ phức tạp của giải thuật. Biết cách phân tích, đánh giá giải thuật thông qua việc tính độ phức tạp.
- Hiểu được các giải thuật sắp xếp và phân tích đánh giá được các giải thuật sắp xếp.
- Hiểu được các kỹ thuật thiết kế giải thuật, vận dụng vào việc giải một số bài toán thực tế.
- Hiểu được các phương pháp tổ chức lưu trữ thông tin trong tập tin và các giải thuật tìm, xen, xoá thông tin trong tập tin.

2. Đối tượng sử dụng

Môn học giải thuật được dùng để giảng dạy cho các sinh viên sau:

- Sinh viên năm thứ 3 chuyên ngành Tin học.
- Sinh viên năm thứ 3 chuyên ngành Điện tử (Viễn thông, Tự động hoá...)
- Sinh viên Toán-Tin.

3. Nội dung cốt lõi

Trong khuôn khổ 45 tiết, giáo trình được cấu trúc thành 4 chương

- **Chương 1:** Kỹ thuật phân tích đánh giá giải thuật. Chương này đặt vấn đề tại sao cần phải phân tích, đánh giá giải thuật và phân tích đánh giá theo phương pháp nào. Nội dung chương 1 tập trung vào khái niệm độ phức tạp thời gian của giải thuật và phương pháp tính độ phức tạp giải thuật của một chương trình bình thường, của chương trình có gọi các chương trình con và của các chương trình đệ quy.
- **Chương 2:** Sắp xếp. Chương này trình bày các giải thuật sắp xếp, một thao tác thường được sử dụng trong việc giải các bài toán máy tính. Sẽ có nhiều giải thuật sắp xếp từ đơn giản đến nâng cao sẽ được giới thiệu ở đây. Với mỗi giải thuật, sẽ trình bày ý tưởng giải thuật, ví dụ minh họa, cài đặt chương trình và phân tích đánh giá.
- **Chương 3:** Kỹ thuật thiết kế giải thuật. Chương này trình bày các kỹ thuật phổ biến để thiết kế các giải thuật. Các kỹ thuật này gồm: Chia để trị, Quy hoạch động, Tham ăn, Quay lui và Tìm kiếm địa phương. Với mỗi kỹ thuật sẽ trình bày nội dung kỹ thuật và vận dụng vào giải các bài toán khá nổi tiếng như bài toán người giao hàng, bài toán cái ba lô, bài toán cây phủ tối thiểu...
- **Chương 4:** Cấu trúc dữ liệu và giải thuật lưu trữ ngoài. Chương này trình bày các cấu trúc dữ liệu được dùng để tổ chức lưu trữ tập tin trên bộ nhớ ngoài và các giải thuật tìm kiếm, xen xoá thông tin trên các tập tin đó.

4. Kiến thức tiên quyết

Để học tốt môn học giải thuật cần phải có các kiến thức sau:

- Kiến thức toán học.
- Kiến thức và kỹ năng lập trình căn bản.

- Kiến thức về cấu trúc dữ liệu và các giải thuật thao tác trên các cấu trúc dữ liệu.

Trong chương trình đào tạo, Cấu trúc dữ liệu là môn học tiên quyết của môn Giải thuật.

5. Danh mục tài liệu tham khảo

- [1] **A.V. Aho, J.E. Hopcroft, J.D. Ullman**; *Data Structures and Algorithms*; Addison-Wesley; 1983.
- [2] **Jeffrey H Kingston**; *Algorithms and Data Structures*; Addison-Wesley; 1998.
- [3] **Đinh Mạnh Tường**; *Cấu trúc dữ liệu & Thuật toán*; Nhà xuất bản khoa học và kỹ thuật; Hà nội-2001.
- [4] **Đỗ Xuân Lôí**; *Cấu trúc dữ liệu & Giải thuật*; 1995.
- [5] **Nguyễn Đức Nghĩa, Tô Văn Thành**; *Toán rời rạc*; 1997.
- [6] Trang web phân tích giải thuật: <http://pauillac.inria.fr/algo/AofA/>
- [7] Trang web bài giảng về giải thuật:
<http://www.cs.pitt.edu/~kirk/algorithmcourses/>
- [8] Trang tìm kiếm các giải thuật:
<http://oopweb.com/Algorithms/Files/Algorithms.html>

CHƯƠNG 1: KỸ THUẬT PHÂN TÍCH GIẢI THUẬT

1.1 TỔNG QUAN

1.1.1 Mục tiêu

Sau khi học chương này, sinh viên cần phải trả lời được các câu hỏi sau:

- Tại sao cần phân tích đánh giá giải thuật?
- Tiêu chuẩn nào để đánh giá một giải thuật là tốt?
- Phương pháp đánh giá như thế nào? (đánh giá chương trình không gọi chương trình con, đánh giá một chương trình có gọi các chương trình con không đệ quy và đánh giá chương trình đệ quy).

1.1.2 Kiến thức cơ bản cần thiết

Các kiến thức cơ bản cần thiết để học chương này bao gồm:

- Kiến thức toán học: Công thức tính tổng n số tự nhiên đầu tiên, công thức tính tổng n số hạng đầu tiên của một cấp số nhân, phương pháp chứng minh quy nạp và các kiến thức liên quan đến logarit (biến đổi logarit, tính chất đồng biến của hàm số logarit).
- Kỹ thuật lập trình và lập trình đệ quy.

1.1.3 Tài liệu tham khảo

A.V. Aho, J.E. Hopcroft, J.D. Ullman. *Data Structures and Algorithms*. Addison-Wesley. 1983. (Chapters 1, 9).

Jeffrey H Kingston; *Algorithms and Data Structures*; Addison-Wesley; 1998. (Chapter 2).

Đinh Mạnh Tường. *Cấu trúc dữ liệu & Thuật toán*. Nhà xuất bản khoa học và kĩ thuật. Hà nội-2001. (Chương 1).

Trang web phân tích giải thuật: <http://pauillac.inria.fr/algo/AofA/>

1.1.4 Nội dung cốt lõi

Trong chương này chúng ta sẽ nghiên cứu các vấn đề sau:

- Sự cần thiết phải phân tích các giải thuật.
- Thời gian thực hiện của chương trình.
- Tỷ suất tăng và độ phức tạp của giải thuật.
- Tính thời gian thực hiện của chương trình.
- Phân tích các chương trình đệ quy.

1.2 SỰ CẦN THIẾT PHẢI PHÂN TÍCH GIẢI THUẬT

Trong khi giải một bài toán chúng ta có thể có một số giải thuật khác nhau, vấn đề là cần phải đánh giá các giải thuật đó để lựa chọn một giải thuật tốt (nhất). Thông thường thì ta sẽ căn cứ vào các tiêu chuẩn sau:

- 1.- Giải thuật đúng đắn.
- 2.- Giải thuật đơn giản.
- 3.- Giải thuật thực hiện nhanh.

Với yêu cầu (1), để kiểm tra tính đúng đắn của giải thuật chúng ta có thể cài đặt giải thuật đó và cho thực hiện trên máy với một số bộ dữ liệu mẫu rồi lấy kết quả thu được so sánh với kết quả đã biết. Thực ra thì cách làm này không chắc chắn bởi vì có thể giải thuật đúng với tất cả các bộ dữ liệu chúng ta đã thử nhưng lại sai với một bộ dữ liệu nào đó. Và lại cách làm này chỉ phát hiện ra giải thuật sai chứ chưa chứng minh được là nó đúng. Tính đúng đắn của giải thuật cần phải được chứng minh bằng toán học. Tất nhiên điều này không đơn giản và do vậy chúng ta sẽ không đề cập đến ở đây.

Khi chúng ta viết một chương trình để sử dụng một vài lần thì yêu cầu (2) là quan trọng nhất. Chúng ta cần một giải thuật dễ viết chương trình để nhanh chóng có được kết quả, thời gian thực hiện chương trình không được đề cao vì dù sao thì chương trình đó cũng chỉ sử dụng một vài lần mà thôi.

Tuy nhiên khi một chương trình được sử dụng nhiều lần thì yêu cầu tiết kiệm thời gian thực hiện chương trình lại rất quan trọng đặc biệt đối với những chương trình mà khi thực hiện cần dữ liệu nhập lớn do đó yêu cầu (3) sẽ được xem xét một cách kĩ càng. Ta gọi nó là hiệu quả thời gian thực hiện của giải thuật.

1.3 THỜI GIAN THỰC HIỆN CỦA CHƯƠNG TRÌNH

Một phương pháp để xác định hiệu quả thời gian thực hiện của một giải thuật là lập trình nó và đo lường thời gian thực hiện của hoạt động trên một máy tính xác định đối với tập hợp được chọn lọc các dữ liệu vào.

Thời gian thực hiện không chỉ phụ thuộc vào giải thuật mà còn phụ thuộc vào tập các chỉ thị của máy tính, chất lượng của máy tính và kĩ xảo của người lập trình. Sự thi hành cũng có thể điều chỉnh để thực hiện tốt trên tập đặc biệt các dữ liệu vào được chọn. Để vượt qua các trở ngại này, các nhà khoa học máy tính đã chấp nhận tính phức tạp của thời gian được tiếp cận như một sự đo lường cơ bản sự thực thi của giải thuật. Thuật ngữ tính hiệu quả sẽ đề cập đến sự đo lường này và đặc biệt đối với sự phức tạp thời gian trong trường hợp xấu nhất.

1.3.1 Thời gian thực hiện chương trình.

Thời gian thực hiện một chương trình là một hàm của kích thước dữ liệu vào, ký hiệu $T(n)$ trong đó n là kích thước (độ lớn) của dữ liệu vào.

Ví dụ 1-1: Chương trình tính tổng của n số có thời gian thực hiện là $T(n) = cn$ trong đó c là một hằng số.

Thời gian thực hiện chương trình là một hàm không âm, tức là $T(n) \geq 0 \forall n \geq 0$.

1.3.2 Đơn vị đo thời gian thực hiện.

Đơn vị của $T(n)$ không phải là đơn vị đo thời gian bình thường như giờ, phút giây... mà thường được xác định bởi số các lệnh được thực hiện trong một máy tính lý tưởng.

Ví dụ 1-2: Khi ta nói thời gian thực hiện của một chương trình là $T(n) = Cn$ thì có nghĩa là chương trình ấy cần Cn chỉ thị thực thi.

1.3.3 Thời gian thực hiện trong trường hợp xấu nhất.

Nói chung thì thời gian thực hiện chương trình không chỉ phụ thuộc vào kích thước mà còn phụ thuộc vào tính chất của dữ liệu vào. Nghĩa là dữ liệu vào có cùng kích thước nhưng thời gian thực hiện chương trình có thể khác nhau. Chẳng hạn chương trình sắp xếp dãy số nguyên tăng dần, khi ta cho vào dãy có thứ tự thì thời gian thực hiện khác với khi ta cho vào dãy chưa có thứ tự, hoặc khi ta cho vào một dãy đã có thứ tự tăng thì thời gian thực hiện cũng khác so với khi ta cho vào một dãy đã có thứ tự giảm.

Vì vậy thường ta coi $T(n)$ là thời gian thực hiện chương trình trong trường hợp xấu nhất trên dữ liệu vào có kích thước n , tức là: $T(n)$ là thời gian lớn nhất để thực hiện chương trình đối với mọi dữ liệu vào có cùng kích thước n .

1.4 TỶ SUẤT TĂNG VÀ ĐỘ PHỨC TẠP CỦA GIẢI THUẬT

1.4.1 Tỷ suất tăng

Ta nói rằng hàm không âm $T(n)$ có tỷ suất tăng (growth rate) $f(n)$ nếu tồn tại các hằng số C và N_0 sao cho $T(n) \leq Cf(n)$ với mọi $n \geq N_0$.

Ta có thể chứng minh được rằng “Cho một hàm không âm $T(n)$ bất kỳ, ta luôn tìm được tỷ suất tăng $f(n)$ của nó”.

Ví dụ 1-3: Giả sử $T(0) = 1$, $T(1) = 4$ và tổng quát $T(n) = (n+1)^2$. Đặt $N_0 = 1$ và $C = 4$ thì với mọi $n \geq 1$ chúng ta dễ dàng chứng minh được rằng $T(n) = (n+1)^2 \leq 4n^2$ với mọi $n \geq 1$, tức là tỷ suất tăng của $T(n)$ là n^2 .

Ví dụ 1-4: Tỷ suất tăng của hàm $T(n) = 3n^3 + 2n^2$ là n^3 . Thực vậy, cho $N_0 = 0$ và $C = 5$ ta dễ dàng chứng minh rằng với mọi $n \geq 0$ thì $3n^3 + 2n^2 \leq 5n^3$.

1.4.2 Khái niệm độ phức tạp của giải thuật

Giả sử ta có hai giải thuật $P1$ và $P2$ với thời gian thực hiện tương ứng là $T1(n) = 100n^2$ (với tỷ suất tăng là n^2) và $T2(n) = 5n^3$ (với tỷ suất tăng là n^3). Giải thuật nào sẽ thực hiện nhanh hơn? Câu trả lời phụ thuộc vào kích thước dữ liệu vào. Với $n < 20$ thì $P2$ sẽ nhanh hơn $P1$ ($T2 < T1$), do hệ số của $5n^3$ nhỏ hơn hệ số của $100n^2$ ($5 < 100$). Nhưng khi $n > 20$ thì ngược lại do số mũ của $100n^2$ nhỏ hơn số mũ của $5n^3$ ($2 < 3$). Ở đây chúng ta chỉ nên quan tâm đến trường hợp $n > 20$ vì khi $n < 20$ thì thời gian thực hiện của cả $P1$ và $P2$ đều không lớn và sự khác biệt giữa $T1$ và $T2$ là không đáng kể.

Như vậy một cách hợp lý là ta xét tỷ suất tăng của hàm thời gian thực hiện chương trình thay vì xét chính bản thân thời gian thực hiện.

Cho một hàm $T(n)$, $T(n)$ gọi là có độ phức tạp $f(n)$ nếu tồn tại các hằng C, N_0 sao cho $T(n) \leq Cf(n)$ với mọi $n \geq N_0$ (tức là $T(n)$ có tỷ suất tăng là $f(n)$) và kí hiệu $T(n)$ là $O(f(n))$ (đọc là “ô của $f(n)$ ”)

Ví dụ 1-5: $T(n) = (n+1)^2$ có tỷ suất tăng là n^2 nên $T(n) = (n+1)^2$ là $O(n^2)$

Chú ý: $O(C.f(n)) = O(f(n))$ với C là hằng số. Đặc biệt $O(C) = O(1)$

Nói cách khác độ phức tạp tính toán của giải thuật là một hàm chặn trên của hàm thời gian. Vì hằng nhân tử C trong hàm chặn trên không có ý nghĩa nên ta có thể bỏ qua vì vậy hàm thể hiện độ phức tạp có các dạng thường gặp sau: $\log_2 n$, n , $n \log_2 n$, n^2 , n^3 , 2^n , $n!$, n^n . Ba hàm cuối cùng ta gọi là dạng *hàm mũ*, các hàm khác gọi là *hàm đa thức*. Một giải thuật mà thời gian thực hiện có độ phức tạp là một hàm đa thức thì chấp nhận được tức là có thể cài đặt để thực hiện, còn các giải thuật có độ phức tạp hàm mũ thì phải tìm cách cải tiến giải thuật.

Vì ký hiệu $\log_2 n$ thường có mặt trong độ phức tạp nên trong khuôn khổ tài liệu này, ta sẽ dùng **logn** thay thế cho **log₂n** với mục đích duy nhất là để cho gọn trong cách viết.

Khi nói đến độ phức tạp của giải thuật là ta muốn nói đến hiệu quả của thời gian thực hiện của chương trình nên ta có thể xem việc xác định thời gian thực hiện của chương trình chính là xác định độ phức tạp của giải thuật.

1.5 CÁCH TÍNH ĐỘ PHỨC TẠP

Cách tính độ phức tạp của một giải thuật bất kỳ là một vấn đề không đơn giản. Tuy nhiên ta có thể tuân theo một số nguyên tắc sau:

1.5.1 Quy tắc cộng

Nếu $T1(n)$ và $T2(n)$ là thời gian thực hiện của hai đoạn chương trình $P1$ và $P2$; và $T1(n) = O(f(n))$, $T2(n) = O(g(n))$ thì thời gian thực hiện của đoạn hai chương trình đó **nối tiếp nhau** là $T(n) = O(\max(f(n), g(n)))$

Ví dụ 1-6: Lệnh gán $x := 15$ tốn một hằng thời gian hay $O(1)$, Lệnh đọc dữ liệu $READ(x)$ tốn một hằng thời gian hay $O(1)$. Vậy thời gian thực hiện cả hai lệnh trên nối tiếp nhau là $O(\max(1, 1)) = O(1)$

1.5.2 Quy tắc nhân

Nếu $T1(n)$ và $T2(n)$ là thời gian thực hiện của hai đoạn chương trình $P1$ và $P2$ và $T1(n) = O(f(n))$, $T2(n) = O(g(n))$ thì thời gian thực hiện của đoạn hai đoạn chương trình đó **lồng nhau** là $T(n) = O(f(n).g(n))$

1.5.3 Quy tắc tổng quát để phân tích một chương trình:

- Thời gian thực hiện của mỗi lệnh gán, READ, WRITE là $O(1)$

- Thời gian thực hiện của một chuỗi tuần tự các lệnh được xác định bằng qui tắc cộng. Như vậy thời gian này là thời gian thi hành một lệnh nào đó lâu nhất trong chuỗi lệnh.
- Thời gian thực hiện cấu trúc IF là thời gian lớn nhất thực hiện lệnh sau THEN hoặc sau ELSE và thời gian kiểm tra điều kiện. Thường thời gian kiểm tra điều kiện là $O(1)$.
- Thời gian thực hiện vòng lặp là tổng (trên tất cả các lần lặp) thời gian thực hiện thân vòng lặp. Nếu thời gian thực hiện thân vòng lặp không đổi thì thời gian thực hiện vòng lặp là tích của số lần lặp với thời gian thực hiện thân vòng lặp.

Ví dụ 1-7: Tính thời gian thực hiện của thủ tục sắp xếp “nổi bọt”

```
PROCEDURE Bubble(VAR a: ARRAY[1..n] OF integer);
VAR i, j, temp: Integer;
BEGIN
{1}  FOR i:=1 TO n-1 DO
{2}      FOR j:=n DOWNTO i+1 DO
{3}          IF a[j-1]>a[j] THEN BEGIN{hoán vị a[i], a[j]}
{4}              temp := a[j-1];
{5}              a[j-1] := a[j];
{6}              a[j] := temp;
          END;
END;
```

Về giải thuật sắp xếp nổi bọt, chúng ta sẽ bàn kĩ hơn trong chương 2. Ở đây, chúng ta chỉ quan tâm đến độ phức tạp của giải thuật.

Ta thấy toàn bộ chương trình chỉ gồm một lệnh lặp {1}, lồng trong lệnh {1} là lệnh {2}, lồng trong lệnh {2} là lệnh {3} và lồng trong lệnh {3} là 3 lệnh nối tiếp nhau {4}, {5} và {6}. Chúng ta sẽ tiến hành tính độ phức tạp theo thứ tự từ trong ra.

Trước hết, cả ba lệnh gán {4}, {5} và {6} đều tốn $O(1)$ thời gian, việc so sánh $a[j-1] > a[j]$ cũng tốn $O(1)$ thời gian, do đó lệnh {3} tốn $O(1)$ thời gian.

Vòng lặp {2} thực hiện $(n-i)$ lần, mỗi lần $O(1)$ do đó vòng lặp {2} tốn $O((n-i).1) = O(n-i)$.

Vòng lặp {1} lặp có i chạy từ 1 đến $n-1$ nên thời gian thực hiện của vòng lặp {1} và cũng là độ phức tạp của giải thuật là

$$T(n) = \sum_{i=1}^{n-1} (n-i) = \frac{n(n-1)}{2} = O(n^2).$$

Chú ý: Trong trường hợp vòng lặp không xác định được số lần lặp thì chúng ta phải lấy số lần lặp trong trường hợp xấu nhất.

Ví dụ 1-8: Tìm kiếm tuần tự. Hàm tìm kiếm Search nhận vào một mảng a có n số nguyên và một số nguyên x , hàm sẽ trả về giá trị logic TRUE nếu tồn tại một phần tử $a[i] = x$, ngược lại hàm trả về FALSE.

Giải thuật tìm kiếm tuần tự là lần lượt so sánh x với các phần tử của mảng a , bắt đầu từ $a[1]$, nếu tồn tại $a[i] = x$ thì dừng và trả về TRUE, ngược lại nếu tất cả các phần tử của a đều khác x thì trả về FALSE.

```

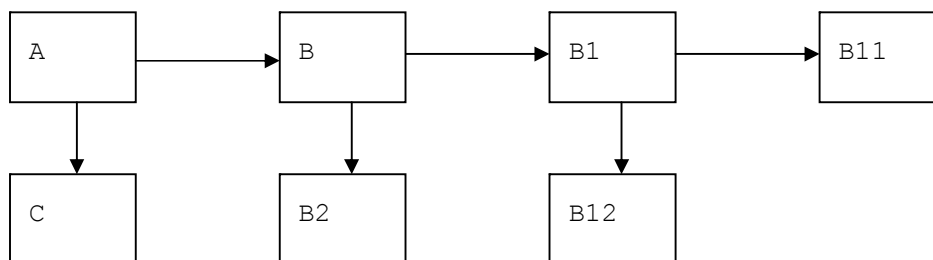
FUNCTION Search(a:ARRAY[1..n] OF Integer;x:Integer):Boolean;
VAR i:Integer; Found:Boolean;
BEGIN
{1}  i:=1;
{2}  Found:=FALSE;
{3}  WHILE (i<=n) AND (not Found) DO
{4}      IF A[i]=X THEN Found:=TRUE
          ELSE i:=i+1;
{5}  Search:=Found;
END;
```

Ta thấy các lệnh {1}, {2}, {3} và {5} nối tiếp nhau, do đó độ phức tạp của hàm Search chính là độ phức tạp lớn nhất trong 4 lệnh này. Dễ dàng thấy rằng ba lệnh {1}, {2} và {5} đều có độ phức tạp $O(1)$ do đó độ phức tạp của hàm Search chính là độ phức tạp của lệnh {3}. Lồng trong lệnh {3} là lệnh {4}. Lệnh {4} có độ phức tạp $O(1)$. Trong trường hợp xấu nhất (tất cả các phần tử của mảng a đều khác x) thì vòng lặp {3} thực hiện n lần, vậy ta có $T(n) = O(n)$.

1.5.4 Độ phức tạp của chương trình có gọi chương trình con không đệ quy

Nếu chúng ta có một chương trình với các chương trình con không đệ quy, để tính thời gian thực hiện của chương trình, trước hết chúng ta tính thời gian thực hiện của các chương trình con không gọi các chương trình con khác. Sau đó chúng ta tính thời gian thực hiện của các chương trình con chỉ gọi các chương trình con mà thời gian thực hiện của chúng đã được tính. Chúng ta tiếp tục quá trình đánh giá thời gian thực hiện của mỗi chương trình con sau khi thời gian thực hiện của tất cả các chương trình con mà nó gọi đã được đánh giá. Cuối cùng ta tính thời gian cho chương trình chính.

Giả sử ta có một hệ thống các chương trình gọi nhau theo sơ đồ sau:



Hình 1-1: Sơ đồ gọi thực hiện các chương trình con không đệ quy

Chương trình A gọi hai chương trình con là B và C, chương trình B gọi hai chương trình con là B1 và B2, chương trình B1 gọi hai chương trình con là B11 và B12.

Để tính thời gian thực hiện của A, ta tính theo các bước sau:

1. Tính thời gian thực hiện của C, B2, B11 và B12. Vì các chương trình con này không gọi chương trình con nào cả.
2. Tính thời gian thực hiện của B1. Vì B1 gọi B11 và B12 mà thời gian thực hiện của B11 và B12 đã được tính ở bước 1.
3. Tính thời gian thực hiện của B. Vì B gọi B1 và B2 mà thời gian thực hiện của B1 đã được tính ở bước 2 và thời gian thực hiện của B2 đã được tính ở bước 1.
4. Tính thời gian thực hiện của A. Vì A gọi B và C mà thời gian thực hiện của B đã được tính ở bước 3 và thời gian thực hiện của C đã được tính ở bước 1.

Ví dụ 1-9: Ta có thể viết lại chương trình sắp xếp bubble như sau: Trước hết chúng ta viết thủ tục Swap để thực hiện việc hoán đổi hai phần tử cho nhau, sau đó trong thủ tục Bubble, khi cần ta sẽ gọi đến thủ tục Swap này.

```
PROCEDURE Swap (VAR x, y: Integer);
VAR temp: Integer;
BEGIN
    temp := x;
    x     := y;
    y     := temp;
END;
PROCEDURE Bubble (VAR a: ARRAY[1..n] OF integer);
VAR i, j : Integer;
BEGIN
    {1} FOR i:=1 TO n-1 DO
    {2}     FOR j:=n DOWNT0 i+1 DO
    {3}         IF a[j-1]>a[j] THEN Swap(a[j-1], a[j]);
    END;
```

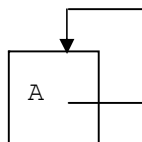
Trong cách viết trên, chương trình Bubble gọi chương trình con Swap, do đó để tính thời gian thực hiện của Bubble, trước hết ta cần tính thời gian thực hiện của Swap. Để thấy thời gian thực hiện của Swap là $O(1)$ vì nó chỉ bao gồm 3 lệnh gán.

Trong Bubble, lệnh {3} gọi Swap nên chỉ tốn $O(1)$, lệnh {2} thực hiện $n-i$ lần, mỗi lần tốn $O(1)$ nên tốn $O(n-i)$. Lệnh {1} thực hiện $n-1$ lần nên

$$T(n) = \sum_{i=1}^{n-1} (n-i) = \frac{n(n-1)}{2} = O(n^2).$$

1.6 PHÂN TÍCH CÁC CHƯƠNG TRÌNH ĐỆ QUY

Với các chương trình có gọi các chương trình con đệ quy, ta không thể áp dụng cách tính như vừa trình bày trong mục 1.5.4 bởi vì một chương trình đệ quy sẽ gọi chính bản thân nó. Có thể thấy hình ảnh chương trình đệ quy A như sau:



Hình 1-2: Sơ đồ chương trình con A đệ quy

Với phương pháp tính độ phức tạp đã trình bày trong mục 1.5.4 thì không thể thực hiện được. Bởi vì nếu theo phương pháp đó thì, để tính thời gian thực hiện của chương trình A, ta phải tính thời gian thực hiện của chương trình A và cái vòng lặp quần ấy không thể kết thúc được.

Với các chương trình đệ quy, trước hết ta cần thành lập các phương trình đệ quy, sau đó giải phương trình đệ quy, nghiệm của phương trình đệ quy sẽ là thời gian thực hiện của chương trình đệ quy.

1.6.1 Thành lập phương trình đệ quy

Phương trình đệ quy là một phương trình biểu diễn mối liên hệ giữa $T(n)$ và $T(k)$, trong đó $T(n)$ là thời gian thực hiện chương trình với kích thước dữ liệu nhập là n , $T(k)$ thời gian thực hiện chương trình với kích thước dữ liệu nhập là k , với $k < n$. Để thành lập được phương trình đệ quy, ta phải căn cứ vào chương trình đệ quy.

Thông thường một chương trình đệ quy để giải bài toán kích thước n , phải có ít nhất một trường hợp dừng ứng với một n cụ thể và lời gọi đệ quy để giải bài toán kích thước k ($k < n$).

Để thành lập phương trình đệ quy, ta gọi $T(n)$ là thời gian để giải bài toán kích thước n , ta có $T(k)$ là thời gian để giải bài toán kích thước k . Khi đệ quy dừng, ta phải xem xét khi đó chương trình làm gì và tốn hết bao nhiêu thời gian, chẳng hạn thời gian này là $c(n)$. Khi đệ quy chưa dừng thì phải xét xem có bao nhiêu lời gọi đệ quy với kích thước k ta sẽ có bấy nhiêu $T(k)$. Ngoài ra ta còn phải xem xét đến thời gian để phân chia bài toán và tổng hợp các lời giải, chẳng hạn thời gian này là $d(n)$.

Dạng tổng quát của một phương trình đệ quy sẽ là:

$$T(n) = \begin{cases} C(n) \\ F(T(k)) + d(n) \end{cases}$$

Trong đó $C(n)$ là thời gian thực hiện chương trình ứng với trường hợp đệ quy dừng. $F(T(k))$ là một đa thức của các $T(k)$. $d(n)$ là thời gian để phân chia bài toán và tổng hợp các kết quả.

Ví dụ 1-10: Xét hàm tính giai thừa viết bằng giải thuật đệ quy như sau:

```
FUNCTION Giai_thua(n:Integer): Integer;
BEGIN
    IF n=0 then Giai_thua :=1
    ELSE Giai_thua := n* Giai_thua(n-1);
END;
```

Gọi $T(n)$ là thời gian thực hiện việc tính n giai thừa, thì $T(n-1)$ là thời gian thực hiện việc tính $n-1$ giai thừa. Trong trường hợp $n = 0$ thì chương trình chỉ thực hiện một lệnh gán $Giai_thua:=1$, nên tốn $O(1)$, do đó ta có $T(0) = C_1$. Trong trường hợp $n > 0$ chương trình phải gọi đệ quy $Giai_thua(n-1)$, việc gọi đệ quy này tốn $T(n-1)$, sau khi có kết quả của việc gọi đệ quy, chương trình phải nhân kết quả đó với n và gán cho $Giai_thua$. Thời gian để thực hiện phép nhân và phép gán là một hằng C_2 . Vậy ta có

$$T(n) = \begin{cases} C_1 & \text{nếu } n = 0 \\ T(n-1) + C_2 & \text{nếu } n > 0 \end{cases}$$

Đây là phương trình đệ quy để tính thời gian thực hiện của chương trình đệ quy Giải_thua.

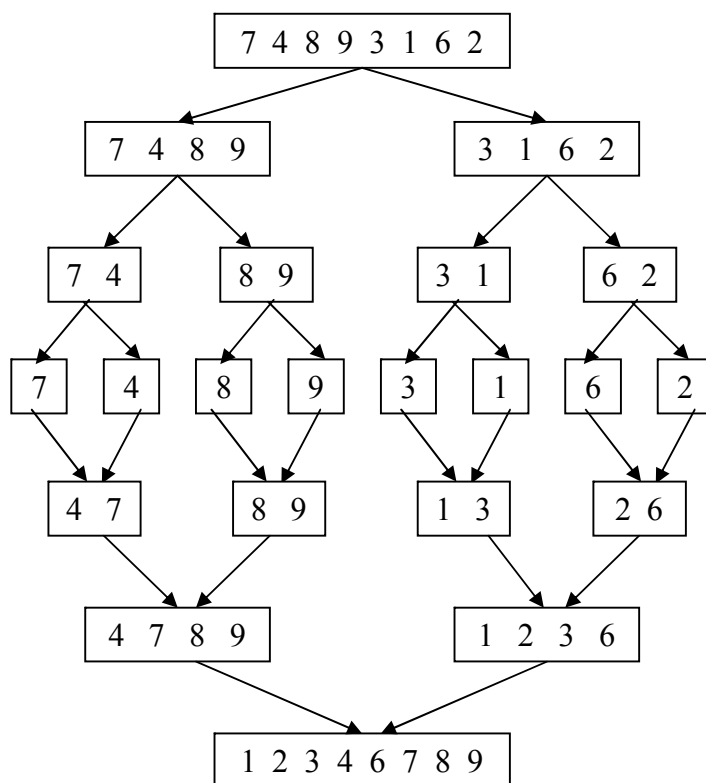
Ví dụ 1-11: Chúng ta xét thủ tục MergeSort một cách phác thảo như sau:

```

FUNCTION MergeSort (L:List; n:Integer):List;
VAR L1,L2:List;
BEGIN
  IF n=1 THEN RETURN(L)
  ELSE BEGIN
    Chia đôi L thành L1 và L2, với độ dài n/2;
    RETURN (Merge (MergeSort (L1,n/2), MergeSort (L2,n/2)));
  END;
END;

```

Chẳng hạn để sắp xếp danh sách L gồm 8 phần tử 7, 4, 8, 9, 3, 1, 6, 2 ta có mô hình minh họa của MergeSort như sau:



Hình 1-3: Minh họa sắp xếp trộn

Hàm MergeSort nhận một danh sách có độ dài n và trả về một danh sách đã được sắp xếp. Thủ tục Merge nhận hai danh sách đã được sắp L1 và L2 mỗi danh sách có độ dài $\frac{n}{2}$, trộn chúng lại với nhau để được một danh sách gồm n phần tử có thứ tự.

Giải thuật chi tiết của Merge ta sẽ bàn sau, chúng ta chỉ để ý rằng thời gian để Merge các danh sách có độ dài $\frac{n}{2}$ là $O(n)$.

Gọi $T(n)$ là thời gian thực hiện MergeSort một danh sách n phần tử thì $T(\frac{n}{2})$ là thời gian thực hiện MergeSort một danh sách $\frac{n}{2}$ phần tử.

Khi L có độ dài 1 ($n = 1$) thì chương trình chỉ làm một việc duy nhất là $\text{return}(L)$, việc này tốn $O(1) = C_1$ thời gian. Trong trường hợp $n > 1$, chương trình phải thực hiện gọi đệ quy MergeSort hai lần cho $L1$ và $L2$ với độ dài $\frac{n}{2}$ do đó thời gian để gọi hai lần đệ quy này là $2T(\frac{n}{2})$. Ngoài ra còn phải tốn thời gian cho việc chia danh sách L thành hai nửa bằng nhau và trộn hai danh sách kết quả (Merge). Người ta xác định được thời gian để chia danh sách và Merge là $O(n) = C_2 n$. Vậy ta có phương trình đệ quy như sau:

$$T(n) = \begin{cases} C_1 & \text{nếu } n = 1 \\ 2T(\frac{n}{2}) + C_2 n & \text{nếu } n > 1 \end{cases}$$

1.6.2 Giải phương trình đệ quy

Có ba phương pháp giải phương trình đệ quy:

- 1.- Phương pháp truy hồi
- 2.- Phương pháp đoán nghiệm.
- 3.- Lời giải tổng quát của một lớp các phương trình đệ quy.

1.6.2.1 Phương pháp truy hồi

Dùng đệ quy để thay thế bất kỳ $T(m)$ với $m < n$ vào phía phải của phương trình cho đến khi tất cả $T(m)$ với $m > 1$ được thay thế bởi biểu thức của các $T(1)$ hoặc $T(0)$. Vì $T(1)$ và $T(0)$ luôn là hằng số nên chúng ta có công thức của $T(n)$ chứa các số hạng chỉ liên quan đến n và các hằng số. Từ công thức đó ta suy ra $T(n)$.

Ví dụ 1-12: Giải phương trình $T(n) = \begin{cases} C_1 & \text{nếu } n = 0 \\ T(n-1) + C_2 & \text{nếu } n > 0 \end{cases}$

Ta có $T(n) = T(n-1) + C_2$

$$T(n) = [T(n-2) + C_2] + C_2 = T(n-2) + 2C_2$$

$$T(n) = [T(n-3) + C_2] + 2C_2 = T(n-3) + 3C_2$$

.....

$$T(n) = T(n-i) + iC_2$$

Quá trình trên kết thúc khi $n - i = 0$ hay $i = n$. Khi đó ta có

$$T(n) = T(0) + nC_2 = C_1 + nC_2 = O(n)$$

Ví dụ 1-13: Giải phương trình $T(n) = \begin{cases} C_1 & \text{nếu } n = 1 \\ 2T(\frac{n}{2}) + C_2 n & \text{nếu } n > 1 \end{cases}$

Ta có $T(n) = 2T(\frac{n}{2}) + 2C_2 n$

$$T(n) = 2[2T(\frac{n}{4}) + C_2 \frac{n}{2}] + 2C_2 n = 4T(\frac{n}{4}) + 2C_2 n$$

$$T(n) = 4[2T(\frac{n}{8}) + C_2 \frac{n}{4}] + 2C_2 n = 8T(\frac{n}{8}) + 3C_2 n$$

.....

$$T(n) = 2^i T(\frac{n}{2^i}) + iC_2 n$$

Quá trình suy rộng sẽ kết thúc khi $\frac{n}{2^i} = 1$ hay $2^i = n$ và do đó $i = \log n$. Khi đó ta có:

$$T(n) = nT(1) + \log n C_2 n = C_1 n + C_2 n \log n = O(n \log n).$$

1.6.2.2 Phương pháp đoán nghiệm

Ta đoán một nghiệm $f(n)$ và dùng chứng minh quy nạp để chứng tỏ rằng $T(n) \leq f(n)$ với mọi n .

Thông thường $f(n)$ là một trong các hàm quen thuộc như $\log n$, n , $n \log n$, n^2 , n^3 , 2^n , $n!$, n^n .

Đôi khi chúng ta chỉ đoán dạng của $f(n)$ trong đó có một vài tham số chưa xác định (chẳng hạn $f(n) = an^2$ với a chưa xác định) và trong quá trình chứng minh quy nạp ta sẽ suy diễn ra giá trị thích hợp của các tham số.

Ví dụ 1-12: Giải phương trình đệ quy $T(n) = \begin{cases} C_1 & \text{nếu } n = 1 \\ 2T(\frac{n}{2}) + C_2 n & \text{nếu } n > 1 \end{cases}$

Giả sử chúng ta đoán $f(n) = an \log n$. Với $n = 1$ ta thấy rằng cách đoán như vậy không được bởi vì $an \log n$ có giá trị 0 không phụ thuộc vào giá trị của a . Vì thế ta thử tiếp theo $f(n) = an \log n + b$.

Với $n = 1$ ta có, $T(1) = C_1$ và $f(1) = b$, muốn $T(1) \leq f(1)$ thì $b \geq C_1$ (*)

Giả sử rằng $T(k) \leq f(k)$, tức là $T(k) \leq ak \log k + b$ với mọi $k < n$ (giả thiết quy nạp). Ta phải chứng minh $T(n) \leq an \log n + b$ với mọi n .

Giả sử $n \geq 2$, từ phương trình đã cho ta có $T(n) = 2T(\frac{n}{2}) + C_2 n$

Áp dụng giả thiết quy nạp với $k = \frac{n}{2} < n$ ta có:

$$T(n) = 2T(\frac{n}{2}) + C_2 n \leq 2[a \frac{n}{2} \log \frac{n}{2} + b] + C_2 n$$

$$T(n) \leq (an \log n - an + 2b) + C_2 n$$

$$T(n) \leq (an \log n + b) + [b + (C_2 - a)n]. \text{ Nếu lấy } a \geq C_2 + b \text{ (**)} \text{ ta được}$$

$$T(n) \leq (an \log n + b) + [b + (C_2 - C_2 - b)n]$$

$$T(n) \leq (an \log n + b) + (1-n)b$$

$$T(n) \leq an \log n + b = f(n). \text{ (do } b > 0 \text{ và } 1-n < 0)$$

Nếu ta lấy a và b sao cho cả (*) và (**) đều thỏa mãn thì $T(n) \leq an \log n + b$ với mọi n .

Ta phải giải hệ $\begin{cases} b \geq C_1 \\ a \geq C_2 + b \end{cases}$ Để đơn giản, ta giải hệ $\begin{cases} b = C_1 \\ a = C_2 + b \end{cases}$

Dễ dàng ta có $b = C_1$ và $a = C_1 + C_2$ ta được $T(n) \leq (C_1 + C_2)n \log n + C_1$ với mọi n .

Hay nói cách khác $T(n)$ là $O(n \log n)$.

1.6.2.3 Lờ giải tổng quát cho một lớp các phương trình đệ quy

Khi thiết kế các giải thuật, người ta thường vận dụng phương pháp chia để trị mà ta sẽ bàn chi tiết hơn trong chương 3. Ở đây chỉ trình bày tóm tắt phương pháp như sau:

Để giải một bài toán kích thước n , ta chia bài toán đã cho thành a bài toán con, mỗi bài toán con có kích thước $\frac{n}{b}$. Giải các bài toán con này và tổng hợp kết quả lại để được kết quả của bài toán đã cho. Với các bài toán con chúng ta cũng sẽ áp dụng phương pháp đó để tiếp tục chia nhỏ ra nữa cho đến các bài toán con kích thước 1. Kĩ thuật này sẽ dẫn chúng ta đến một giải thuật đệ quy.

Giả thiết rằng mỗi bài toán con kích thước 1 lấy một đơn vị thời gian và thời gian để chia bài toán kích thước n thành các bài toán con kích thước $\frac{n}{b}$ và tổng hợp kết quả từ các bài toán con để được lời giải của bài toán ban đầu là $d(n)$. (Chẳng hạn đối với ví dụ MergeSort, chúng ta có $a = b = 2$, và $d(n) = C_2 n$. Xem C_1 là một đơn vị).

Tất cả các giải thuật đệ quy như trên đều có thể thành lập một phương trình đệ quy tổng quát, chung cho lớp các bài toán ấy.

Nếu gọi $T(n)$ là thời gian để giải bài toán kích thước n thì $T(\frac{n}{b})$ là thời gian để giải bài toán con kích thước $\frac{n}{b}$. Khi $n = 1$ theo giả thiết trên thì thời gian giải bài toán kích thước 1 là 1 đơn vị, tức là $T(1) = 1$. Khi n lớn hơn 1, ta phải giải đệ quy a bài toán con kích thước $\frac{n}{b}$, mỗi bài toán con tốn $T(\frac{n}{b})$ nên thời gian cho a lời giải đệ quy này là $aT(\frac{n}{b})$. Ngoài ra ta còn phải tốn thời gian để phân chia bài toán và tổng hợp các kết quả, thời gian này theo giả thiết trên là $d(n)$. Vậy ta có phương trình đệ quy:

$$T(n) = \begin{cases} 1 & \text{neu } n = 1 \\ aT(\frac{n}{b}) + d(n) & \text{neu } n > 1 \end{cases} \quad (I.1)$$

Ta sử dụng phương pháp truy hồi để giải phương trình này. Khi $n > 1$ ta có

$$T(n) = aT(\frac{n}{b}) + d(n)$$

$$T(n) = a[aT(\frac{n}{b^2}) + d(\frac{n}{b})] + d(n) = a^2T(\frac{n}{b^2}) + ad(\frac{n}{b}) + d(n)$$

$$T(n) = a^2[aT(\frac{n}{b^3}) + d(\frac{n}{b^2})] + ad(\frac{n}{b}) + d(n) = a^3T(\frac{n}{b^3}) + a^2d(\frac{n}{b^2}) + ad(\frac{n}{b}) + d(n)$$

=

$$= a^i T(\frac{n}{b^i}) + \sum_{j=0}^{i-1} a^j d(\frac{n}{b^j})$$

Giả sử $n = b^k$, quá trình suy rộng trên sẽ kết thúc khi $i = k$.

Khi đó ta được $T(\frac{n}{b^k}) = T(1) = 1$. Thay vào trên ta có:

$$T(n) = a^k + \sum_{j=0}^{k-1} a^j d(b^{k-j}) \quad (I.2)$$

1.6.2.3.1 Hàm tiến triển, nghiệm thuần nhất và nghiệm riêng

Trong phương trình đệ quy (I.1) hàm thời gian $d(n)$ được gọi là **hàm tiến triển** (driving function)

Trong công thức (I.2), $a^k = n^{\log_b a}$ được gọi là **ng nghiệm thuần nhất** (homogeneous solutions).

Ng nghiệm thuần nhất là ng nghiệm chính xác khi $d(n) = 0$ với mọi n . Nói một cách khác, ng nghiệm thuần nhất biểu diễn thời gian để giải tất cả các bài toán con.

Trong công thức (I.2), $\sum_{j=0}^{k-1} a^j d(b^{k-j})$ được gọi là **ng nghiệm riêng** (particular solutions).

Ng nghiệm riêng biểu diễn thời gian phải tốn để tạo ra các bài toán con và tổng hợp các kết quả của chúng. Nhìn vào công thức ta thấy ng nghiệm riêng phụ thuộc vào hàm tiến triển, số lượng và kích thước các bài toán con.

Khi tìm ng nghiệm của phương trình (I.1), chúng ta phải tìm ng nghiệm riêng và so sánh với ng nghiệm thuần nhất. Nếu ng nghiệm nào lớn hơn, ta lấy ng nghiệm đó làm ng nghiệm của phương trình (I.1).

Việc xác định ng nghiệm riêng không đơn giản chút nào, tuy vậy, chúng ta cũng tìm được một lớp các hàm tiến triển có thể dễ dàng xác định ng nghiệm riêng.

1.6.2.3.2 Hàm nhân

Một hàm $f(n)$ được gọi là hàm nhân (multiplicative function) nếu $f(m.n) = f(m).f(n)$ với mọi số nguyên dương m và n .

Ví dụ 1-13: Hàm $f(n) = n^k$ là một hàm nhân, vì $f(m.n) = (m.n)^k = m^k.n^k = f(m).f(n)$

Tính nghiệm của phương trình tổng quát trong trường hợp $d(n)$ là hàm nhân:

Nếu $d(n)$ trong (I.1) là một hàm nhân thì theo tính chất của hàm nhân ta có

$d(b^{k-j}) = [d(b)]^{k-j}$ và nghiệm riêng của (I.2) là

$$\sum_{j=0}^{k-1} a^j d(b^{k-j}) = \sum_{j=0}^{k-1} a^j [d(b)]^{k-j} = [d(b)]^k \sum_{j=0}^{k-1} \left[\frac{a}{d(b)} \right]^j = [d(b)]^k \frac{\left[\frac{a}{d(b)} \right]^k - 1}{\frac{a}{d(b)} - 1}$$

$$\text{Hay nghiệm riêng} = \frac{a^k - [d(b)]^k}{\frac{a}{d(b)} - 1} \quad (\text{I.3})$$

Xét ba trường hợp sau:

1.- **Trường hợp 1:** $a > d(b)$ thì trong công thức (I.3) ta có $a^k > [d(b)]^k$, theo quy tắc lấy độ phức tạp ta có nghiệm riêng là $O(a^k) = O(n^{\log_b a})$. Như vậy nghiệm riêng và nghiệm thuần nhất bằng nhau do đó **$T(n)$ là $O(n^{\log_b a})$** .

Trong trường hợp này ta thấy thời gian thực hiện chỉ phụ thuộc vào a, b mà không phụ thuộc vào hàm tiến triển $d(n)$. Vì vậy **để cải tiến giải thuật ta cần giảm a hoặc tăng b** .

2.- **Trường hợp 2:** $a < d(b)$ thì trong công thức (I.3) ta có $[d(b)]^k > a^k$, theo quy tắc lấy độ phức tạp ta có nghiệm riêng là $O([d(b)]^k) = O(n^{\log_b d(b)})$. Trong trường hợp này nghiệm riêng lớn hơn nghiệm thuần nhất nên **$T(n)$ là $O(n^{\log_b d(b)})$** .

Để cải tiến giải thuật chúng ta cần giảm $d(b)$ hoặc tăng b .

Trường hợp đặc biệt quan trọng khi $d(n) = n$. Khi đó $d(b) = b$ và $\log_b b = 1$. Vì thế nghiệm riêng là $O(n)$ và do vậy **$T(n)$ là $O(n)$** .

3.- **Trường hợp 3:** $a = d(b)$ thì công thức (I.3) không xác định nên ta phải tính trực tiếp nghiệm riêng:

$$\text{Nghiệm riêng} = [d(b)]^k \sum_{j=0}^{k-1} \left[\frac{a}{d(b)} \right]^j = a^k \sum_{j=0}^{k-1} 1 = a^k k \quad (\text{do } a = d(b))$$

Do $n = b^k$ nên $k = \log_b n$ và $a^k = n^{\log_b a}$. Vậy nghiệm riêng là $n^{\log_b a} \log_b n$ và nghiệm này lớn gấp $\log_b n$ lần nghiệm thuần nhất. Do đó **$T(n)$ là $O(n^{\log_b a} \log_b n)$** .

Chú ý khi giải một phương trình đệ quy cụ thể, ta phải xem phương trình đó có thuộc dạng phương trình tổng quát hay không. Nếu có thì phải xét xem hàm tiến triển có phải là hàm nhân không. Nếu có thì ta xác định $a, d(b)$ và dựa vào sự so sánh giữa a và $d(b)$ mà vận dụng một trong ba trường hợp nói trên.

Ví dụ 1-14: Giải các phương trình đệ quy sau với $T(1) = 1$ và

$$1/- T(n) = 4T\left(\frac{n}{2}\right) + n$$

$$2/- T(n) = 4T\left(\frac{n}{2}\right) + n^2$$

$$3/- T(n) = 4T\left(\frac{n}{2}\right) + n^3$$

Các phương trình đã cho đều có dạng phương trình tổng quát, các hàm tiến triển $d(n)$ đều là các hàm nhân và $a = 4$, $b = 2$.

Với phương trình thứ nhất, ta có $d(n) = n \Rightarrow d(b) = b = 2 < a$, áp dụng trường hợp 1 ta có $T(n) = O(n^{\log_b a}) = O(n^{\log_2 4}) = O(n^2)$.

Với phương trình thứ hai, $d(n) = n^2 \Rightarrow d(b) = b^2 = 4 = a$, áp dụng trường hợp 3 ta có $T(n) = O(n^{\log_b a} \log_b n) = O(n^{\log_2 4} \log n) = O(n^2 \log n)$.

Với phương trình thứ 3, ta có $d(n) = n^3 \Rightarrow d(b) = b^3 = 8 > a$, áp dụng trường hợp 2, ta có $T(n) = O(n^{\log_b d(b)}) = O(n^{\log_2 8}) = O(n^3)$.

1.6.2.3.3 Các hàm tiến triển khác

Trong trường hợp hàm tiến triển không phải là một hàm nhân thì chúng ta không thể áp dụng các công thức ứng với ba trường hợp nói trên mà chúng ta phải tính trực tiếp nghiệm riêng, sau đó so sánh với nghiệm thuần nhất để lấy nghiệm lớn nhất trong hai nghiệm đó làm nghiệm của phương trình.

Ví dụ 1-15: Giải phương trình đệ quy sau :

$$T(1) = 1$$

$$T(n) = 2T\left(\frac{n}{2}\right) + n \log n$$

Phương trình đã cho thuộc dạng phương trình tổng quát nhưng $d(n) = n \log n$ không phải là một hàm nhân.

Ta có nghiệm thuần nhất $= n^{\log_b a} = n^{\log_2 2} = n$

Do $d(n) = n \log n$ không phải là hàm nhân nên ta phải tính nghiệm riêng bằng cách xét trực tiếp

$$\text{Nghiệm riêng} = \sum_{j=0}^{k-1} a^j d(b^{k-j}) = \sum_{j=0}^{k-1} 2^j 2^{k-j} \log 2^{k-j} = 2^k \sum_{j=0}^{k-1} (k-j) = 2^k \frac{k(k+1)}{2} = O(2^k k^2)$$

Theo giả thiết trong phương trình tổng quát thì $n = b^k$ nên $k = \log_b n$, ở đây do $b = 2$ nên $2^k = n$ và $k = \log n$, chúng ta có nghiệm riêng là $O(n \log^2 n)$, nghiệm này lớn hơn nghiệm thuần nhất do đó $T(n) = O(n \log^2 n)$.

1.7 TỔNG KẾT CHƯƠNG 1

Trong chương này, chúng ta cần phải nắm vững các ý sau:

- 1.- Sự phân tích, đánh giá giải thuật là cần thiết để lựa chọn giải thuật tốt, hoặc để cải tiến giải thuật.
- 2.- Sử dụng khái niệm độ phức tạp và ký hiệu O lớn để đánh giá giải thuật.
- 3.- Đối với các chương trình không gọi chương trình con, thì dùng quy tắc cộng, quy tắc nhân và quy tắc chung để phân tích, tính độ phức tạp.
- 4.- Đối với các chương trình gọi chương trình con, thì tính độ phức tạp theo nguyên tắc “từ trong ra”.
- 5.- Đối với các chương trình đệ quy thì trước hết phải thành lập phương trình đệ quy, sau đó giải phương trình đệ quy, nghiệm của phương trình đệ quy chính là độ phức tạp của giải thuật.
- 6.- Khi giải một phương trình đệ quy không thuộc dạng phương trình tổng quát thì sử dụng phương pháp truy hồi hoặc phương pháp đoán nghiệm.
- 7.- Khi giải một phương trình đệ quy thuộc dạng phương trình tổng quát, nếu hàm tiến triển $d(n)$ là một hàm nhân thì vận dụng công thức nghiệm của một trong ba trường hợp để xác định nghiệm, còn nếu $d(n)$ không phải là hàm nhân thì phải tính trực tiếp nghiệm riêng và so sánh với nghiệm thuần nhất để chọn nghiệm.

BÀI TẬP CHƯƠNG 1

Bài 1: Tính thời gian thực hiện của các đoạn chương trình sau:

a) Tính tổng của các số

```
{1} Sum := 0;
{2} for i:=1 to n do begin
{3}     readln(x);
{4}     Sum := Sum + x;
end;
```

b) Tính tích hai ma trận vuông cấp n $C = A*B$:

```
{1} for i := 1 to n do
{2}   for j := 1 to n do begin
{3}     c[i,j] := 0;
{4}     for k := 1 to n do
{5}       c[i,j] := c[i,j] + a[i,k] * b[k,j];
end;
```

Bài 2: Giải các phương trình đệ quy sau với $T(1) = 1$ và

- a) $T(n) = 3T(n/2) + n$
- b) $T(n) = 3T(n/2) + n^2$
- c) $T(n) = 8T(n/2) + n^3$

Bài 3: Giải các phương trình đệ quy sau với $T(1) = 1$ và

- a) $T(n) = 4T(n/3) + n$
- b) $T(n) = 4T(n/3) + n^2$

c) $T(n) = 9T(n/3) + n^2$

Bài 4: Giải các phương trình đệ quy sau với $T(1) = 1$ và

a) $T(n) = T(n/2) + 1$

b) $T(n) = 2T(n/2) + \log n$

c) $T(n) = 2T(n/2) + n$

d) $T(n) = 2T(n/2) + n^2$

Bài 5: Giải các phương trình đệ quy sau bằng phương pháp đoán nghiệm:

a) $T(1) = 2$ và $T(n) = 2T(n-1) + 1$ với $n > 1$

b) $T(1) = 1$ và $T(n) = 2T(n-1) + n$ với $n > 1$

Bài 6: Cho một mảng n số nguyên được sắp thứ tự tăng. Viết hàm tìm một số nguyên trong mảng đó theo phương pháp **tìm kiếm nhị phân**, nếu tìm thấy thì trả về TRUE, ngược lại trả về FALSE.

Sử dụng hai kỹ thuật là đệ quy và vòng lặp. Với mỗi kỹ thuật hãy viết một hàm tìm và tính thời gian thực hiện của hàm đó.

Bài 7: Tính thời gian thực hiện của giải thuật đệ quy giải bài toán Tháp Hà nội với n tầng?

Bài 8: Xét công thức truy toán để tính số tổ hợp chập k của n như sau:

$$C_n^k = \begin{cases} 1 & \text{nếu } k = 0 \text{ hoặc } k = n \\ C_{n-1}^{k-1} + C_{n-1}^k & \text{nếu } 0 < k < n \end{cases}$$

a) Viết một hàm đệ quy để tính số tổ hợp chập k của n .

b) Tính thời gian thực hiện của giải thuật nói trên.

CHƯƠNG 2: SẮP XẾP

2.1 TỔNG QUAN

2.1.1 Mục tiêu

Chương này sẽ trình bày một số phương pháp sắp xếp. Với mỗi phương pháp cần nắm vững các phần sau:

- Giải thuật sắp xếp.
- Minh họa việc sắp xếp theo giải thuật.
- Chương trình sắp xếp.
- Đánh giá giải thuật.

2.1.2 Kiến thức cơ bản cần thiết

Các kiến thức cơ bản cần thiết để học chương này bao gồm:

- Cấu trúc dữ liệu kiểu mẫu tin (record) và kiểu mảng (array) của các mẫu tin.
- Kiểu dữ liệu trừu tượng danh sách và thủ tục xen một phần tử vào danh sách (insert).
- Kỹ thuật lập trình và lập trình đệ quy.

2.1.3 Tài liệu tham khảo

A.V. Aho, J.E. Hopcroft, J.D. Ullman. *Data Structures and Algorithms*. Addison-Wesley. 1983. (Chapter 8).

Jeffrey H Kingston; *Algorithms and Data Structures*; Addison-Wesley; 1998. (Chapter 9).

Đinh Mạnh Tường. *Cấu trúc dữ liệu & Thuật toán*. Nhà xuất bản khoa học và kỹ thuật. Hà nội-2001. (Chương 9).

Đỗ Xuân Lôi. *Cấu trúc dữ liệu & Giải thuật*. 1995. (Chương 9).

2.1.4 Nội dung cốt lõi

Trong chương này chúng ta sẽ nghiên cứu các vấn đề sau:

- Bài toán sắp xếp.
- Một số giải thuật sắp xếp đơn giản.
- QuickSort
- HeapSort
- BinSort

2.2 BÀI TOÁN SẮP XẾP

2.2.1 Tầm quan trọng của bài toán sắp xếp

Sắp xếp một danh sách các đối tượng theo một thứ tự nào đó là một bài toán thường được vận dụng trong các ứng dụng tin học. Ví dụ ta cần sắp xếp danh sách thí sinh theo tên với thứ tự Alphabet, hoặc sắp xếp danh sách sinh viên theo điểm trung bình với thứ tự từ cao đến thấp. Một ví dụ khác là khi cần tìm kiếm một đối tượng trong một danh sách các đối tượng bằng giải thuật tìm kiếm nhị phân thì danh sách các đối tượng này phải được sắp xếp trước đó.

Tóm lại sắp xếp là một yêu cầu không thể thiếu trong khi thiết kế các phần mềm. Do đó việc nghiên cứu các phương pháp sắp xếp là rất cần thiết để vận dụng trong khi lập trình.

2.2.2 Sắp xếp trong và sắp xếp ngoài

Sắp xếp trong là sự sắp xếp dữ liệu được tổ chức trong bộ nhớ trong của máy tính, ở đó ta có thể sử dụng khả năng truy nhập ngẫu nhiên của bộ nhớ và do vậy sự thực hiện rất nhanh.

Sắp xếp ngoài là sự sắp xếp được sử dụng khi số lượng đối tượng cần sắp xếp lớn không thể lưu trữ trong bộ nhớ trong mà phải lưu trữ trên **bộ nhớ ngoài**. Cụ thể là ta sẽ sắp xếp dữ liệu được lưu trữ trong các tập tin.

Chương này tập trung giải quyết vấn đề sắp xếp trong còn sắp xếp ngoài sẽ được nghiên cứu trong chương IV.

2.2.3 Tổ chức dữ liệu và ngôn ngữ cài đặt

Các đối tượng cần được sắp xếp là các mẫu tin gồm một hoặc nhiều trường. Một trong các trường được gọi là khóa (key), kiểu của nó là một kiểu có quan hệ thứ tự (như các kiểu số nguyên, số thực, chuỗi ký tự...).

Danh sách các đối tượng cần sắp xếp sẽ là một mảng của các mẫu tin vừa nói ở trên. Mục đích của việc sắp xếp là tổ chức lại các mẫu tin sao cho các khóa của chúng được sắp thứ tự tương ứng với quy luật sắp xếp.

Để trình bày các ví dụ minh họa chúng ta sẽ dùng PASCAL làm ngôn ngữ thể hiện và sử dụng khai báo sau:

```
CONST N = 10;
TYPE
    KeyType = integer;
    OtherType = real;

    RecordType = Record
        Key : KeyType;
        OtherFields : OtherType;
    end;
VAR
    a : array[1..N] of RecordType;
```

```
PROCEDURE Swap(var x,y:RecordType);  
VAR temp : RecordType;  
BEGIN  
    temp := x;  
    x := y;  
    y := temp;  
END;
```

Cần thấy rằng thủ tục Swap lấy $O(1)$ thời gian vì chỉ thực hiện 3 lệnh gán nối tiếp nhau.

2.3 CÁC PHƯƠNG PHÁP SẮP XẾP ĐƠN GIẢN

Các giải thuật đơn giản thường lấy $O(n^2)$ thời gian để sắp xếp n đối tượng và các giải thuật này thường chỉ dùng để sắp các danh sách có ít đối tượng.

Với mỗi giải thuật chúng ta sẽ nghiên cứu các phần: giải thuật, ví dụ, chương trình và phân tích đánh giá.

2.3.1 Sắp xếp chọn (Selection Sort)

2.3.1.1 Giải thuật

Đây là phương pháp sắp xếp đơn giản nhất được tiến hành như sau:

- Đầu tiên chọn phần tử có khóa nhỏ nhất trong n phần tử từ $a[1]$ đến $a[n]$ và hoán vị nó với phần tử $a[1]$.
- Chọn phần tử có khóa nhỏ nhất trong $n-1$ phần tử từ $a[2]$ đến $a[n]$ và hoán vị nó với $a[2]$.
- Tổng quát ở bước thứ i , chọn phần tử có khóa nhỏ nhất trong $n-i+1$ phần tử từ $a[i]$ đến $a[n]$ và hoán vị nó với $a[i]$.
- Sau $n-1$ bước này thì mảng đã được sắp xếp.

Phương pháp này được gọi là phương pháp chọn bởi vì nó lặp lại quá trình chọn phần tử nhỏ nhất trong số các phần tử chưa được sắp.

Ví dụ 2-1: Sắp xếp mảng gồm 10 mẫu tin có khóa là các số nguyên: 5, 6, 2, 2, 10, 12, 9, 10, 9 và 3

Bước 1: Ta chọn được phần tử có khóa nhỏ nhất (bằng 2) trong các phần tử từ $a[1]$ đến $a[10]$ là $a[3]$, hoán đổi $a[1]$ và $a[3]$ cho nhau. Sau bước này thì $a[1]$ có khóa nhỏ nhất là 2.

Bước 2: Ta chọn được phần tử có khóa nhỏ nhất (bằng 2) trong các phần tử từ $a[2]$ đến $a[10]$ là $a[4]$, hoán đổi $a[2]$ và $a[4]$ cho nhau.

Tiếp tục quá trình này và sau 9 bước thì kết thúc.

Bảng sau ghi lại các giá trị khóa tương ứng với từng bước.

Khóa Bước	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]	a[8]	a[9]	a[10]
Ban đầu	5	6	2	2	10	12	9	10	9	3
Bước 1	2	6	5	2	10	12	9	10	9	3
Bước 2		2	5	6	10	12	9	10	9	3
Bước 3			3	6	10	12	9	10	9	5
Bước 4				5	10	12	9	10	9	6
Bước 5					6	12	9	10	9	10
Bước 6						9	12	10	9	10
Bước 7							9	10	12	10
Bước 8								10	12	10
Bước 9									10	12
Kết quả	2	2	3	5	6	9	9	10	10	12

Hình 2-1: Sắp xếp chọn

2.3.1.2 Chương trình:

```

PROCEDURE SelectionSort;
VAR
    i, j, LowIndex: integer;
    LowKey: KeyType;
BEGIN
    {1} FOR i := 1 TO n-1 DO BEGIN
    {2}     LowIndex := i;
    {3}     LowKey := a[i].key;
    {4}     FOR j := i+1 TO n DO
    {5}         IF a[j].key < LowKey THEN
    {6}             BEGIN
    {7}                 LowKey := a[j].key;
    {8}                 LowIndex := j;
    {9}             END;
    {10}    Swap(a[i], a[LowIndex]);
    END;
END;

```

2.3.1.3 Đánh giá: Phương pháp sắp xếp chọn lấy $O(n^2)$ để sắp xếp n phần tử.

Trước hết ta có thủ tục Swap lấy một hằng thời gian như đã nói ở mục 2.2.3.

Các lệnh {2}, {3} đều lấy $O(1)$ thời gian. Vòng lặp for {4} – {7} thực hiện $n-i$ lần, vì j chạy từ $i+1$ đến n , mỗi lần lấy $O(1)$, nên lấy $O(n-i)$ thời gian. Do đó thời gian tổng cộng là:

$$T(n) = \sum_{i=1}^{n-1} (n-i) = \frac{n(n-1)}{2} \text{ tức là } O(n^2).$$

2.3.2 Sắp xếp xen (Insertion Sort)**2.3.2.1 Giải thuật**

Trước hết ta xem phần tử $a[1]$ là một dãy đã có thứ tự.

- Bước 1, xen phần tử $a[2]$ vào danh sách đã có thứ tự $a[1]$ sao cho $a[1], a[2]$ là một danh sách có thứ tự.
- Bước 2, xen phần tử $a[3]$ vào danh sách đã có thứ tự $a[1], a[2]$ sao cho $a[1], a[2], a[3]$ là một danh sách có thứ tự.
- Tổng quát, bước i , xen phần tử $a[i+1]$ vào danh sách đã có thứ tự $a[1], a[2], \dots, a[i]$ sao cho $a[1], a[2], \dots, a[i+1]$ là một danh sách có thứ tự.
- Phần tử đang xét $a[j]$ sẽ được xen vào vị trí thích hợp trong danh sách các phần tử đã được sắp trước đó $a[1], a[2], \dots, a[j-1]$ bằng cách so sánh khoá của $a[j]$ với khoá của $a[j-1]$ đứng ngay trước nó. Nếu khoá của $a[j]$ nhỏ hơn khoá của $a[j-1]$ thì hoán đổi $a[j-1]$ và $a[j]$ cho nhau và tiếp tục so sánh khoá của $a[j-1]$ (lúc này $a[j-1]$ chứa nội dung của $a[j]$) với khoá của $a[j-2]$ đứng ngay trước nó...

Ví dụ 2-2: Sắp xếp mảng gồm 10 mẫu tin đã cho trong ví dụ 2-1.

Bước 1: Xen $a[2]$ vào dãy chỉ có một phần tử $a[1]$ ta được dãy hai phần tử $a[1]..a[2]$ có thứ tự. Việc xen này thực ra không phải làm gì cả vì hai phần tử $a[1], a[2]$ có khoá tương ứng là 5 và 6 đã có thứ tự.

Bước 2: Xen $a[3]$ vào dãy $a[1]..a[2]$ ta được dãy ba phần tử $a[1]..a[3]$ có thứ tự. Việc xen này được thực hiện bằng cách : so sánh khoá của $a[3]$ với khoá của $a[2]$, do khoá của $a[3]$ nhỏ hơn khoá của $a[2]$ ($2 < 6$) nên hoán đổi $a[3]$ và $a[2]$ cho nhau. Lại so sánh khoá của $a[2]$ với khoá của $a[1]$, do khoá của $a[2]$ nhỏ hơn khoá của $a[1]$ ($2 < 5$) nên hoán đổi $a[2]$ và $a[1]$ cho nhau.

Tiếp tục quá trình này và sau 9 bước thì kết thúc.

Bảng sau ghi lại các giá trị khoá tương ứng với từng bước.

Khóa Bước	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]	A[8]	a[9]	a[10]
Ban đầu	5	6	2	2	10	12	9	10	9	3
Bước 1	5	6								
Bước 2	2	5	6							
Bước 3	2	2	5	6						
Bước 4	2	2	5	6	10					
Bước 5	2	2	5	6	10	12				
Bước 6	2	2	5	6	9	10	12			
Bước 7	2	2	5	6	9	10	10	12		
Bước 8	2	2	5	6	9	9	10	10	12	
Bước 9	2	2	3	5	6	9	9	10	10	12

Hình 2-2: Sắp xếp xen

2.3.2.2 Chương trình

```
PROCEDURE InsertionSort;
VAR
    i, j: integer;
```

```

BEGIN
{1}  FOR i := 2 TO n DO BEGIN
{2}      J := i;
{3}      WHILE (j>1) AND (a[j].key < a[j-1].key) DO BEGIN
{4}          swap(a[j], a[j-1]);
{5}          j := j-1;
      END;
    END;
END;

```

2.3.2.3 Đánh giá: Phương pháp sắp xếp xen lầy $O(n^2)$ để sắp xếp n phần tử.

Ta thấy các lệnh {4} và {5} đều lấy $O(1)$. Vòng lặp {3} chạy nhiều nhất $i-1$ lần, mỗi lần tốn $O(1)$ nên {3} lấy $i-1$ thời gian. Lệnh {2} và {3} là hai lệnh nối tiếp nhau, lệnh {2} lấy $O(1)$ nên cả hai lệnh này lấy $i-1$.

Vòng lặp {1} có i chạy từ 2 đến n nên nếu gọi $T(n)$ là thời gian để sắp n phần tử thì ta có

$$T(n) = \sum_{i=2}^n (i-1) = \frac{n(n-1)}{2} \text{ tức là } O(n^2).$$

2.3.3 Sắp xếp nổi bọt (Bubble Sort)

2.3.3.1 Giải thuật

Chúng ta tưởng tượng rằng các mẫu tin được lưu trong một mảng dọc, qua quá trình sắp, mẫu tin nào có khoá “nhẹ” sẽ được nổi lên trên. Chúng ta duyệt toàn mảng, từ dưới lên trên. Nếu hai phần tử ở cạnh nhau mà không đúng thứ tự tức là nếu phần tử “nhẹ hơn” lại nằm dưới thì phải cho nó “nổi lên” bằng cách đổi chỗ hai phần tử này cho nhau. Cụ thể là:

- Bước 1: Xét các phần tử từ $a[n]$ đến $a[2]$, với mỗi phần tử $a[j]$, so sánh khoá của nó với khoá của phần tử $a[j-1]$ đứng ngay trước nó. Nếu khoá của $a[j]$ nhỏ hơn khoá của $a[j-1]$ thì hoán đổi $a[j]$ và $a[j-1]$ cho nhau.
- Bước 2: Xét các phần tử từ $a[n]$ đến $a[3]$, và làm tương tự như trên.
- Sau $n-1$ bước thì kết thúc.

Ví dụ 2-3: Sắp xếp mảng gồm 10 mẫu tin đã cho trong ví dụ 2-1.

Bước 1: Xét $a[10]$ có khoá là 3, nhỏ hơn khoá của $a[9]$ nên ta hoán đổi $a[10]$ và $a[9]$ cho nhau. Khoá của $a[9]$ bây giờ là 3 nhỏ hơn khoá của $a[8]$ nên ta hoán đổi $a[9]$ và $a[8]$ cho nhau. Khoá của $a[8]$ bây giờ là 3 nhỏ hơn khoá của $a[7]$ nên ta hoán đổi $a[8]$ và $a[7]$ cho nhau. Khoá của $a[7]$ bây giờ là 3 nhỏ hơn khoá của $a[6]$ nên ta hoán đổi $a[7]$ và $a[6]$ cho nhau. Khoá của $a[6]$ bây giờ là 3 nhỏ hơn khoá của $a[5]$ nên ta hoán đổi $a[6]$ và $a[5]$ cho nhau. Khoá của $a[5]$ bây giờ là 3 **không nhỏ hơn** khoá của $a[4]$ nên bỏ qua. Khoá của $a[4]$ là 2 **không nhỏ hơn** khoá của $a[3]$ nên bỏ qua. Khoá của $a[3]$ là 2 nhỏ hơn khoá của $a[2]$ nên ta hoán đổi $a[3]$ và $a[2]$ cho nhau. Khoá của $a[2]$ bây giờ là 2 nhỏ hơn khoá của $a[1]$ nên ta hoán đổi $a[2]$ và $a[1]$ cho nhau. Đến đây kết thúc bước 1 và $a[1]$ có khoá nhỏ nhất là 2.

Bước 2: Xét a[10] có khoá là 9, nhỏ hơn khoá của a[9] nên ta hoán đổi a[10] và a[9] cho nhau. Khoá của a[9] bây giờ là 9 **không nhỏ hơn** khoá của a[8] nên bỏ qua. Khoá của a[8] là 9 nhỏ hơn khoá của a[7] nên ta hoán đổi a[8] và a[7] cho nhau. Khoá của a[7] bây giờ là 9 nhỏ hơn khoá của a[6] nên ta hoán đổi a[7] và a[6] cho nhau. Khoá của a[6] bây giờ là 9 **không nhỏ hơn** khoá của a[5] nên bỏ qua. Khoá của a[5] bây giờ là 3 **không nhỏ hơn** khoá của a[4] nên bỏ qua. Khoá của a[4] là 2 nhỏ hơn khoá của a[3] nên ta hoán đổi a[4] và a[3] cho nhau. Khoá của a[3] bây giờ là 2 nhỏ hơn khoá của a[2] nên ta hoán đổi a[3] và a[2] cho nhau. Đến đây kết thúc bước 2 và a[2] có khoá là 2.

Tiếp tục quá trình này và sau 9 bước thì kết thúc.

Bảng sau ghi lại các giá trị khoá tương ứng với từng bước.

Khóa Bước	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]	a[8]	a[9]	a[10]
Ban đầu	5	6	2	2	10	12	9	10	9	3
Bước 1	2	5	6	2	3	10	12	9	10	9
Bước 2		2	5	6	3	9	10	12	9	10
Bước 3			3	5	6	9	9	10	12	10
Bước 4				5	6	9	9	10	10	12
Bước 5					6	9	9	10	10	12
Bước 6						9	9	10	10	12
Bước 7							9	10	10	12
Bước 8								10	10	12
Bước 9									10	12
Kết quả	2	2	3	5	6	9	9	10	10	12

Hình 2-3: Sắp xếp nổi bọt

2.3.3.2 Chương trình

```

PROCEDURE BubbleSort;
VAR
    i, j: integer;
BEGIN
    {1} FOR i := 1 to n-1 DO
    {2}     FOR j := n DOWNTO i+1 DO
    {3}         IF a[j].key < a[j-1].key THEN
    {4}             Swap(a[j], a[j-1]);
    END;

```

2.3.3.3 Đánh giá: Phương pháp sắp xếp nổi bọt lấy $O(n^2)$ để sắp n phần tử.

Dòng lệnh {3} lấy một hằng thời gian. Vòng lặp {2} thực hiện (n-i) bước, mỗi bước lấy $O(1)$ nên lấy $O(n-i)$ thời gian. Như vậy đối với toàn bộ chương trình ta có:

$$T(n) = \sum_{i=1}^{n-1} (n-i) = \frac{n(n-1)}{2} = O(n^2).$$

2.4 QUICKSORT

Trong phần này chúng ta sẽ nghiên cứu một giải thuật sắp xếp được dùng một cách phổ biến là Quick Sort do A.R. Hoare phát minh vào năm 1960. Quick Sort đã được cải tiến để trở thành phương pháp được chọn trong các ứng dụng sắp xếp thực tế khác nhau.

2.4.1 Ý tưởng

Chúng ta vẫn xét mảng a các mẫu tin $a[1]..a[n]$. Giả sử v là 1 giá trị khóa mà ta gọi là chốt (pivot). Ta phân hoạch dãy $a[1]..a[n]$ thành hai mảng con "bên trái" và "bên phải". Mảng con "bên trái" bao gồm các phần tử có **khóa nhỏ hơn chốt**, mảng con "bên phải" bao gồm các phần tử có **khóa lớn hơn hoặc bằng chốt**.

Sắp xếp mảng con "bên trái" và mảng con "bên phải" thì mảng đã cho sẽ được sắp bởi vì tất cả các khóa trong mảng con "bên trái" đều nhỏ hơn các khóa trong mảng con "bên phải".

Việc sắp xếp các mảng con "bên trái" và "bên phải" cũng được tiến hành bằng phương pháp nói trên.

Một mảng chỉ gồm một phần tử hoặc gồm nhiều phần tử có khóa bằng nhau thì đã có thứ tự.

2.4.2 Thiết kế giải thuật

2.4.2.1 Vấn đề chọn chốt

Chọn **khóa lớn nhất** trong hai phần tử có khóa khác nhau đầu tiên kể từ trái qua. Nếu mảng chỉ gồm một phần tử hay gồm nhiều phần tử có khóa bằng nhau thì không có chốt.

Ví dụ 2-5: Chọn chốt trong các mảng sau

Cho mảng gồm các phần tử có khóa là 6, 6, 5, 8, 7, 4, ta chọn chốt là 6 (khóa của phần tử đầu tiên).

Cho mảng gồm các phần tử có khóa là 6, 6, 7, 5, 7, 4, ta chọn chốt là 7 (khóa của phần tử thứ 3).

Cho mảng gồm các phần tử có khóa là 6, 6, 6, 6, 6, 6 thì không có chốt (các phần tử có khóa bằng nhau).

Cho mảng gồm một phần tử có khóa là 6 thì không có chốt (do chỉ có một phần tử).

2.4.2.2 Vấn đề phân hoạch

Để phân hoạch mảng ta dùng 2 "con nháy" L và R trong đó L từ bên trái và R từ bên phải, ta cho L chạy sang phải cho tới khi gặp phần tử có khóa $\geq$ chốt và cho R chạy sang trái cho tới khi gặp phần tử có khóa $<$ chốt. Tại chỗ dừng của L và R nếu $L < R$ thì hoán vị $a[L], a[R]$. Lặp lại quá trình dịch sang phải, sang trái của 2 "con nháy" L và R cho đến khi $L > R$. Khi đó L sẽ là điểm phân hoạch, cụ thể là $a[L]$ là phần tử đầu tiên của mảng con "bên phải".

2.4.2.3 Giải thuật QuickSort

Để sắp xếp mảng $a[i]..a[j]$ ta tiến hành các bước sau:

- Xác định chốt.
- Phân hoạch mảng đã cho thành hai mảng con $a[i]..a[k-1]$ và $a[k]..a[j]$.
- Sắp xếp mảng $a[i]..a[k-1]$ (Đệ quy).
- Sắp xếp mảng $a[k]..a[j]$ (Đệ quy).

Quá trình đệ quy sẽ dừng khi không còn tìm thấy chốt.

Ví dụ 2-4: Sắp xếp mảng gồm 10 mẫu tin có khóa là các số nguyên: 5, 8, 2, 10, 5, 12, 8, 1, 15 và 4.

Với mảng $a[1]..a[10]$, hai phần tử đầu tiên có khóa khác nhau là $a[1]$ và $a[2]$ với khóa tương ứng là 5 và 8, ta chọn chốt $v = 8$.

Để phân hoạch, khởi đầu ta cho $L := 1$ (đặt L ở cực trái) và $R := 10$ (đặt R ở cực phải). Do $a[L]$ có khóa là 5 nhỏ hơn chốt nên $L := L+1 = 2$ (di chuyển L sang phải), lúc này $a[L]$ có khóa là 8 = chốt nên dừng lại. Do $a[R]$ có khóa là 4 nhỏ hơn chốt nên R cũng không chuyển sang trái được. Tại các điểm dừng của L và R ta có $L < R$ ($L=2$ và $R=10$) nên hoán đổi $a[L]$ và $a[R]$ ($a[2]$ và $a[10]$) cho nhau. Sau khi hoán đổi, $a[L]$ lại có khóa là 4 nhỏ hơn chốt nên di chuyển L sang phải ($L := L+1 = 3$). Khóa của $a[L]$ là 2 nhỏ hơn chốt nên lại di chuyển L sang phải ($L := L+1 = 4$). Khóa của $a[L]$ là 10 lớn hơn chốt nên dừng lại. Với R , khóa của $a[R]$ bây giờ là 8 bằng chốt nên di chuyển R sang trái ($R := R-1 = 9$). Khóa của $a[R]$ là 15 lớn hơn chốt nên di chuyển R sang trái ($R := R-1 = 8$). Khóa của $a[R]$ là 1 nhỏ hơn chốt nên dừng lại. Tại các điểm dừng của L và R ta có $L < R$ ($L=4$ và $R=8$) nên hoán đổi $a[L]$ và $a[R]$ ($a[4]$ và $a[8]$) cho nhau. Sau khi hoán đổi, $a[L]$ có khóa là 1 nhỏ hơn chốt nên di chuyển L sang phải ($L := L+1 = 5$). Khóa của $a[L]$ là 5 nhỏ hơn chốt nên lại di chuyển L sang phải ($L := L+1 = 6$). Khóa của $a[L]$ là 12 lớn hơn chốt nên dừng lại. Với R , khóa của $a[R]$ bây giờ là 10 lớn hơn chốt nên di chuyển R sang trái ($R := R-1 = 7$). Khóa của $a[R]$ là 8 bằng chốt nên di chuyển R sang trái ($R := R-1 = 6$). Khóa của $a[R]$ là 12 lớn hơn chốt nên di chuyển R sang trái ($R := R-1 = 5$). Khóa của $a[R]$ là 5 nhỏ hơn chốt nên dừng lại. Tại các điểm dừng của L và R ta có $L > R$ ($L=6$ và $R=5$) nên ta đã xác định được điểm phân hoạch ứng với $L = 6$. Tức là mảng đã cho ban đầu được phân thành hai mảng con bên trái $a[1]..a[5]$ và mảng con bên phải $a[6]..a[10]$. Hình ảnh của sự phân hoạch này được biểu diễn trong hình sau:

Chỉ số	1	2	3	4	5	6	7	8	9	10
Khoá	5	8	2	10	5	12	8	1	15	4
Ban đầu		4		1				10		8
$v = 8$										
Cấp 1	5	4	2	1	5	12	8	10	15	8

Hình 2-4 : Chọn chốt và phân hoạch mảng $a[1]..a[10]$

Trong bảng trên, dòng *chỉ số* ghi các chỉ số của các phần tử của mảng (từ 1 đến 10).

Trong dòng *khoá ban đầu*, các giá trị khoá ở dòng trên (5, 8, 2, 10, 5, 12, 8, 1, 15 và 4) là các giá trị khoá của mảng đã cho ban đầu, các giá trị khoá ở dòng dưới (4, 1, 10 và 8) là các giá trị khoá mới sau khi thực hiện hoán đổi $a[2]$ với $a[10]$ và $a[4]$ với $a[8]$.

Giá trị chốt là $v = 8$.

Dòng *cấp 1*, biểu diễn hai mảng con sau khi phân hoạch. Mảng bên trái từ $a[1]$ đến $a[5]$ gồm các phần tử có khoá là 5, 4, 2, 1 và 5. Mảng con bên phải từ $a[6]$ đến $a[10]$ gồm các phần tử có khoá 12, 8, 10, 15 và 8.

Tiếp tục sắp xếp đệ quy cho mảng con bên trái và mảng con bên phải.

Với mảng con bên trái $a[1]..a[5]$, hai phần tử đầu tiên có khoá khác nhau là $a[1]$ và $a[2]$ với khoá tương ứng là 5 và 4, ta chọn chốt $v = 5$.

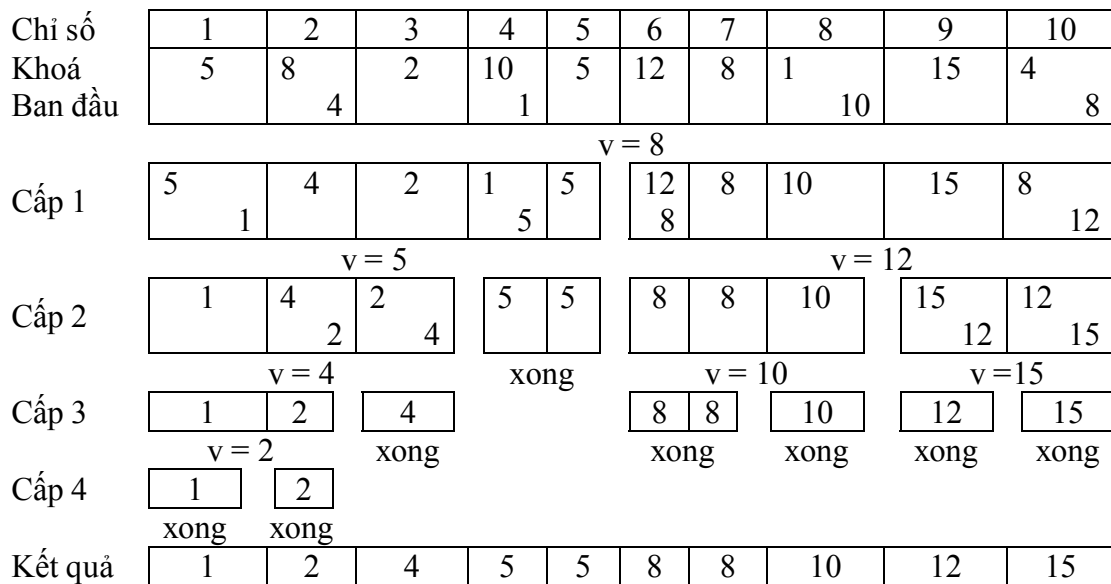
Để phân hoạch, khởi đầu ta cho $L := 1$ (đặt L ở cực trái) và $R := 5$ (đặt R ở cực phải). Do $a[L]$ có khoá là 5 bằng chốt nên không thể di chuyển L . Do $a[R]$ có khoá là 5 bằng chốt nên di chuyển R sang trái ($R := R-1 = 4$). Khoá của $a[R]$ bây giờ là 1 nhỏ hơn chốt nên dừng lại. Tại các điểm dừng của L và R ta có $L < R$ ($L=$ và $R=4$) nên hoán đổi $a[L]$ và $a[R]$ ($a[1]$ và $a[4]$) cho nhau. Sau khi hoán đổi, $a[L]$ lại có khoá là 1 nhỏ hơn chốt nên di chuyển L sang phải ($L := L+1 = 2$). Khoá của $a[L]$ là 4 nhỏ hơn chốt nên lại di chuyển L sang phải ($L := L+1 = 3$). Khoá của $a[L]$ là 2 nhỏ hơn chốt nên lại di chuyển L sang phải ($L := L+1 = 4$). Khoá của $a[L]$ là 5 bằng chốt nên dừng lại. Với R , khoá của $a[R]$ bây giờ là 5 bằng chốt nên di chuyển R sang trái ($R := R-1 = 4$). Khoá của $a[R]$ là 5 bằng chốt nên di chuyển R sang trái ($R := R-1 = 3$). Khoá của $a[R]$ là 2 nhỏ hơn chốt nên dừng lại. Tại các điểm dừng của L và R ta có $L > R$ ($L=4$ và $R=3$) nên ta đã xác định được điểm phân hoạch ứng với $L = 4$. Tức là mảng bên trái phân thành hai mảng con bên trái $a[1]..a[3]$ và mảng con bên phải $a[4]..a[6]$.

Hình ảnh của sự phân hoạch này được biểu diễn trong hình sau:

Chỉ số	1	2	3	4	5	6	7	8	9	10
Khoá	5	8	2	10	5	12	8	1	15	4
Ban đầu		4		1				10		8
$v = 8$										
Cấp 1	5 1	4	2	1	5	12	8	10	15	8
$v = 5$										
Cấp 2	1	4	2	5	5					

Hình 2-5 : Chọn chốt và phân hoạch mảng $a[1]..a[5]$

Tiếp tục sắp xếp cho các mảng con của cấp 1 và mảng con bên phải của mảng ban đầu cho đến khi dừng (các mảng không có chốt). Cuối cùng ta có mảng được sắp thứ tự. Hình sau biểu diễn toàn bộ quá trình sắp xếp.



Hình 2-6 : QuickSort

2.4.3 Cài đặt giải thuật

2.4.3.1 Hàm FindPivot

Ta thiết kế hàm FindPivot để xác định trong dãy $a[i]..a[j]$ có hay không hai phần tử có khóa khác nhau. Nếu không tìm thấy hai phần tử có khóa khác nhau thì trả về giá trị 0 (không tìm thấy chốt), ngược lại hàm trả về giá trị là chỉ số của phần tử có khóa lớn hơn trong hai phần tử có khóa khác nhau đầu tiên. Khóa lớn hơn này sẽ trở thành phần tử chốt mà ta sẽ xác định trong thủ tục QuickSort.

Để tiện so sánh ta sử dụng biến FirstKey để lưu giữ khóa của phần tử đầu tiên trong mảng $a[i]..a[j]$ (FirstKey chính là $a[i].key$).

Ta sẽ dùng một chỉ số k để dò tìm trong mảng $a[i]..a[j]$, kể từ vị trí $i+1$ đến hết mảng, một phần tử $a[k]$ mà $a[k].key < \text{FirstKey}$. Nếu không tìm thấy một $a[k]$ như thế thì hoặc là mảng chỉ gồm một phần tử hoặc gồm nhiều phần tử có khóa bằng nhau. Trong trường hợp đó thì không tìm thấy chốt và hàm FindPivot sẽ trả về 0. Ngược lại ta sẽ phải xét xem $a[k].key$ có lớn hơn FirstKey hay không, nếu đúng như thế thì chốt sẽ là khóa của $a[k]$ và hàm FindPivot sẽ trả về k, nếu không thì chốt sẽ là khóa của $a[i]$ và hàm FindPivot sẽ trả về i.

```

FUNCTION FindPivot(i,j:integer): integer;
VAR  FirstKey : KeyType;
      k : integer;
BEGIN
{1}   k := i+1;
{2}   FirstKey := a[i].key;
{3}   WHILE (k <= j) AND (a[k].key = FirstKey) DO k:= k+1;
{4}   IF k > j THEN FindPivot := 0
      ELSE

```

```

{5}      IF a[k].key > FirstKey THEN FindPivot := k
        ELSE FindPivot := i;
END;

```

Trong hàm FindPivot các lệnh {1}, {2}, {3} và {4} nối tiếp nhau, trong đó chỉ có lệnh WHILE là tốn nhiều thời gian nhất do đó thời gian thực hiện của hàm FindPivot phụ thuộc vào thời gian thực hiện của lệnh này. Trong trường hợp xấu nhất (không tìm thấy chốt) thì k chạy từ i+1 đến j, tức là vòng lặp thực hiện j-i lần, mỗi lần $O(1)$ do đó tốn j-i thời gian. Đặc biệt khi i=1 và j=n, thì thời gian thực hiện là n-1 hay $T(n) = O(n)$.

2.4.3.2 Hàm Partition

Hàm Partition nhận vào ba tham số i, j và Pivot để thực hiện việc phân hoạch mảng a[i]..a[j] theo chốt Pivot và trả về giá trị L là chỉ số đầu tiên của mảng “bên phải”.

Hai con nháy L, R sẽ được sử dụng để thực hiện việc phân hoạch như đã trình bày trong phần 2.4.2.3.

```

FUNCTION Partition(i,j:integer; pivot :KeyType):integer ;
VAR  L,R : integer;
BEGIN
{1}  L := i;  {Đặt con nháy L ở cực trái}
{2}  R := j;  {Đặt con nháy R ở cực phải}
{3}  WHILE L <= r DO BEGIN
        {L tiến sang phải}
{4}      WHILE a[L].key < pivot DO L := L+1;
        {R tiến sang trái}
{5}      WHILE a[R].key >= pivot DO R := R-1;
{6}      IF L < R THEN Swap(a[L],a[R]);
    END;
{7}  Partition := L;  {Trả về điểm phân hoạch}
END;

```

Trong hàm Partition các lệnh {1}, {2}, {3} và {7} nối tiếp nhau, trong đó thời gian thực hiện của lệnh {3} là lớn nhất, do đó thời gian thực hiện của lệnh {3} sẽ là thời gian thực hiện của hàm Partition. Các lệnh {4}, {5} và {6} là thân của lệnh {3}, trong đó lệnh {6} lấy $O(1)$ thời gian. Lệnh {4} và lệnh {5} thực hiện việc di chuyển L sang phải và R sang trái, thực chất là duyệt các phần tử mảng, mỗi phần tử một lần, mỗi lần tốn $O(1)$ thời gian. Tổng cộng việc duyệt này tốn j-i thời gian. Vòng lặp {3} thực chất là để xét xem khi nào thì duyệt xong, do đó thời gian thực hiện của lệnh {3} chính là thời gian thực hiện của hai lệnh {4} và {5} và do đó là j-i. Đặc biệt khi i=1 và j=n ta có $T(n) = O(n)$.

2.4.3.3 Thủ tục QuickSort

Bây giờ chúng ta trình bày thủ tục cuối cùng có tên là QuickSort và chú ý rằng để sắp xếp mảng A các record gồm n phần tử của kiểu Recordtype ta chỉ cần gọi QuickSort(1,n).

Ta sẽ sử dụng biến PivotIndex để lưu giữ kết quả trả về của hàm FindPivot, nếu biến PivotIndex nhận được một giá trị khác 0 thì mới tiến hành phân hoạch mảng.

Ngược lại, mảng không có chốt và do đó đã có thứ tự. Biến Pivot sẽ được sử dụng để lưu giữ giá trị chốt và biến k để lưu giữ giá trị của điểm phân hoạch do hàm Partition trả về. Sau khi đã phân hoạch xong ta sẽ gọi đệ quy QuickSort cho mảng con “bên trái” $a[i]..a[k-1]$ và mảng con “bên phải” $a[k]..a[j]$.

```
PROCEDURE Quicksort(i,j:integer);
VAR
    Pivot : KeyType;
    PivotIndex, k : integer;
BEGIN
    PivotIndex := FindPivot(i,j);
    IF PivotIndex <> 0 THEN BEGIN
        Pivot := a[PivotIndex].key;
        k := Partition(i,j,Pivot);
        Quicksort(i,k-1);
        Quicksort(k,j);
    END;
END;
```

2.4.4 Thời gian thực hiện của QuickSort

QuickSort lấy $O(n \log n)$ thời gian để sắp xếp n phần tử trong trường hợp tốt nhất và $O(n^2)$ trong trường hợp xấu nhất.

Giả sử các giá trị khóa của mảng khác nhau nên hàm FindPivot luôn tìm được chốt và đệ quy chỉ dừng khi kích thước bài toán bằng 1.

Gọi $T(n)$ là thời gian thực hiện việc QuickSort mảng có n phần tử.

Thời gian để tìm chốt và phân hoạch mảng như đã phân tích trong các phần 2.4.3.1 và 2.4.3.2 đều là $O(n) = n$.

Khi $n = 1$, thủ tục QuickSort chỉ làm một nhiệm vụ duy nhất là gọi hàm Findpivot với kích thước bằng 1, hàm này tốn thời gian $O(1) = 1$.

Trong trường hợp xấu nhất là ta luôn chọn phải phần tử có khóa lớn nhất làm chốt, lúc bấy giờ việc phân hoạch bị lệch tức là mảng bên phải chỉ gồm một phần tử chốt, còn mảng bên trái gồm $n-1$ phần tử còn lại. Khi đó ta có thể thành lập phương trình đệ quy như sau:

$$T(n) = \begin{cases} 1 & \text{nếu } n = 1 \\ T(n-1) + T(1) + n & \text{nếu } n > 1 \end{cases}$$

Giải phương trình này bằng phương pháp truy hồi

$$\begin{aligned} \text{Ta có } T(n) &= T(n-1) + T(1) + n = T(n-1) + (n+1) \\ &= [T(n-2) + T(1) + (n-1)] + (n+1) = T(n-2) + n + (n+1) \\ &= [T(n-3) + T(1) + (n-2)] + n + (n+1) = T(n-3) + (n-1) + n + (n+1) \\ &\dots \end{aligned}$$

$$T(n) = T(n-i) + (n-i+2) + (n-i+3) + \dots + n + (n+1) = T(n-i) + \sum_{j=n-i+2}^{n+1} j$$

Quá trình trên kết thúc khi $i = n-1$, khi đó ta có $T(n) = T(1) + \sum_{j=3}^{n+1} 2 = 1 + \sum_{j=3}^{n+1} 2$

$$= \sum_{j=1}^{n+1} 2 - 2 = \frac{n^2 + 3n - 2}{2} = O(n^2)$$

Trong trường hợp tốt nhất khi ta chọn được chốt sao cho hai mảng con có kích thước bằng nhau và bằng $n/2$. Lúc đó ta có phương trình đệ quy như sau:

$$T(n) = \begin{cases} 1 & \text{nếu } n = 1 \\ 2T(\frac{n}{2}) + n & \text{nếu } n > 1 \end{cases}$$

Giải phương trình đệ quy này ta được $T(n) = O(n \log n)$.

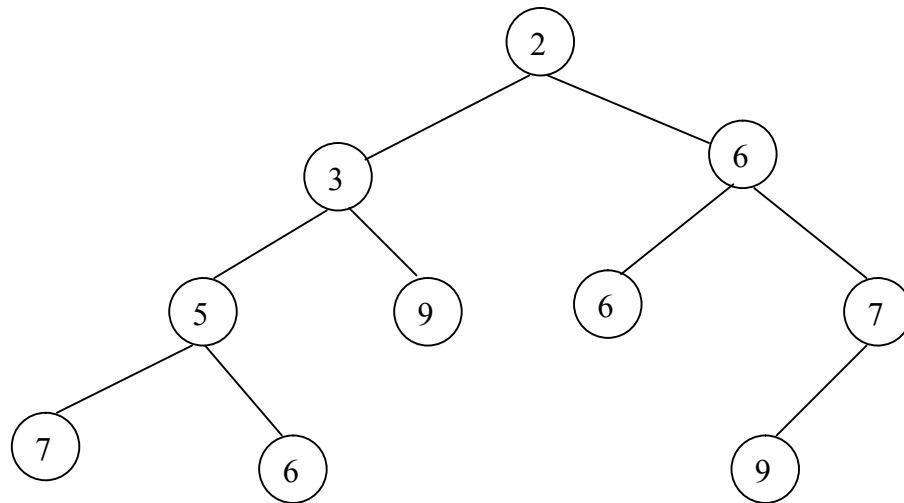
2.5 HEAPSORT

2.5.1 Định nghĩa Heap

Cây sắp thứ tự bộ phận hay còn gọi là heap là cây nhị phân mà giá trị tại mỗi nút (khác nút lá) đều không lớn hơn giá trị của các con của nó.

Ta có nhận xét rằng nút gốc $a[1]$ của cây sắp thứ tự bộ phận có giá trị nhỏ nhất.

Ví dụ 2-5: Cây sau là một heap.



Hình 2-7: Một heap

2.5.2 Ý tưởng

- (1) Xem mảng ban đầu là một cây nhị phân. Mỗi nút trên cây lưu trữ một phần tử mảng, trong đó $a[1]$ là nút gốc và mỗi nút không là nút lá $a[i]$ có con trái là $a[2i]$ và con phải là $a[2i+1]$. Với cách tổ chức này thì cây nhị phân thu được sẽ có các nút trong là các nút $a[1], \dots, a[n \text{ DIV } 2]$. Tất cả các nút trong đều có 2 con, ngoại trừ nút $a[n \text{ DIV } 2]$ có thể chỉ có một con trái (trong trường hợp n là một số chẵn).
 - (2) Sắp xếp cây ban đầu thành một heap căn cứ vào giá trị khoá của các nút.
 - (3) Hoán đổi $a[1]$ cho phần tử cuối cùng.
 - (4) Sắp lại cây sau khi đã bỏ đi phần tử cuối cùng để nó trở thành một heap mới.
- Lặp lại quá trình (3) và (4) cho tới khi cây chỉ còn một nút ta sẽ được mảng sắp theo thứ tự giảm.

2.5.3 Thiết kế và cài đặt giải thuật

2.5.3.1 Thủ tục PushDown

Thủ tục PushDown nhận vào 2 tham số first và last để đẩy nút first xuống.

Giả sử $a[\text{first}], \dots, a[\text{last}]$ đã đúng vị trí (giá trị khoá tại mỗi nút nhỏ hơn hoặc bằng giá trị khoá tại các nút con của nó) ngoại trừ $a[\text{first}]$. PushDown dùng để đẩy phần tử $a[\text{first}]$ xuống đúng vị trí của nó trong cây (và có thể gây ra việc đẩy xuống các phần tử khác).

Xét $a[\text{first}]$, có các khả năng có thể xảy ra:

- Nếu $a[\text{first}]$ chỉ có một con trái và nếu khoá của nó lớn hơn khoá của con trái ($a[\text{first}].\text{key} > a[2*\text{first}].\text{key}$) thì hoán đổi $a[\text{first}]$ cho con trái của nó và kết thúc.
- Nếu $a[\text{first}]$ có khoá lớn hơn con trái của nó ($a[\text{first}].\text{key} > a[2*\text{first}].\text{key}$) và khoá của con trái không lớn hơn khoá của con phải ($a[2*\text{first}].\text{key} \leq a[2*\text{first}+1].\text{key}$) thì hoán đổi $a[\text{first}]$ cho con trái $a[2*\text{first}]$ của nó, việc này có thể gây ra tình trạng con trái sẽ không đúng vị trí nên phải xem xét lại con trái để có thể đẩy xuống.
- Ngược lại, nếu $a[\text{first}]$ có khoá lớn hơn khoá của con phải của nó ($a[\text{first}].\text{key} > a[2*\text{first}+1].\text{key}$) và khoá của con phải nhỏ hơn khoá của con trái ($a[2*\text{first}+1].\text{key} < a[2*\text{first}].\text{key}$) thì hoán đổi $a[\text{first}]$ cho con phải $a[2*\text{first}+1]$ của nó, việc này có thể gây ra tình trạng con phải sẽ không đúng vị trí nên phải tiếp tục xem xét con phải để có thể đẩy xuống.
- Nếu tất cả các trường hợp trên đều không xảy ra thì $a[\text{first}]$ đã đúng vị trí.

Như trên ta thấy việc đẩy $a[\text{first}]$ xuống có thể gây ra việc đẩy xuống một số phần tử khác, nên tổng quát là ta sẽ xét việc đẩy xuống của một phần tử $a[r]$ bất kỳ, bắt đầu từ $a[\text{first}]$.

```

PROCEDURE PushDown(first,last:integer);
VAR  r:integer;
BEGIN
    r:= first;  {Xét nút a[first] trước hết}
    WHILE r <= last DIV 2 DO
        If last = 2*r THEN BEGIN {nút r chỉ có con trái }
            IF a[r].key > a[last].key THEN swap(a[r],a[last]);
            r:=last;  {Kết thúc}
        END ELSE
        IF (a[r].key>a[2*r].key)and(a[2*r].key<= a[2*r+1].key)
        THEN BEGIN
            swap(a[r],a[2*r]);
            r := 2*r ; {Xét tiếp nút con trái }
        END
        ELSE
        IF (a[r].key>a[2*r+1].key)and(a[2*r+1].key<a[2*r].key)
        THEN BEGIN
            swap(a[r],a[2*r+1]);
            r := 2*r+1 ; {Xét tiếp nút con phải }
        END
        ELSE
            r := last; {Nút r đã đúng vị trí }
    END;

```

Thủ tục PushDown chỉ duyệt trên một nhánh nào đó của cây nhị phân, tức là sau mỗi lần lặp thì số nút còn lại một nửa. Nếu số nút lúc đầu là n , trong trường hợp xấu nhất (luôn phải thực hiện việc đẩy xuống) thì lệnh lặp WHILE phải thực hiện i lần sao cho $2^i = n$ tức là $i = \log n$. Mà mỗi lần lặp chỉ thực hiện một lệnh IF với thân lệnh IF là gọi thủ tục Swap và gán, do đó tốn $O(1) = 1$ đơn vị thời gian. Như vậy thủ tục PushDown lấy $O(\log n)$ để đẩy xuống một nút trong cây có n nút.

2.5.3.2 Thủ tục HeapSort

- Việc sắp xếp cây ban đầu thành một heap được tiến hành bằng cách sử dụng thủ tục PushDown để đẩy tất cả các nút trong chưa đúng vị trí xuống đúng vị trí của nó, khởi đầu từ nút $a[n \text{ DIV } 2]$, lần ngược tới gốc.
- Lặp lại việc hoán đổi $a[1]$ cho $a[i]$, sắp xếp cây $a[1]..a[i-1]$ thành heap, i chạy từ n đến 2.

```

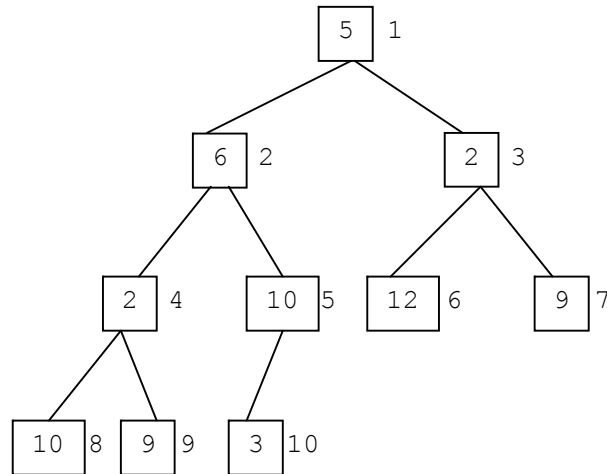
PROCEDURE HeapSort;
VAR  i:integer;
BEGIN
    {1} FOR i := (n div 2) DOWNT0 1 DO
    {2}     PushDown(i,n);
    {3} FOR i := n DOWNT0 2 DO BEGIN
    {4}     swap(a[1],a[i]);
    {5}     pushdown(1,i-1);
    END;
END;

```

Ví dụ 2-6: Sắp xếp mảng bao gồm 10 phần tử có khoá là các số nguyên như trong các ví dụ 2.1:

Chỉ số	1	2	3	4	5	6	7	8	9	10
Khoá ban đầu	5	6	2	2	10	12	9	10	9	3

Mảng này được xem như là một cây nhị phân ban đầu như sau:

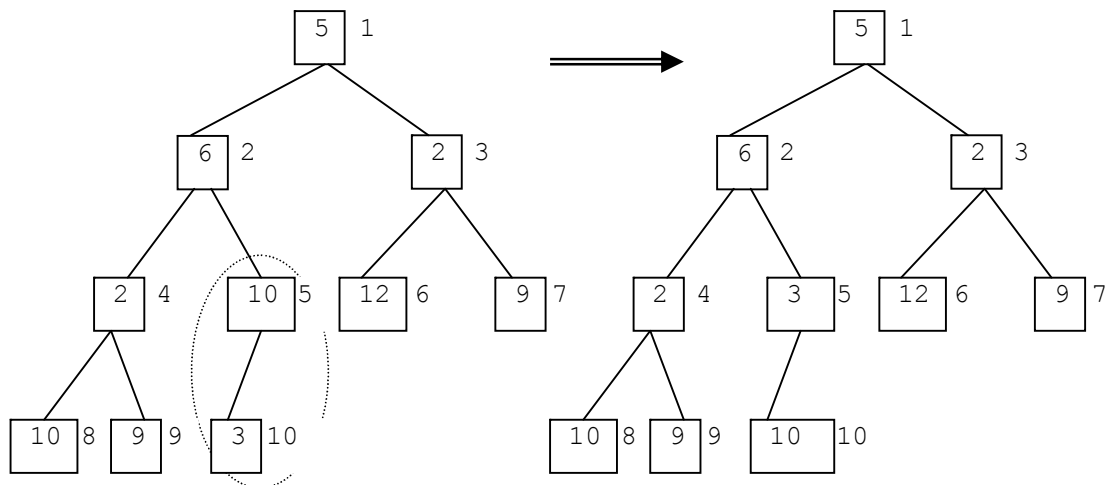


Hình 2-8: Cây ban đầu

Trong cây trên, giá trị ghi trong các nút là khoá của các phần tử mảng, giá trị ghi bên ngoài các nút là chỉ số của các phần tử mảng.

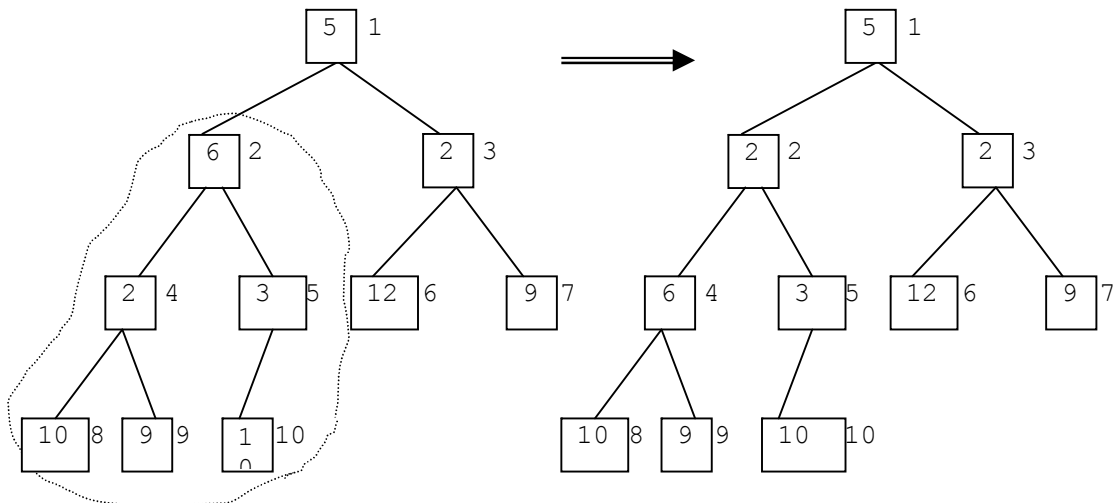
Việc sắp xếp cây này thành một heap sẽ bắt đầu từ việc đẩy xuống nút $a[5]$ (vì $5 = 10 \text{ DIV } 2$)

Xét nút 5 ta thấy $a[5]$ chỉ có một con trái và giá trị khóa tương ứng của nó lớn hơn con trái của nó nên ta đổi hai nút này cho nhau và do con trái của $a[5]$ là $a[10]$ là một nút lá nên việc đẩy xuống của $a[5]$ kết thúc.



Hình 2-9: Thực hiện đẩy xuống của nút 5

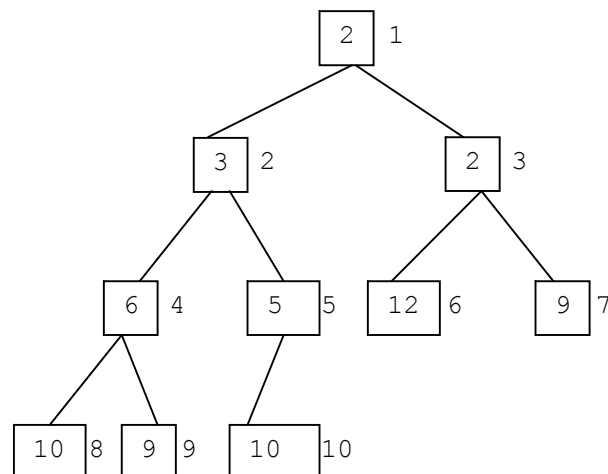
Nút 4 và nút 3 đã đúng vị trí nên không phải thực hiện sự hoán đổi. Tại nút 2, giá trị khoá của nó lớn hơn khoá con trái và khoá của con trái nhỏ hơn khoá của con phải nên ta hoán đổi nút 2 cho con trái của nó (nút 4), sau khi hoán đổi, ta xét lại nút 4, thấy nó vẫn đúng vị trí nên kết thúc việc đẩy xuống của nút 2.



Hình 2-10: Thực hiện đẩy xuống của nút 2

Cuối cùng ta xét nút 1, ta thấy giá trị khoá của nút 1 lớn hơn khoá của con trái và con trái có khoá bằng khoá của con phải nên hoán đổi nút 1 cho con trái của nó (nút 2).

Sau khi thực hiện phép hoán đổi nút 1 cho nút 2, ta thấy nút 2 có giá trị khoá lớn hơn khoá của con phải của nó (nút 5) và con phải có khoá nhỏ hơn khoá của con trái nên phải thực hiện phép hoán đổi nút 2 cho nút 5. Xét lại nút 5 thì nó vẫn đúng vị trí nên ta được cây mới trong hình 2-11.



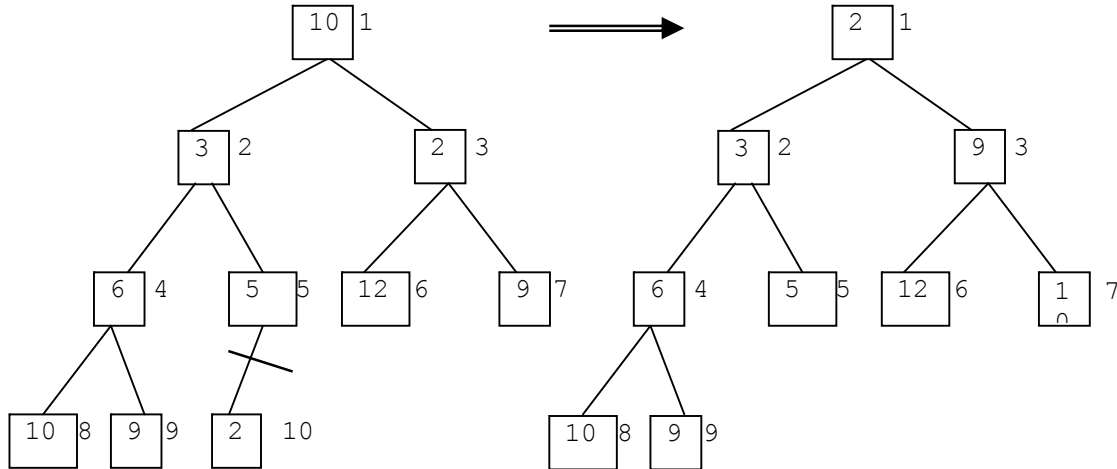
Hình 2-11: Cây ban đầu đã được tạo thành heap

Cây này là một heap tương ứng với mảng

Chỉ số	1	2	3	4	5	6	7	8	9	10
Heap	2	3	2	6	5	12	9	10	9	10

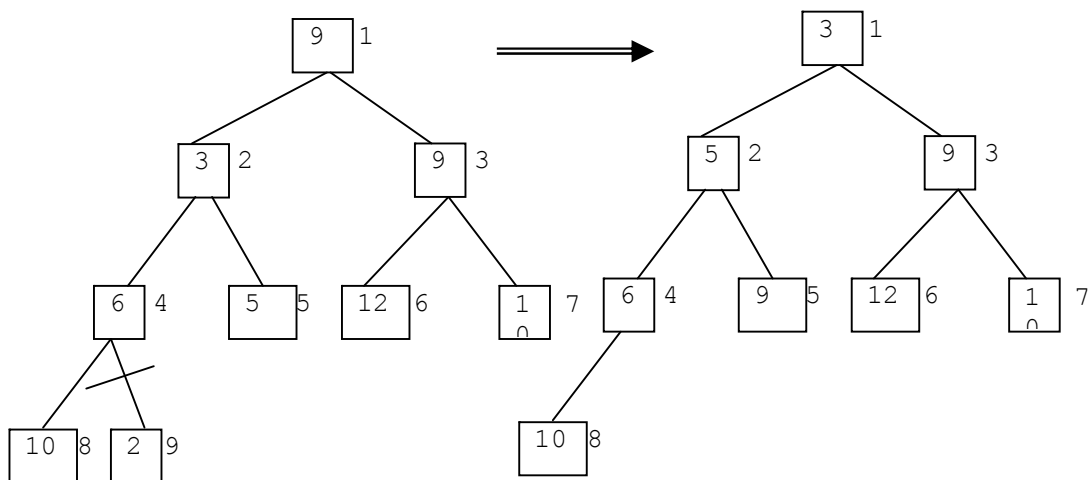
Từ heap đã có ở trên, hoán đổi $a[1]$ cho $a[10]$ ta có $a[10]$ là nút có khóa nhỏ nhất, cắt bỏ nút $a[10]$ ra khỏi cây. Như vậy phần cuối mảng chỉ gồm một phần tử $a[10]$ đã được sắp.

Thực hiện việc đẩy $a[1]$ xuống đúng vị trí của nó trong cây $a[1]..a[9]$ ta được cây:



Hình 2-12: Hoán đổi $a[1]$ cho $a[10]$ và đẩy $a[1]$ xuống trong $a[1..9]$

Hoán đổi $a[1]$ cho $a[9]$ và cắt $a[9]$ ra khỏi cây. Ta được phần cuối mảng bao gồm hai phần tử $a[9]..a[10]$ đã được sắp. Thực hiện việc đẩy $a[1]$ xuống đúng vị trí của nó trong cây $a[1]..a[8]$ ta được cây



Hình 2-13: Hoán đổi $a[1]$ cho $a[9]$ và đẩy $a[1]$ xuống trong $a[1..8]$

Tiếp tục quá trình trên ta sẽ được một mảng có thứ tự giảm.

Trình bày heapsort bằng mảng

Như trong phần ý tưởng đã nói, chúng ta chỉ xem mảng như là một cây. Điều đó có nghĩa là các thao tác thực chất vẫn là các thao tác trên mảng. Để hiểu rõ hơn, ta sẽ trình bày ví dụ trên sử dụng mô hình mảng.

Mảng của 10 mẫu tin, có khoá là các số nguyên đã cho là:

Chỉ số	1	2	3	4	5	6	7	8	9	10
Khoá ban đầu	5	6	2	2	10	12	9	10	9	3

Mặc dù không vẽ thành cây, nhưng ta vẫn tưởng tượng mảng này như là một cây nhị phân với nút gốc là $a[1]$, các nút $a[i]$ có con trái là $a[2i]$ và con phải là $a[2i+1]$. Chỉ có các nút từ $a[1]$ đến $a[5]$ là nút trong, còn các nút từ $a[6]$ đến $a[10]$ là nút lá.

Từ mảng ban đầu, chúng ta sẽ tạo thành heap bằng cách áp dụng thủ tục PushDown từ $a[5]$ đến $a[1]$.

Xét $a[5]$, nút này chỉ có một con trái là $a[10]$ và khoá của $a[5]$ lớn hơn khoá của $a[10]$ ($10 > 3$) nên đẩy $a[5]$ xuống (hoán đổi $a[5]$ và $a[10]$ cho nhau).

Xét $a[4]$, nút này có hai con là $a[8]$ và $a[9]$ và khoá của nó đều nhỏ hơn khoá của hai con ($2 < 10$ và $2 < 9$) nên không phải đẩy xuống.

Tương tự $a[3]$ cũng không phải đẩy xuống.

Xét $a[2]$, nút này có con trái là $a[4]$ và con phải là $a[5]$. Khoá của $a[2]$ lớn hơn khoá của con trái ($6 > 2$) và khoá của con trái nhỏ hơn khoá của con phải ($2 < 3$) do đó đẩy $a[2]$ xuống bên trái (hoán đổi $a[2]$ và $a[4]$ cho nhau). Tiếp tục xét con trái của $a[2]$, tức là $a[4]$. Khoá của $a[4]$ bây giờ là 6, nhỏ hơn khoá của con trái $a[8]$ ($6 < 10$) và khoá của con phải $a[9]$ ($6 < 9$) nên không phải đẩy $a[4]$ xuống.

Xét $a[1]$, nút này có con trái là $a[2]$ và con phải là $a[3]$. Khoá của $a[1]$ lớn hơn khoá của con trái $a[2]$ ($5 > 2$) và khoá của con trái bằng khoá của con phải ($2 = 2$) nên đẩy $a[1]$ xuống bên trái (hoán đổi $a[1]$ và $a[2]$ cho nhau). Tiếp tục xét con trái $a[2]$. Nút này có con trái là $a[4]$ và con phải là $a[5]$. Khoá của $a[2]$ bây giờ là 5 lớn hơn khoá của con phải $a[5]$ ($5 > 3$) và khoá của con phải $a[5]$ nhỏ hơn khoá của con trái $a[4]$ ($3 < 6$) nên đẩy $a[2]$ xuống bên phải (hoán đổi $a[2]$ và $a[5]$ cho nhau). Tiếp tục xét con phải $a[5]$. Nút này chỉ có một con trái là $a[10]$ và khoá của $a[5]$ nhỏ hơn khoá của $a[10]$ nên không phải đẩy $a[5]$ xuống. Quá trình đến đây kết thúc và ta có được heap trong bảng sau:

Chỉ số	1	2	3	4	5	6	7	8	9	10
Ban đầu	5 2	6 2 5 3	2	2 6	10 3 5	12	9	10	9	3 10
Heap	2	3	2	6	5	12	9	10	9	10

Hình 2-14: Mảng ban đầu đã tạo thành heap

Trong bảng trên, dòng *Ban đầu* bao gồm hai dòng. Dòng trên ghi các giá trị khoá ban đầu của mảng. Dòng dưới ghi các giá trị khoá sau khi đã có một sự hoán đổi.

Thứ tự ghi từ trái sang phải, tức là số bên trái là giá trị khoá sau khi thực hiện việc hoán đổi đầu tiên trong quá trình PushDown.

Sau khi đã có heap, ta bắt đầu quá trình sắp xếp.

Ở bước đầu tiên, ứng với $i = 10$, hoán đổi $a[1]$ và $a[10]$ cho nhau, ta được $a[10]$ có khóa nhỏ nhất. Để đẩy $a[1]$ xuống trong cây $a[1]..a[9]$, ta thấy khóa của $a[1]$ bây giờ lớn hơn khóa của con phải $a[3]$ ($10 > 2$) và khóa của con phải $a[3]$ nhỏ hơn khóa của con trái $a[2]$ ($2 < 3$) do đó đẩy $a[1]$ xuống bên phải (hoán đổi $a[1]$ và $a[3]$ cho nhau). Tiếp tục xét $a[3]$, khóa của $a[3]$ lớn hơn khóa của con phải $a[7]$ và khóa của con phải nhỏ hơn khóa của con trái, do đó ta đẩy $a[3]$ xuống bên phải (hoán đổi $a[3]$ và $a[7]$ cho nhau) và vì $a[7]$ là nút lá nên việc đẩy xuống kết thúc. Ta có bảng sau:

Chỉ số	1	2	3	4	5	6	7	8	9	10
Ban đầu	5 2	6 2 5 3	2	2 6	10 3 5	12	9	10	9	3 10
Heap	2 10 2	3	2 10 9	6	5	12	9 10	10	9	10 2
$i = 10$	2	3	9	6	5	12	10	10	9	2

Hình 2-15: Hoán đổi $a[1]$ với $a[10]$ và đẩy $a[1]$ xuống trong $a[1..9]$

Với $i = 9$, ta hoán đổi $a[1]$ và $a[9]$ cho nhau. Để đẩy $a[1]$ xuống trong cây $a[1]..a[8]$, ta thấy khóa của $a[1]$ bây giờ lớn hơn khóa của con trái $a[2]$ và khóa của con trái nhỏ hơn khóa của con phải $a[3]$ nên đẩy $a[1]$ xuống bên trái (hoán đổi $a[1]$ và $a[2]$ cho nhau). Tiếp tục xét $a[2]$, khóa của $a[2]$ lớn hơn khóa của con phải $a[5]$ và khóa của con phải nhỏ hơn khóa của con trái $a[4]$ nên đẩy $a[2]$ xuống bên phải (hoán đổi $a[2]$ và $a[5]$ cho nhau) và vì $a[5]$ là nút lá (trong cây $a[1]..a[8]$) nên việc đẩy xuống kết thúc. Ta có bảng sau

Chỉ số	1	2	3	4	5	6	7	8	9	10
Ban đầu	5 2	6 2 5 3	2	2 6	10 3 5	12	9	10	9	3 10
Heap	2 10 2	3	2 10 9	6	5	12	9 10	10	9	10 2
$i = 10$	2 9 3	3 9 5	9	6	5 9	12	10	10	9 2	2
$i = 9$	3	5	9	6	9	12	10	10	2	

Hình 2-16: Hoán đổi $a[1]$ với $a[9]$ và đẩy $a[1]$ xuống trong $a[1..8]$

Với $i = 8$, ta hoán đổi $a[1]$ và $a[8]$ cho nhau. Để đẩy $a[1]$ xuống trong cây $a[1]..a[7]$, ta thấy khóa của $a[1]$ bây giờ lớn hơn khóa của con trái $a[2]$ và khóa của con trái nhỏ hơn khóa của con phải $a[3]$ nên đẩy $a[1]$ xuống bên trái (hoán đổi $a[1]$ và $a[2]$ cho nhau). Tiếp tục xét $a[2]$, khóa của $a[2]$ lớn hơn khóa của con trái $a[4]$ và khóa của con trái nhỏ hơn khóa của con phải $a[5]$ nên đẩy $a[2]$ xuống bên trái (hoán đổi $a[2]$ và $a[4]$ cho nhau) và vì $a[4]$ là nút lá (trong cây $a[1]..a[7]$) nên việc đẩy xuống kết thúc. Ta có bảng sau

Chỉ số	1	2	3	4	5	6	7	8	9	10
Ban đầu	5 2	6 2 5 3	2	2 6	10 3 5	12	9	10	9	3 10
Heap	2 10 2	3	2 10 9	6	5	12	9 10	10	9	10 2
i = 10	2 9 3	3 9 5	9	6	5 9	12	10	10	9 2	2
i = 9	3 10 5	5 10 6	9	6 10	9	12	10	10 3	2	
i = 8	5	6	9	10	9	12	10	3		

Hình 2-17: Hoán đổi $a[1]$ với $a[8]$ và đẩy $a[1]$ xuống trong $a[1..7]$

Tiếp tục quá trình trên và giải thuật kết thúc sau bước 9, ứng với bước $i = 2$.

2.5.4 Phân tích HeapSort

Thời gian thực hiện của HeapSort là $O(n \log n)$

Như đã phân tích trong mục 2.5.3.1, thủ tục PushDown lấy $O(\log n)$ để đẩy một nút xuống trong cây có n nút.

Trong thủ tục HeapSort dòng lệnh $\{1\}-\{2\}$ lặp $n/2$ lần mà mỗi lần PushDown lấy $O(\log n)$ nên thời gian thực hiện $\{1\}-\{2\}$ là $O(n \log n)$. Vòng lặp $\{3\}-\{4\}-\{5\}$ lặp $n-1$ lần, mỗi lần PushDown lấy $O(\log n)$ nên thời gian thực hiện của $\{3\}-\{4\}-\{5\}$ là $O(n \log n)$. Tóm lại thời gian thực hiện HeapSort là $O(n \log n)$.

2.6 BINSORT

2.6.1 Giải thuật

Nói chung các giải thuật đã trình bày ở trên đều có độ phức tạp là $O(n^2)$ hoặc $O(n \log n)$. Tuy nhiên khi kiểu dữ liệu của trường khoá là một kiểu đặc biệt, việc sắp xếp có thể chỉ chiếm $O(n)$ thời gian. Sau đây ta sẽ xét một số trường hợp.

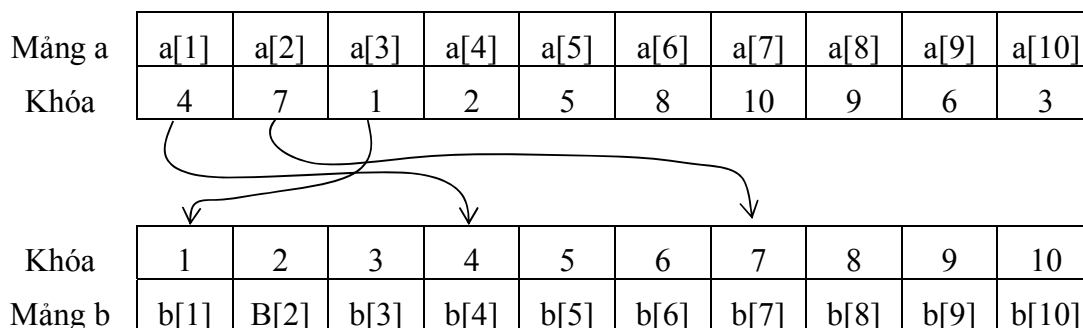
2.6.1.1 Trường hợp đơn giản:

Giả sử ta phải sắp xếp một mảng A gồm n phần tử có khoá là các số nguyên có giá trị khác nhau và là các giá trị từ 1 đến n . Ta sử dụng B là một mảng cùng kiểu với A và phân phối vào phần tử $b[j]$ một phần tử $a[i]$ mà $a[i].key = j$. Khi đó mảng B lưu trữ kết quả đã được sắp xếp của mảng A .

Ví dụ 2-7: Sắp xếp mảng A gồm 10 phần tử có khoá là các số nguyên có giá trị là các số 4, 7, 1, 2, 5, 8, 10, 9, 6 và 3

Ta sử dụng mảng B có cùng kiểu với A và thực hiện việc phân phối $a[1]$ vào $b[4]$ vì $a[1].key = 4$, $a[2]$ vào $b[7]$ vì $a[2].key = 7$, $a[3]$ vào $b[1]$ vì $a[3].key = 1$,...

Hình sau minh họa cho việc phân phối các phần tử của mảng a vào mảng b .



Hình 2-18: Phân phối các phần tử $a[i]$ vào các bin $b[j]$

Để thực hiện việc phân phối này ta chỉ cần một lệnh lặp:

```
for i:=1 to n do b[a[i].key] := a[i]
```

Đây cũng là lệnh chính trong chương trình sắp xếp. Lệnh này lấy $O(n)$ thời gian.

Các phần tử $b[j]$ được gọi là các **bin** và phương pháp sắp xếp này được gọi là bin sort.

2.6.1.2 Trường hợp tổng quát

Là trường hợp có thể có nhiều phần tử có chung một giá trị khóa, chẳng hạn để sắp một mảng A có n phần tử mà các giá trị khóa của chúng là các số nguyên lấy giá trị trong khoảng $1..m$ với $m \leq n$. Trong trường hợp này ta không thể sử dụng các phần tử của mảng B làm bin được vì nếu có hai phần tử của mảng A có cùng một khóa thì không thể lưu trữ trong cùng một bin.

Để giải quyết sự đụng độ này ta chuẩn bị một cấu trúc có m bin, mỗi bin có thể lưu trữ nhiều hơn một phần tử. Cụ thể là bin thứ j sẽ lưu các phần tử có khóa là j ($1 \leq j \leq m$) sau đó ta sẽ nối các bin lại với nhau để được một dãy các phần tử được sắp.

Cách tốt nhất là ta thiết kế mỗi bin là một danh sách liên kết của các phần tử mà mỗi phần tử có kiểu `RecordType`. Ta sẽ gọi kiểu của danh sách này là `ListType`.

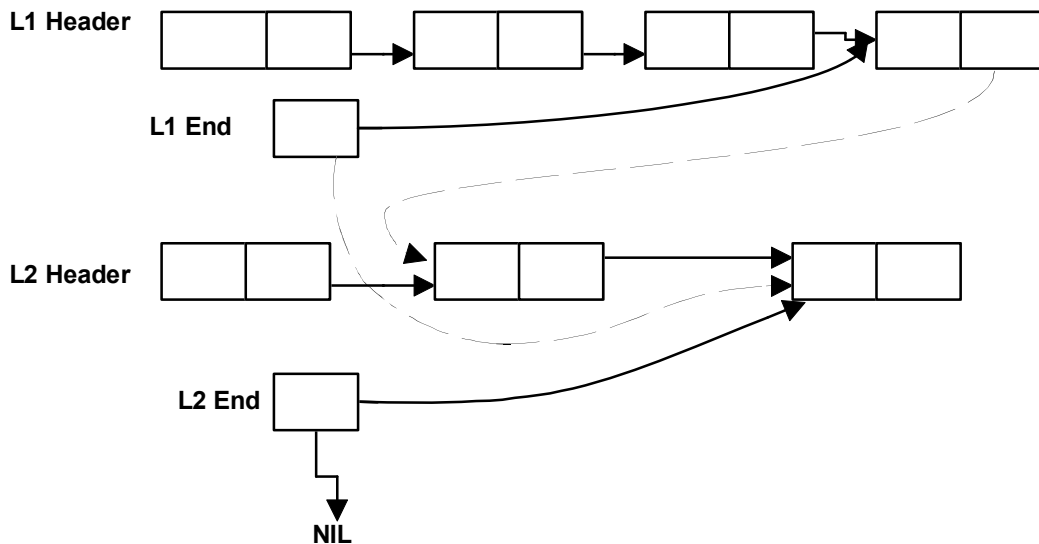
Ta có thể tạo kiểu `ListType` bằng cách ghép `RecordType` với một con trỏ để trỏ tới phần tử kế tiếp.

Lấy B là một mảng kiểu `Array[KeyType] of ListType`. Như vậy B là mảng các bin, mỗi bin là một danh sách. B được đánh chỉ số bởi `KeyType`, như thế có ít nhất một bin cho mỗi giá trị khóa.

Ta vẫn sẽ phân phối phần tử $a[i]$ vào bin $b[j]$ nếu $j = a[i].key$. Dĩ nhiên mỗi bin $b[j]$ có thể chứa nhiều phần tử của mảng A . Các phần tử mới sẽ được đưa vào cuối danh sách $b[j]$.

Sau khi tất cả các phần tử của mảng A đã được phân phối vào trong các bin, công việc cuối cùng là ta phải nối các bin lại với nhau, ta sẽ được một danh sách có thứ tự. Ta sẽ dùng thủ tục `concatenate(L1,L2)` để nối hai danh sách $L1, L2$. Nó thay thế danh sách $L1$ bởi danh sách nối $L1L2$. Việc nối sẽ được thực hiện bằng cách gán con trỏ của phần tử cuối cùng của $L1$ vào đầu của $L2$. Ta biết rằng để đến được phần tử cuối cùng của danh sách liên kết $L1$ ta phải duyệt qua tất cả các phần tử của

nó. Để cho có hiệu quả, ta thêm một con trỏ nữa, trỏ đến phần tử cuối cùng của mỗi danh sách, điều này giúp ta đi thẳng tới phần tử cuối cùng mà không phải duyệt qua toàn bộ danh sách. Hình sau minh họa việc nối hai danh sách.



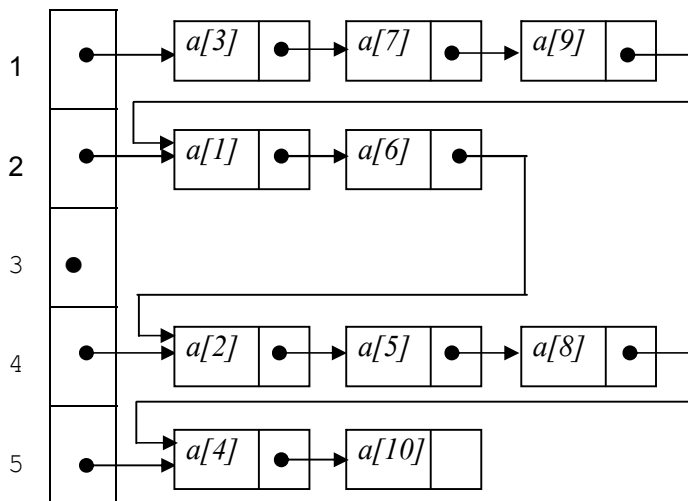
Hình 2-19: Nối các bin

Sau khi nối thì header và end của danh sách L2 không còn tác dụng nữa.

Ví dụ 2-8: Sắp xếp mảng A gồm 10 phần tử có khoá là các số nguyên có giá trị là các số 2, 4, 1, 5, 4, 2, 1, 4, 1, 5.

A	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]	a[8]	a[9]	a[10]
Khoá của A	2	4	1	5	4	2	1	4	1	5

Ta thấy các giá trị khoá nằm trong khoảng 1..5. Ta tổ chức một mảng B gồm 5 phần tử, mỗi phần tử là một con trỏ, trỏ đến một danh sách liên kết.



Hình 2-20: Binsort trong trường hợp tổng quát

Chương trình sử dụng cấu trúc danh sách liên kết làm các bin

```

VAR
a: ARRAY[1..n] OF RecordType;
b: ARRAY[keytype] OF ListType;
{Ta giả thiết keytype là kiểu miền con 1..m }
PROCEDURE BinSort;
VAR
    i: integer;
    j: KeyType;
BEGIN
{1}FOR i:=1 TO n DO
    Insert(A[i], END(B[A[i].key]), B[A[i].key]);
{2}FOR j:= 2 TO m DO
    Concatenate(B[1], B[j]);
END;
```

2.6.2 Phân tích Bin Sort

Bin sort lấy $O(n)$ thời gian để sắp xếp mảng gồm n phần tử.

Trước hết thủ tục INSERT cần một thời gian $O(1)$ để xen một phần tử vào trong danh sách. Do cách tổ chức danh sách có giữ con trỏ đến phần tử cuối cùng nên việc nối hai danh sách bằng thủ tục CONCATENATE cũng chỉ mất $O(1)$ thời gian. Ta thấy vòng lặp {1} thực hiện n lần, mỗi lần tốn $O(1) = 1$ nên lấy $O(n)$ đơn vị thời gian. Vòng lặp {2} thực hiện $m-1$ lần, mỗi lần $O(1)$ nên tốn $O(m)$ đơn vị thời gian. Hai lệnh {1} và {2} nối tiếp nhau nên thời gian thực hiện của BinSort là $T(n) = O(\max(n, m)) = O(n)$ vì $m \leq n$.

2.6.3 Sắp xếp tập giá trị có khoá lớn

Nếu m số các khoá không lớn hơn n số các phần tử cần sắp xếp, khi đó $O(\max(n, m))$ thực sự là $O(n)$. Nếu $n > m$ thì $T(n)$ là $O(m)$ và đặc biệt khi $m = n^2$ thì $T(n)$ là $O(n^2)$, như vậy Bin sort không tốt hơn các sắp xếp đơn giản khác.

Tuy nhiên trong một số trường hợp, ta vẫn có thể tổng quát hoá kỹ thuật bin sort để nó vẫn lấy $O(n)$ thời gian.

Giả sử ta cần sắp xếp n phần tử có các giá trị khoá thuộc $0..n^2-1$. Nếu sử dụng phương pháp cũ, ta cần n^2 bin (từ bin 0 đến bin n^2-1) và do đó việc nối n^2 bin này tốn $O(n^2)$, nên bin sort lấy $O(n^2)$.

Để giải quyết vấn đề này, ta sẽ sử dụng n bin $b[0], b[1], \dots, b[n-1]$ và tiến hành việc sắp xếp trong hai kì.

Kì 1: Phân phối phần tử $a[i]$ vào bin $b[j]$ mà $j = a[i].key \text{ MOD } n$.

Kì 2: Phân phối các phần tử trong danh sách kết quả của kỳ 1 vào các bin. Phần tử $a[i]$ sẽ được phân phối vào bin $b[j]$ mà $j = a[i].key \text{ DIV } n$.

Chú ý rằng trong cả hai kỳ, ta xen các phần tử mới được phân phối vào cuối danh sách.

Ví dụ 2-9: Cần sắp xếp mảng gồm 10 phần tử có khoá là các số nguyên: 36, 9, 10, 25, 1, 8, 34, 16, 81 và 99.

Ta sử dụng 10 bin được đánh số từ 0 đến 9. Kỳ một ta phân phối phần tử $a[i]$ vào bin có chỉ số $a[i].\text{key} \bmod 10$. Nối các bin của kỳ một lại với nhau ta được danh sách có khoá là: 10, 1, 81, 34, 25, 36, 16, 8, 9, 99. Kỳ hai sử dụng kết quả của kỳ 1 để sắp tiếp. Phân phối phần tử $a[i]$ vào bin có chỉ số $a[i].\text{key} \div 10$. Nối các bin của kỳ hai lại với nhau ta được danh sách có thứ tự.

Kỳ một			Kỳ hai			
Bin			Bin			
0	10		0	1	8	9
1	1	81	1	10	16	
2			2	25		
3			3	34	36	
4	34		4			
5	25		5			
6	36	16	6			
7			7			
8	8		8	81		
9	9	99	9	99		

Hình 2-21: Sắp xếp theo hai kỳ

Theo sự phân tích giải thuật Bin Sort thì mỗi kỳ lấy $O(n)$ thời gian, hai kỳ này nối tiếp nhau nên thời gian tổng cộng là $O(n)$.

2.6.3.1 Chứng minh giải thuật đúng

Để thấy tính đúng đắn của giải thuật ta xem các giá trị khoá nguyên từ 0 đến $n^2 - 1$ như các số có hai chữ số trong hệ đếm cơ số n . Xét hai số $K = s.n + t$ (lấy K chia cho n được s , dư t) và $L = u.n + v$ trong đó s, t, u, v là các số $0..n-1$. Giả sử $K < L$, ta cần chứng minh rằng sau 2 kỳ sắp thì K phải đứng trước L .

Vì $K < L$ nên $s \leq u$. Ta có hai trường hợp là $s < u$ và $s = u$.

Trường hợp 1: Nếu $s < u$ thì K đứng trước L trong danh sách kết quả vì trong kỳ hai, K được sắp vào bin $b[s]$ và L được sắp vào bin $b[u]$ mà $b[s]$ đứng trước $b[u]$. Chẳng hạn trong ví dụ trên, ta chọn $K = 16$ và $L = 25$. Ta có $K = 1 \times 10 + 6$ và $L = 2 \times 10 + 5$ ($s = 1, t = 6, u = 2$ và $v = 5; s < u$). Trong kỳ hai, $K = 16$ được sắp vào bin 1 và $L = 25$ được sắp vào bin 2 nên $K = 16$ đứng trước $L = 25$.

Trường hợp 2: Nếu $s = u$ thì $t < v$ (do $K < L$). Sau kỳ một thì K đứng trước L , vì K được sắp vào trong bin $b[t]$ và L được sắp vào trong bin $b[v]$. Đến kỳ hai, mặc dù cả K và L đều được sắp vào trong bin $b[s]$, nhưng K được xen vào trước L nên kết quả

là K đứng trước L. Chẳng hạn trong ví dụ trên ta chọn $K = 34$ và $L = 36$. Ta có $K = 3 \times 10 + 4$ và $L = 3 \times 10 + 6$. Sau kì một thì $K = 34$ đứng trước $L = 36$ vì K được sắp vào bin 4 còn L được sắp vào bin 6. Trong kì hai, cả K và L đều được sắp vào bin 3, nhưng do K được xét trước nên K đứng trước L trong bin 3 và do đó K đứng trước L trong kết quả cuối cùng.

Chú ý: Từ chứng minh trên ta thấy để sắp các phần tử có khóa là các số nguyên (hệ đếm cơ số 10) từ 0 đến 99 ta dùng 10 bin có chỉ số từ 0 đến 9. Để sắp các phần tử có khóa là các số nguyên từ 0 đến 9999 ta dùng 100 bin có chỉ số từ 0 đến 99...

2.7 TỔNG KẾT CHƯƠNG 2

Các giải thuật sắp xếp đơn giản có giải thuật đơn giản nhưng kém hiệu quả về mặt thời gian. Tất cả các giải thuật sắp xếp đơn giản đều lấy $O(n^2)$ để sắp xếp n mẫu tin.

Các giải thuật QuickSort và HeapSort đều rất hiệu quả về mặt thời gian (độ phức tạp $O(n \log n)$), do đó chúng thường được sử dụng trong thực tế, nhất là QuickSort.

BinSort chỉ sử dụng được cho dữ liệu đặc biệt.

BÀI TẬP CHƯƠNG 2

Bài 1: Sắp xếp mảng gồm 12 phần tử có khóa là các số nguyên: 5, 15, 12, 2, 10, 12, 9, 1, 9, 3, 2, 3 bằng cách sử dụng:

- Sắp xếp chọn.
- Sắp xếp xen.
- Sắp xếp nổi bọt.
- QuickSort.
- HeapSort (Sắp thứ tự giảm, sử dụng mô hình cây và sử dụng bảng).

Bài 2: Viết thủ tục sắp xếp trộn (xem giải thuật thô trong chương 1).

Bài 3: Viết lại hàm FindPivot để hàm trả về giá trị chốt và viết lại thủ tục QuickSort phù hợp với hàm FindPivot mới này.

Bài 4: Có một biến thể của QuickSort như sau: Chọn chốt là khóa của phần tử nhỏ nhất trong hai phần tử có khóa khác nhau đầu tiên. Mảng con bên trái gồm các phần tử có khóa nhỏ hơn hoặc bằng chốt, mảng con bên phải gồm các phần tử có khóa lớn hơn chốt. Hãy viết lại các thủ tục cần thiết cho biến thể này.

Bài 5: Một biến thể khác của QuickSort là chọn khóa của phần tử đầu tiên làm chốt. Hãy viết lại các thủ tục cần thiết cho biến thể này.

Bài 6: Hãy viết lại thủ tục PushDown trong HeapSort bằng giải thuật đệ quy.

Bài 7: Hãy viết lại thủ tục PushDown trong HeapSort để có thể sắp xếp theo thứ tự tăng.

CHƯƠNG 3: KỸ THUẬT THIẾT KẾ GIẢI THUẬT

3.1 TỔNG QUAN

3.1.1 Mục tiêu

Nắm vững các kỹ thuật thiết kế giải thuật: chia để trị, quy hoạch động, tham ăn, quay lui, cắt tỉa alpha-beta, nhánh cận và tìm kiếm địa phương. Với mỗi kỹ thuật cần nắm được:

- Nội dung kỹ thuật.
- Vận dụng kỹ thuật vào giải các bài toán thực tế.
- Đánh giá được giải thuật.

3.1.2 Kiến thức cơ bản cần thiết

Các cấu trúc dữ liệu, đặc biệt là cấu trúc cây và đồ thị.

3.1.3 Tài liệu tham khảo

A.V. Aho, J.E. Hopcroft, J.D. Ullman; *Data Structures and Algorithms*; Addison-Wesley; 1983. (Chapter 10).

Jeffrey H Kingston; *Algorithms and Data Structures*; Addison-Wesley; 1998. (Chapter 12).

Đinh Mạnh Tường; *Cấu trúc dữ liệu & Thuật toán*; Nhà xuất bản khoa học và kỹ thuật; Hà Nội-2001. (Chương 8).

Nguyễn Đức Nghĩa, Tô Văn Thành; *Toán rời rạc*; 1997 (Chương 3, 5).

3.1.4 Nội dung cốt lõi

Nói chung khi thiết kế một giải thuật chúng ta thường dựa vào một số kỹ thuật nào đó. Chương này sẽ trình bày một số kỹ thuật quan trọng để thiết kế giải thuật như: Chia để trị (Divide-and-Conquer), quy hoạch động (dynamic programming), kỹ thuật tham ăn (greedy techniques), quay lui (backtracking) và tìm kiếm địa phương (local search). Các kỹ thuật này được áp dụng vào một lớp rộng các bài toán, trong đó có những bài toán cổ điển nổi tiếng như bài toán tìm đường đi ngắn nhất của người giao hàng, bài toán cây phủ tối thiểu...

3.2 KỸ THUẬT CHIA ĐỂ TRỊ

3.2.1 Nội dung kỹ thuật

Có thể nói rằng kỹ thuật quan trọng nhất, được áp dụng rộng rãi nhất để thiết kế các giải thuật có hiệu quả là kỹ thuật "chia để trị" (divide and conquer). Nội dung của nó là: Để giải một bài toán kích thước n , ta chia bài toán đã cho thành một số bài toán con có kích thước nhỏ hơn. Giải các bài toán con này rồi tổng hợp kết quả lại để được lời giải của bài toán ban đầu. Đối với các bài toán con, chúng ta lại sử dụng kỹ

thuật chia để trị để có được các bài toán kích thước nhỏ hơn nữa. Quá trình trên sẽ dẫn đến những bài toán mà lời giải chúng là hiển nhiên hoặc dễ dàng thực hiện, ta gọi các bài toán này là **bài toán cơ sở**.

Tóm lại kĩ thuật chia để trị bao gồm hai quá trình: Phân tích bài toán đã cho thành các bài toán cơ sở và tổng hợp kết quả từ bài toán cơ sở để có lời giải của bài toán ban đầu. Tuy nhiên đối với một số bài toán, thì quá trình phân tích đã chứa đựng việc tổng hợp kết quả do đó nếu chúng ta đã giải xong các bài toán cơ sở thì bài toán ban đầu cũng đã được giải quyết. Ngược lại có những bài toán mà quá trình phân tích thì đơn giản nhưng việc tổng hợp kết quả lại rất khó khăn. Trong các phần tiếp sau ta sẽ trình bày một số ví dụ để thấy rõ hơn điều này.

Kĩ thuật này sẽ cho chúng ta một giải thuật đệ quy mà việc xác định độ phức tạp của nó sẽ phải giải một phương trình đệ quy như trong chương I đã trình bày.

3.2.2 Nhìn nhận lại giải thuật MergeSort và QuickSort

Hai giải thuật sắp xếp đã được trình bày trong các chương trước (MergeSort trong chương I và QuickSort trong chương II) thực chất là đã sử dụng kĩ thuật chia để trị.

Với MergeSort, để sắp một danh sách L gồm n phần tử, chúng ta chia L thành hai danh sách con $L1$ và $L2$ mỗi danh sách có $n/2$ phần tử. Sắp xếp $L1$, $L2$ và trộn hai danh sách đã được sắp này để được một danh sách có thứ tự. Quá trình phân tích ở đây là quá trình chia đôi một danh sách, quá trình này sẽ dẫn đến bài toán sắp xếp một danh sách có độ dài bằng 1, đây chính là bài toán cơ sở vì việc sắp xếp danh sách này là “không làm gì cả”. Việc tổng hợp các kết quả ở đây là “trộn 2 danh sách đã được sắp để được một danh sách có thứ tự”.

Với QuickSort, để sắp xếp một danh sách gồm n phần tử, ta tìm một giá trị chốt và phân hoạch danh sách đã cho thành hai danh sách con “bên trái” và “bên phải”. Sắp xếp “bên trái” và “bên phải” thì ta được danh sách có thứ tự. Quá trình phân chia sẽ dẫn đến các bài toán sắp xếp một danh sách chỉ gồm một phần tử hoặc gồm nhiều phần tử có khoá bằng nhau, đó chính là các bài toán cơ sở, vì bản thân chúng đã có thứ tự rồi. Ở đây chúng ta cũng không có việc tổng hợp kết quả một cách tường minh, vì việc đó đã được thực hiện trong quá trình phân hoạch.

3.2.3 Bài toán nhân các số nguyên lớn

Trong các ngôn ngữ lập trình đều có kiểu dữ liệu số nguyên (chẳng hạn kiểu integer trong Pascal, Int trong C...), nhưng nhìn chung các kiểu này đều có miền giá trị hạn chế (chẳng hạn từ -32768 đến 32767) nên khi có một ứng dụng trên số nguyên lớn (hàng chục, hàng trăm chữ số) thì kiểu số nguyên định sẵn không đáp ứng được. Trong trường hợp đó, người lập trình phải tìm một cấu trúc dữ liệu thích hợp để biểu diễn cho một số nguyên, chẳng hạn ta có thể dùng một chuỗi kí tự để biểu diễn cho một số nguyên, trong đó mỗi kí tự lưu trữ một chữ số. Để thao tác được trên các số nguyên được biểu diễn bởi một cấu trúc mới, người lập trình phải xây dựng các phép toán cho số nguyên như phép cộng, phép trừ, phép nhân... Sau đây ta sẽ đề cập đến bài toán nhân hai số nguyên lớn.

Xét bài toán nhân hai số nguyên lớn X và Y , mỗi số có n chữ số.

Đầu tiên ta nghĩ đến giải thuật nhân hai số thông thường, nghĩa là nhân từng chữ số của X với số Y rồi cộng các kết quả lại. Việc nhân từng chữ số của X với số Y đòi hỏi phải nhân từng chữ số của X với từng chữ số của Y, vì X và Y đều có n chữ số nên cần n^2 phép nhân hai chữ số, mỗi phép nhân hai chữ số này tốn $O(1)$ thì phép nhân cũng tốn $O(n^2)$ thời gian.

Áp dụng kĩ thuật "chia để trị" vào phép nhân các số nguyên lớn, ta chia mỗi số nguyên lớn X và Y thành các số nguyên lớn có $n/2$ chữ số. Để đơn giản cho việc phân tích giải thuật ta giả sử **n là lũy thừa của 2**, còn về khía cạnh lập trình, ta vẫn có thể viết chương trình với n bất kì.

$$X = A10^{n/2} + B \text{ và } Y = C10^{n/2} + D$$

Trong đó A, B, C, D là các số nguyên lớn có $n/2$ chữ số.

Chẳng hạn với $X = 1234$ thì $A = 12$ và $B = 34$ bởi vì $X = 12 * 10^2 + 34$.

$$\text{Khi đó tích của X và Y là: } XY = AC10^n + (AD + BC)10^{n/2} + BD \quad (\text{III.1})$$

Với mỗi số có $n/2$ chữ số, chúng ta lại tiếp tục phân tích theo cách trên, quá trình phân tích sẽ dẫn đến bài toán cơ sở là nhân các số nguyên lớn chỉ gồm một chữ số mà ta dễ dàng thực hiện. Việc tổng hợp kết quả chính là thực hiện các phép toán theo công thức (III.1).

Theo (III.1) thì chúng ta phải thực hiện 4 phép nhân các số nguyên lớn $n/2$ chữ số (AC, AD, BC, BD), sau đó tổng hợp kết quả bằng 3 phép cộng các số nguyên lớn n chữ số và 2 phép nhân với 10^n và $10^{n/2}$.

Các phép cộng các số nguyên lớn n chữ số dĩ nhiên chỉ cần $O(n)$. Phép nhân với 10^n có thể thực hiện một cách đơn giản bằng cách thêm vào n chữ số 0 và do đó cũng chỉ lấy $O(n)$. Gọi $T(n)$ là thời gian để nhân hai số nguyên lớn, mỗi số có n chữ số thì từ (III.1) ta có phương trình đệ quy:

$$T(1) = 1$$

$$T(n) = 4T(n/2) + cn \quad (\text{III.2})$$

Giải (III.2) ta được $T(n) = O(n^2)$. Như vậy thì chẳng cải tiến được chút nào so với giải thuật nhân hai số bình thường. Để cải thiện tình hình, chúng ta có thể viết lại (III.1) thành dạng:

$$XY = AC10^n + [(A-B)(D-C) + AC + BD] 10^{n/2} + BD \quad (\text{III.3})$$

Công thức (III.3) chỉ đòi hỏi 3 phép nhân của các số nguyên lớn $n/2$ chữ số là: AC, BD và $(A-B)(D-C)$, 6 phép cộng trừ và 2 phép nhân với 10^n . Các phép toán này đều lấy $O(n)$ thời gian. Từ (III.3) ta có phương trình đệ quy:

$$T(1) = 1$$

$$T(n) = 3T(n/2) + cn$$

Giải phương trình đệ quy này ta được nghiệm $T(n) = O(n^{\log_3 3}) = O(n^{1.59})$. Giải thuật này rõ ràng đã được cải thiện rất nhiều.

Giải thuật thô để nhân hai số nguyên lớn (dương hoặc âm) n chữ số là:

```
FUNCTION Mult(X, Y: Big_integer; n:integer) : Big_integer;
```

```

VAR
  m1,m2,m3,A,B,C,D: Big_integer;
  s: integer;{Luu trữ dấu của tích xy}
BEGIN
  s := sign(X)*sign(Y);
  x := ABS(X);{Lấy trị tuyệt đối của x}
  y := ABS(Y);
  IF n = 1 THEN mult := X*Y*s
  ELSE BEGIN
    A := left(X, n DIV 2);
    B := right(X, n DIV 2);
    C := left(Y, n DIV 2);
    D := right(Y, n DIV 2);
    m1 := mult(A,C, n DIV 2);
    m2 := mult(A-B,D-C, n DIV 2);
    m3 := mult(B,D, n DIV 2);
    mult := (s * (m1 * 10n + (m1+m2+m3)* 10n DIV 2 + m3));
  END
END;

```

Hàm Mult nhận vào ba tham số, trong đó X và Y là hai số nguyên lớn (kiểu Big_integer), n là số chữ số của X và Y và trả về một số nguyên lớn là tích XY.

A, B, C, D là các biến thuộc kiểu Big_integer, lưu trữ các số nguyên lớn trong việc chia đôi các số nguyên lớn X và Y. m1, m2 và m3 là các biến thuộc kiểu Big_integer lưu trữ các số nguyên lớn trung gian trong công thức (III.3), cụ thể là $m1 = AC$, $m2 = (A-B)(D-C)$ và $m3 = BD$.

Hàm sign nhận vào một số nguyên lớn X và cho giá trị 1 nếu X dương và -1 nếu X âm.

Hàm ABS nhận vào một số nguyên lớn X và cho kết quả là giá trị tuyệt đối của X.

Hàm Left nhận vào một số nguyên lớn X và một số nguyên k, cho kết quả là một số nguyên lớn có k chữ số bên trái của X. Tương tự như thế cho hàm Right.

3.2.4 Xếp lịch thi đấu thể thao

Kĩ thuật chia để trị không những chỉ có ứng dụng trong thiết kế giải thuật mà còn trong nhiều lĩnh vực khác của cuộc sống. Chẳng hạn xét việc xếp lịch thi đấu thể thao theo thể thức đấu vòng tròn 1 lượt cho n đấu thủ. Mỗi đấu thủ phải đấu với các đấu thủ khác, và mỗi đấu thủ chỉ đấu nhiều nhất một trận mỗi ngày. Yêu cầu là xếp một lịch thi đấu sao cho số ngày thi đấu là ít nhất. Ta dễ dàng thấy rằng tổng số trận đấu của toàn giải là $\frac{n(n-1)}{2}$. Như vậy nếu n là một số chẵn thì ta có thể sắp n/2 cặp

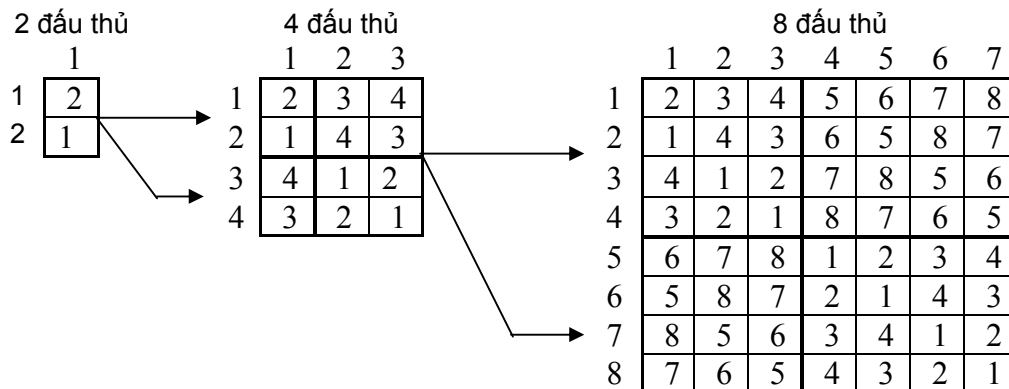
thi đấu trong một ngày và do đó cần ít nhất n-1 ngày. Ngược lại nếu n là một số lẻ thì n-1 là một số chẵn nên ta có thể sắp (n-1)/2 cặp thi đấu trong một ngày và do đó ta cần n ngày. Giả sử $n = 2^k$ thì n là một số chẵn và do đó cần tối thiểu n-1 ngày.

Lịch thi đấu là một bảng n dòng và n-1 cột. Các dòng được đánh số từ 1 đến n và các cột được đánh số từ 1 đến n-1, trong đó dòng i biểu diễn cho đấu thủ i, cột j biểu diễn cho ngày thi đấu j và ô(i,j) ghi đấu thủ phải thi đấu với đấu thủ i trong ngày j.

Chiến lược chia để trị xây dựng lịch thi đấu như sau: Để sắp lịch cho n đấu thủ, ta sẽ sắp lịch cho $n/2$ đấu thủ, để sắp lịch cho $n/2$ đấu thủ, ta sẽ sắp lịch cho $n/4$ đấu thủ... Quá trình này sẽ dẫn đến bài toán cơ sở là sắp lịch thi đấu cho 2 đấu thủ. Hai đấu thủ này sẽ thi đấu một trận trong một ngày, lịch thi đấu cho họ thật dễ sắp. Khó khăn chính là ở chỗ từ các lịch thi đấu cho hai đấu thủ, ta tổng hợp lại để được lịch thi đấu của 4 đấu thủ, 8 đấu thủ, ...

Xuất phát từ lịch thi đấu cho hai đấu thủ ta có thể xây dựng lịch thi đấu cho 4 đấu thủ như sau: Lịch thi đấu cho 4 đấu thủ sẽ là một bảng 4 dòng, 3 cột. Lịch thi đấu cho 2 đấu thủ 1 và 2 trong ngày thứ 1 chính là lịch thi đấu của hai đấu thủ (bài toán cơ sở). Như vậy ta có $\hat{O}(1,1) = "2"$ và $\hat{O}(2,1) = "1"$. Tương tự ta có lịch thi đấu cho 2 đấu thủ 3 và 4 trong ngày thứ 1. Nghĩa là $\hat{O}(3,1) = "4"$ và $\hat{O}(4,1) = "3"$. (Ta có thể thấy rằng $\hat{O}(3,1) = \hat{O}(1,1) + 2$ và $\hat{O}(4,1) = \hat{O}(2,1) + 2$). Bây giờ để hoàn thành lịch thi đấu cho 4 đấu thủ, ta lấy góc trên bên trái của bảng lắp vào cho góc dưới bên phải và lấy góc dưới bên trái lắp cho góc trên bên phải.

Lịch thi đấu cho 8 đấu thủ là một bảng gồm 8 dòng, 7 cột. Góc trên bên trái chính là lịch thi đấu trong 3 ngày đầu của 4 đấu thủ từ 1 đến 4. Các ô của góc dưới bên trái sẽ bằng các ô tương ứng của góc trên bên trái cộng với 4. Đây chính là lịch thi đấu cho 4 đấu thủ 5, 6, 7 và 8 trong 3 ngày đầu. Bây giờ chúng ta hoàn thành việc sắp lịch bằng cách lắp đầy góc dưới bên phải bởi góc trên bên trái và góc trên bên phải bởi góc dưới bên trái.



Hình 3-1: Lịch thi đấu của 2, 4 và 8 đấu thủ

3.2.5 Bài toán con cân bằng (Balancing Subproblems)

Đối với kĩ thuật chia để trị, nói chung sẽ tốt hơn nếu ta chia bài toán cần giải thành các bài toán con có kích thước gần bằng nhau. Ví dụ, sắp xếp trộn (MergeSort) phân chia bài toán thành hai bài toán con có cùng kích thước $n/2$ và do đó thời gian của nó chỉ là $O(n \log n)$. Ngược lại trong trường hợp xấu nhất của QuickSort, khi mảng bị phân hoạch lệch thì thời gian thực hiện là $O(n^2)$.

Nguyên tắc chung là chúng ta tìm cách chia bài toán thành các bài toán con có kích thước xấp xỉ bằng nhau thì hiệu suất sẽ cao hơn.

3.3 KỸ THUẬT “THAM ĂN”

3.3.1 Bài toán tối ưu tổ hợp

Là một dạng của bài toán tối ưu, nó có dạng tổng quát như sau:

- Cho hàm $f(X)$ = xác định trên một tập hữu hạn các phần tử D . Hàm $f(X)$ được gọi là hàm mục tiêu.
- Mỗi phần tử $X \in D$ có dạng $X = (x_1, x_2, \dots, x_n)$ được gọi là một phương án.
- Cần tìm một phương án $X \in D$ sao cho hàm $f(X)$ đạt min (max). Phương án X như thế được gọi là phương án tối ưu.

Ta có thể tìm thấy phương án tối ưu bằng phương pháp “vét cạn” nghĩa là xét tất cả các phương án trong tập D (hữu hạn) để xác định phương án tốt nhất. Mặc dù tập hợp D là hữu hạn nhưng để tìm phương án tối ưu cho một bài toán kích thước n bằng phương pháp “vét cạn” ta có thể cần một thời gian mũ.

Các phần tiếp theo của chương này sẽ trình bày một số kĩ thuật giải bài toán tối ưu tổ hợp mà thời gian có thể chấp nhận được.

3.3.2 Nội dung kĩ thuật tham ăn

Tham ăn hiểu một cách dân gian là: trong một mâm có nhiều món ăn, món nào ngon nhất ta sẽ ăn trước và ăn cho hết món đó thì chuyển sang món ngon thứ hai, lại ăn hết món ngon thứ hai này và chuyển sang món ngon thứ ba...

Kĩ thuật tham ăn thường được vận dụng để giải bài toán tối ưu tổ hợp bằng cách xây dựng một phương án X . Phương án X được xây dựng bằng cách lựa chọn từng thành phần X_i của X cho đến khi hoàn chỉnh (đủ n thành phần). Với mỗi X_i , ta sẽ chọn X_i tối ưu. Với cách này thì có thể ở bước cuối cùng ta không còn gì để chọn mà phải chấp nhận một giá trị cuối cùng còn lại.

Áp dụng kĩ thuật tham ăn sẽ cho một giải thuật thời gian đa thức, tuy nhiên nói chung chúng ta chỉ đạt được **một phương án tốt chứ chưa hẳn là tối ưu**.

Có rất nhiều bài toán mà ta có thể giải bằng kĩ thuật này, sau đây là một số ví dụ.

3.3.3 Bài toán trả tiền của máy rút tiền tự động ATM.

Trong máy rút tiền tự động ATM, ngân hàng đã chuẩn bị sẵn các loại tiền có mệnh giá 100.000 đồng, 50.000 đồng, 20.000 đồng và 10.000 đồng. Giả sử mỗi loại tiền đều có số lượng không hạn chế. Khi có một khách hàng cần rút một số tiền n đồng (tính chẵn đến 10.000 đồng, tức là n chia hết cho 10000). Hãy tìm một phương án trả tiền sao cho trả đủ n đồng và số tờ giấy bạc phải trả là ít nhất.

Gọi $X = (X_1, X_2, X_3, X_4)$ là một phương án trả tiền, trong đó X_1 là số tờ giấy bạc mệnh giá 100.000 đồng, X_2 là số tờ giấy bạc mệnh giá 50.000 đồng, X_3 là số tờ giấy bạc mệnh giá 20.000 đồng và X_4 là số tờ giấy bạc mệnh giá 10.000 đồng. Theo yêu cầu ta phải có $X_1 + X_2 + X_3 + X_4$ nhỏ nhất và $X_1 * 100.000 + X_2 * 50.000 + X_3 * 20.000 + X_4 * 10.000 = n$.

Áp dụng kĩ thuật tham ăn để giải bài toán này là: để có số tờ giấy bạc phải trả ($X1 + X2 + X3 + X4$) nhỏ nhất thì các tờ giấy bạc mệnh giá lớn phải được chọn nhiều nhất.

Trước hết ta chọn tối đa các tờ giấy bạc mệnh giá 100.000 đồng, nghĩa là $X1$ là số nguyên lớn nhất sao cho $X1 * 100.000 \leq n$. Tức là $X1 = n \text{ DIV } 100.000$.

Xác định số tiền cần rút còn lại là hiệu $n - X1 * 100000$ và chuyển sang chọn loại giấy bạc 50.000 đồng...

Ví dụ khách hàng cần rút 1.290.000 đồng ($n = 1290000$), phương án trả tiền như sau:

$$X1 = 1290000 \text{ DIV } 100000 = 12.$$

$$\text{Số tiền cần rút còn lại là } 1290000 - 12 * 100000 = 90000.$$

$$X2 = 90000 \text{ DIV } 50000 = 1.$$

$$\text{Số tiền cần rút còn lại là } 90000 - 1 * 50000 = 40000.$$

$$X3 = 40000 \text{ DIV } 20000 = 2.$$

$$\text{Số tiền cần rút còn lại là } 40000 - 2 * 20000 = 0.$$

$$X4 = 0 \text{ DIV } 10000 = 0.$$

Ta có $X = (12, 1, 2, 0)$, tức là máy ATM sẽ trả cho khách hàng 12 tờ 100.000 đồng, 1 tờ 50.000 đồng và 2 tờ 20.000 đồng.

3.3.4 Bài toán đường đi của người giao hàng

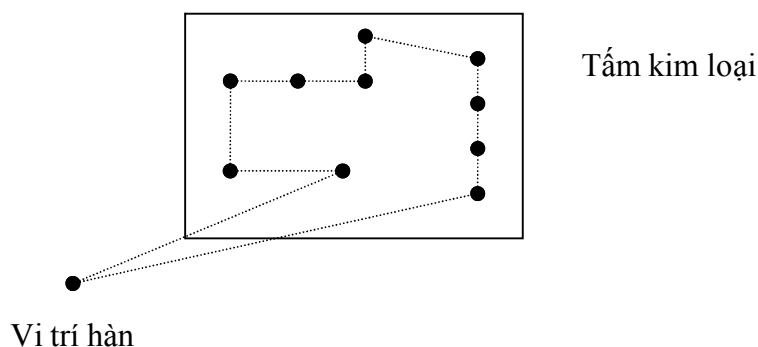


Chúng ta sẽ xét một bài toán rất nổi tiếng có tên là bài toán tìm đường đi của người giao hàng (TSP - Traveling Salesman Problem): Có một người giao hàng cần đi giao hàng tại n thành phố. Xuất phát từ một thành phố nào đó, đi qua các thành phố khác để giao hàng và trở về thành phố ban đầu. Mỗi thành phố chỉ đến một lần, khoảng cách từ một thành phố đến các thành phố khác là xác định được. Giả thiết rằng mỗi thành phố đều có đường đi đến các thành phố còn lại. Khoảng cách giữa hai thành phố có thể là khoảng cách địa lý, có thể là cước phí di chuyển hoặc thời gian di chuyển. Ta gọi chung là độ dài. Hãy tìm một chu trình (một đường đi khép kín thỏa mãn điều kiện trên) sao cho tổng độ dài các cạnh là nhỏ nhất. Hay còn nói là tìm một phương án có giá nhỏ nhất. Bài toán này cũng được gọi là bài toán người du lịch.

Một cách tổng quát, có thể không tồn tại một đường đi giữa hai thành phố a và b nào đó. Trong trường hợp đó ta cho một đường đi ảo giữa a và b với độ dài bằng ∞ .

Bài toán có thể biểu diễn bởi một đồ thị vô hướng có trọng số $G = (V, E)$, trong đó mỗi thành phố được biểu diễn bởi một đỉnh, cạnh nối hai đỉnh biểu diễn cho đường đi giữa hai thành phố và trọng số của cạnh là khoảng cách giữa hai thành phố. Một chu trình đi qua tất cả các đỉnh của G , mỗi đỉnh một lần duy nhất, được gọi là chu trình Hamilton. Vấn đề là tìm một chu trình Hamilton mà tổng độ dài các cạnh là nhỏ nhất.

Bài toán này có những ứng dụng rất quan trọng. Thí dụ một máy hàn các điểm được điều khiển bởi máy tính. Nhiệm vụ của nó là hàn một số điểm dự định ở trên một tấm kim loại. Người thợ hàn bắt đầu từ một điểm bên ngoài tấm kim loại và kết thúc tại chính điểm này, do đó tấm kim loại phải được di chuyển để điểm cần hàn được đưa vào vị trí hàn (tương tự như ta đưa tấm vải vào đầu mũi kim của máy khâu). Cần phải tìm một phương án di chuyển tấm kim loại sao cho việc di chuyển ít nhất. Hình ảnh sau cho chúng ta hình dung về bài toán đặt ra.



Hình 3-2: Hàn các điểm trên một tấm kim loại

Dễ dàng thấy rằng, có thể áp dụng bài toán đường đi của người giao hàng để giải bài toán này.

Với phương pháp vét cạn ta xét tất cả các chu trình, mỗi chu trình tính tổng độ dài các cạnh của nó rồi chọn một chu trình có tổng độ dài nhỏ nhất. Tuy nhiên chúng ta cần xét tất cả là $\frac{(n-1)!}{2}$ chu trình. Thực vậy, do mỗi chu trình đều đi qua tất cả các đỉnh (thành phố) nên ta có thể cố định một đỉnh. Từ đỉnh này ta có $n-1$ cạnh tới $n-1$ đỉnh khác, nên ta có $n-1$ cách chọn cạnh đầu tiên của chu trình. Sau khi đã chọn được cạnh đầu tiên, chúng ta còn $n-2$ cách chọn cạnh thứ hai, do đó ta có $(n-1)(n-2)$ cách chọn hai cạnh. Cứ lý luận như vậy ta sẽ thấy có $(n-1)!$ cách chọn một chu trình. Tuy nhiên với mỗi chu trình ta chỉ quan tâm đến tổng độ dài các cạnh chứ không quan tâm đến hướng đi theo chiều dương hay âm vì vậy có tất cả $\frac{(n-1)!}{2}$ phương án. Đó là một giải thuật thời gian mũ!

Kĩ thuật tham ăn áp dụng vào đây là:

1. Sắp xếp các cạnh theo thứ tự tăng của độ dài.
2. Xét các cạnh có độ dài từ nhỏ đến lớn để đưa vào chu trình.
3. Một cạnh sẽ được đưa vào chu trình nếu cạnh đó thỏa mãn hai điều kiện sau:
 - Không tạo thành một chu trình thiếu (không đi qua đủ n đỉnh)
 - Không tạo thành một đỉnh có cấp ≥ 3 (tức là không được có nhiều hơn hai cạnh xuất phát từ một đỉnh, do yêu cầu của bài toán là mỗi thành phố chỉ được đến một lần: một lần đến và một lần đi)

4. Lặp lại bước 3 cho đến khi xây dựng được một chu trình.

Với kĩ thuật này ta chỉ cần $n(n-1)/2$ phép chọn nên ta có một giải thuật cần $O(n^2)$ thời gian.

Ví dụ 3-1: Cho bài toán TSP với 6 đỉnh được cho bởi các tọa độ như sau:

• c(1,7)

• d(15,7)

• b(4,3)

• e(15,4)

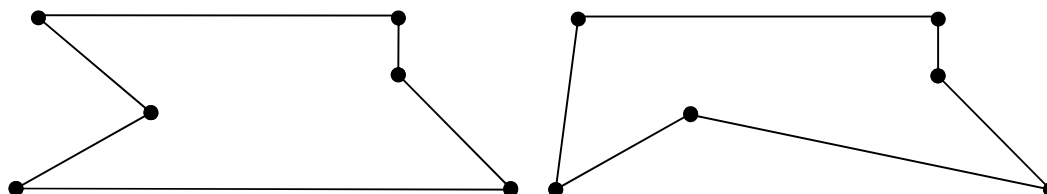
• a(0,0)

• f(18,0)

Hình 3-3: Sáu thành phố được cho bởi tọa độ

Do có 6 đỉnh nên có tất cả 15 cạnh. Đó là các cạnh: ab, ac, ad, ae, af, bc, bd, be, bf, cd, ce, cf, de, df và ef. Độ dài các cạnh ở đây là khoảng cách Euclide. Trong 15 cạnh này thì de = 3 là nhỏ nhất, nên de được chọn vào chu trình. Kế đến là 3 cạnh ab, bc và ef đều có độ dài là 5. Cả 3 cạnh đều thỏa mãn hai điều kiện nói trên, nên đều được chọn vào chu trình. Cạnh có độ dài nhỏ kế tiếp là ac = 7.08, nhưng không thể đưa cạnh này vào chu trình vì nó sẽ tạo ra chu trình thiếu (a-b-c-a). Cạnh df cũng bị loại vì lý do tương tự. Cạnh be được xem xét nhưng rồi cũng bị loại do tạo ra đỉnh b và đỉnh e có cấp 3. Tương tự chúng ta cũng loại bd. cd là cạnh tiếp theo được xét và được chọn. Cuối cùng ta có chu trình a-b-c-d-e-f-a với tổng độ dài là 50. Đây chỉ là một phương án tốt.

Phương án tối ưu là chu trình a-c-d-e-f-b-a với tổng độ dài là 48.39.



Hình 3-4: Phương án Greedy và phương án tối ưu

Giải thuật sơ bộ như sau:

PROCEDURE TSP;

BEGIN

{E là tập hợp các cạnh, Chu_trình là tập hợp các cạnh được chọn để đưa vào chu trình, mở đầu Chu_trình rỗng}

{Sắp xếp các cạnh trong E theo thứ tự tăng của độ dài}

Chu_Trình := Φ ;

Gia := 0.0;

WHILE E $\neq \Phi$ DO BEGIN

IF cạnh e có thể chọn THEN BEGIN

Chu_Trình := Chu_Trình + [e] ;

Gia := Gia + độ dài của e;

```

END;
E := E - [e];

END;
END;

```

Một cách tiếp cận khác của kĩ thuật tham ăn vào bài toán này là:

1. Xuất phát từ một đỉnh bất kỳ, chọn một cạnh có độ dài nhỏ nhất trong tất cả các cạnh đi ra từ đỉnh đó để đến đỉnh kế tiếp.
2. Từ đỉnh kế tiếp ta lại chọn một cạnh có độ dài nhỏ nhất đi ra từ đỉnh này thoả mãn hai điều kiện nói trên để đi đến đỉnh kế tiếp.
3. Lặp lại bước 2 cho đến khi đi tới đỉnh n thì quay trở về đỉnh xuất phát.

3.3.5 Bài toán cái ba lô



Cho một cái ba lô có thể đựng một trọng lượng W và n loại đồ vật, mỗi đồ vật i có một trọng lượng g_i và một giá trị v_i . Tất cả các loại đồ vật đều có số lượng không hạn chế. Tìm một cách lựa chọn các đồ vật đựng vào ba lô, chọn các loại đồ vật nào, mỗi loại lấy bao nhiêu sao cho tổng trọng lượng không vượt quá W và tổng giá trị là lớn nhất.

Theo yêu cầu của bài toán thì ta cần những đồ vật có giá trị cao mà trọng lượng lại nhỏ để sao cho có thể mang được nhiều “đồ quý”, sẽ là hợp lý khi ta quan tâm đến yếu tố “đơn giá” của từng loại đồ vật tức là tỷ lệ giá trị/trọng lượng. Đơn giá càng cao thì đồ càng quý. Từ đó ta có kĩ thuật greedy áp dụng cho bài toán này là:

1. Tính đơn giá cho các loại đồ vật.
2. Xét các loại đồ vật theo thứ tự đơn giá từ lớn đến nhỏ.
3. Với mỗi đồ vật được xét sẽ lấy một số lượng tối đa mà trọng lượng còn lại của ba lô cho phép.
4. Xác định trọng lượng còn lại của ba lô và quay lại bước 3 cho đến khi không còn có thể chọn được đồ vật nào nữa.

Ví dụ 3-2: Ta có một ba lô có trọng lượng là 37 và 4 loại đồ vật với trọng lượng và giá trị tương ứng được cho trong bảng bên.

Loại đồ vật	Trọng lượng	Giá trị
A	15	30
B	10	25
C	2	2
D	4	6

Từ bảng đã cho ta tính đơn giá cho các loại đồ vật và sắp xếp các loại đồ vật này theo thứ tự đơn giá giảm dần ta có bảng sau.

Loại đồ vật	Trọng lượng	Giá trị	Đơn giá
B	10	25	2.5
A	15	30	2.0
D	4	6	1.5
C	2	2	1.0

Theo đó thì thứ tự ưu tiên để chọn đồ vật là B, A, D và cuối cùng là C.

Vật B được xét đầu tiên và ta chọn tối đa 3 cái vì mỗi cái có trọng lượng mỗi cái là 10 và ba lô có trọng lượng 37. Sau khi đã chọn 3 vật loại B, trọng lượng còn lại trong ba lô là $37 - 3 \cdot 10 = 7$. Ta xét đến vật A, vì A có trọng lượng 15 mà trọng lượng còn lại của ba lô chỉ còn 7 nên không thể chọn vật A. Xét vật D và ta thấy có thể chọn 1 vật D, khi đó trọng lượng còn lại của ba lô là $7 - 4 = 3$. Cuối cùng ta chọn được một vật C.

Như vậy chúng ta đã chọn 3 cái loại B, một cái loại D và 1 cái loại C. Tổng trọng lượng là $3 \cdot 10 + 1 \cdot 4 + 1 \cdot 2 = 36$ và tổng giá trị là $3 \cdot 25 + 1 \cdot 6 + 1 \cdot 2 = 83$.

Giải thuật thô giải bài toán cái ba lô bằng kỹ thuật tham ăn như sau:

Tổ chức dữ liệu:

- Mỗi đồ vật được biểu diễn bởi một mẫu tin có các trường:
 - Ten: Lưu trữ tên đồ vật.
 - Trong_luong: Lưu trữ trọng lượng của đồ vật.
 - Gia_tri: Lưu trữ giá trị của đồ vật
 - Don_gia: Lưu trữ đơn giá của đồ vật
 - Phuong_an: Lưu trữ số lượng đồ vật được chọn theo phương án.
- Danh sách các đồ vật được biểu diễn bởi một mảng các đồ vật.

Khai báo bằng pascal:

```
Type
Do_vat = Record
    Ten: String[20]
    Trong_luong, Gia_tri, Don_gia : Real;
    Phuong_an : Integer;
End;
Danh_sach_do_vat = ARRAY[1..n] OF do_vat;

Procedure Greedy (VAR dsdv : Danh_sach_do_vat; W: real);
VAR i: integer;
BEGIN
    {Sắp xếp mảng dsdv theo thứ tự giảm của don_gia}
    FOR i:=1 TO n DO BEGIN
        Dsdv[i].Phuong_an:= Chon(dsdv[i].Trong_luong, W);
        W := W - dsdv[i].phuong_an * dsdv[i].Trong_luong;
    END;
END;
```

Trong đó hàm Chon(trong_luong, W) nhận vào trọng lượng trong_luong của một vật và trọng lượng còn lại W của ba lô, trả về số lượng đồ vật được chọn, sao cho tổng trọng lượng của các vật được chọn không lớn hơn W. Nói riêng, trong trường hợp trong_luong và W là hai số nguyên thì Chon(Trong_luong, W) chính là $W \text{ DIV } \text{Trong_luong}$.

Chú ý: Có một số biến thể của bài toán cái ba lô như sau:

1. Mỗi đồ vật i chỉ có một số lượng s_i . Với bài toán này khi lựa chọn vật i ta không được lấy một số lượng vượt quá s_i .
2. Mỗi đồ vật chỉ có một cái. Với bài toán này thì với mỗi đồ vật ta chỉ có thể chọn hoặc không chọn.

3.4 QUY HOẠCH ĐỘNG

3.4.1 Nội dung kỹ thuật

Như trong 3.1 đã nói, kỹ thuật chia để trị thường dẫn chúng ta tới một giải thuật đệ quy. Trong các giải thuật đó, có thể có một số giải thuật có độ phức tạp thời gian mũ. Tuy nhiên, thường chỉ có một số đa thức các bài toán con, điều đó có nghĩa là chúng ta đã phải giải một số bài toán con nào đó nhiều lần. Để tránh việc giải dư thừa một số bài toán con, chúng ta tạo ra một bảng để lưu trữ kết quả của các bài toán con và khi cần chúng ta sẽ sử dụng kết quả đã được lưu trong bảng mà không cần phải giải lại bài toán đó. Lấp đầy bảng kết quả các bài toán con theo một quy luật nào đó để nhận được kết quả của bài toán ban đầu (cũng đã được lưu trong một số ô nào đó của bảng) được gọi là quy hoạch động (dynamic programming). Trong một số trường hợp, để tiết kiệm ô nhớ, thay vì dùng một bảng, ta chỉ dùng một vectơ.

Có thể tóm tắt giải thuật quy hoạch động như sau:

1. Tạo bảng bằng cách:
 - a. Gán giá trị cho một số ô nào đó.
 - b. Gán trị cho các ô khác nhờ vào giá trị của các ô trước đó.
2. Tra bảng và xác định kết quả của bài toán ban đầu.

Ưu điểm của phương pháp quy hoạch động là chương trình thực hiện nhanh do không phải tốn thời gian giải lại một bài toán con đã được giải.

Kỹ thuật quy hoạch động có thể vận dụng để giải các bài toán tối ưu, các bài toán có công thức truy hồi.

Phương pháp quy hoạch động sẽ không đem lại hiệu quả trong các trường hợp sau:

- Không tìm được công thức truy hồi.
- Số lượng các bài toán con cần giải quyết và lưu giữ kết quả là rất lớn.
- Sự kết hợp lời giải của các bài toán con chưa chắc cho ta lời giải của bài toán ban đầu.

Sau đây chúng ta sẽ trình bày một số bài toán có thể giải bằng kỹ thuật quy hoạch động.

3.4.2 Bài toán tính số tổ hợp

Một bài toán khá quen thuộc là tính số tổ hợp chập k của n theo công thức truy hồi:

$$C_n^k = \begin{cases} 1 & \text{nếu } k=0 \text{ hoặc } k=n \\ C_{n-1}^{k-1} + C_{n-1}^k & \text{nếu } 0 < k < n \end{cases}$$

Công thức trên đã gợi ý cho chúng ta một giải thuật đệ quy như sau:

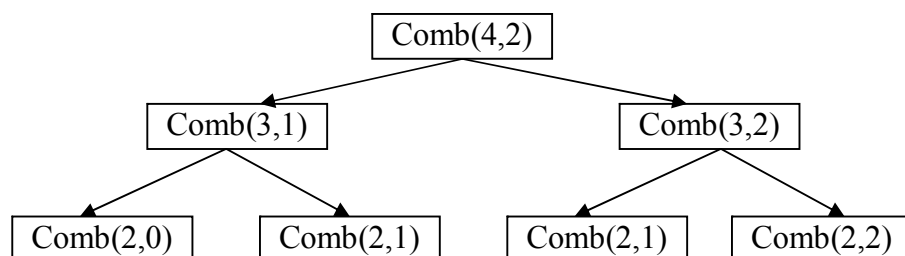
```
FUNCTION Comb(n, k : integer) : Integer;
BEGIN
    IF (k=0) OR (k=n) THEN Comb := 1
    ELSE Comb := Comb(n-1, k-1) + Comb(n-1, k);
END;
```

Gọi $T(n)$ là thời gian để tính số tổ hợp chập k của n , thì ta có phương trình đệ quy:

$$T(1) = C1 \text{ và } T(n) = 2T(n-1) + C2$$

Giải phương trình này ta được $T(n) = O(2^n)$, như vậy là một giải thuật thời gian mũ, trong khi chỉ có một đa thức các bài toán con. Điều đó chứng tỏ rằng có những bài toán con được giải nhiều lần.

Chẳng hạn để tính $\text{Comb}(4,2)$ ta phải tính $\text{Comb}(3,1)$ và $\text{Comb}(3,2)$. Để tính $\text{Comb}(3,1)$ ta phải tính $\text{Comb}(2,0)$ và $\text{Comb}(2,1)$. Để tính $\text{Comb}(3,2)$ ta phải tính $\text{Comb}(2,1)$ và $\text{Comb}(2,2)$. Như vậy để tính $\text{Comb}(4,2)$ ta phải tính $\text{Comb}(2,1)$ hai lần. Hình sau minh họa rõ điều đó.



Hình 3-5 : Sơ đồ gọi thực hiện $\text{Comb}(4,2)$

Áp dụng kĩ thuật quy hoạch động để khắc phục tình trạng trên, ta xây dựng một bảng gồm $n+1$ dòng (từ 0 đến n) và $n+1$ cột (từ 0 đến n) và điền giá trị cho $O(i,j)$ theo quy tắc sau: (Quy tắc tam giác Pascal):

$$O(0,0) = 1;$$

$$O(i,0) = 1;$$

$$O(i,i) = 1 \text{ với } 0 < i < n;$$

$$O(i,j) = O(i-1,j-1) + O(i-1,j) \text{ với } 0 < j < i < n.$$

Chẳng hạn với $n = 4$ ta có bảng bên.

$O(n,k)$ chính là $\text{Comb}(n,k)$ và ta có giải thuật như sau:

$\begin{smallmatrix} j \\ i \end{smallmatrix}$	0	1	2	3	4
0	1				
1	1	1			
2	1	2	1		
3	1	3	3	1	
4	1	4	6	4	1

Tam giác Pascal

```
FUNCTION Comb(n, k : Integer) : Integer
VAR C: array[0..n, 0..n] of integer;
    i, j : integer;
BEGIN
```

```

{1}  C[0,0] := 1;
{2}  FOR i := 1 TO n DO BEGIN
{3}      C[i,0] := 1;
{4}      C[i,i] := 1;
{5}      FOR j := 1 TO i-1 DO C[i,j]:=C[i-1,j-1]+C[i-1,j];
      END;
{6}  Comb := C[n,k];
END;

```

Vòng lặp {5} thực hiện $i-1$ lần, mỗi lần $O(1)$. Vòng lặp {2} có i chạy từ 1 đến n , nên nếu gọi $T(n)$ là thời gian thực hiện giải thuật thì ta có:

$$T(n) = \sum_{i=1}^n (i-1) = \frac{n(n-1)}{2} = O(n^2)$$

Nhận xét: Thông qua việc xác định độ phức tạp, ta thấy rõ ràng giải thuật quy hoạch động hiệu quả hơn nhiều so với giải thuật đệ qui ($n^2 < 2^n$). Tuy nhiên việc sử dụng bảng (mảng hai chiều) như trên còn lãng phí ô nhớ, do đó ta sẽ cải tiến thêm một bước bằng cách sử dụng vectơ (mảng một chiều) để lưu trữ kết quả trung gian. Cách làm cụ thể như sau:

Ta sẽ dùng một vectơ V có $n+1$ phần tử từ $V[0]$ đến $V[n]$. Vectơ V sẽ lưu trữ các giá trị tương ứng với dòng i trong tam giác Pascal ở trên. Trong đó $V[j]$ lưu trữ giá trị số tổ hợp chập j của i (C_i^j) ($j = 0$ đến i). Dĩ nhiên do chỉ có một vectơ V mà phải lưu trữ nhiều dòng i do đó tại mỗi bước, V chỉ lưu trữ được một dòng và ở bước cuối cùng, V lưu trữ các giá trị ứng với $i = n$, trong đó $V[k]$ chính là C_n^k .

Khởi đầu, ứng với $i=1$, ta cho $V[0] = 1$ và $V[1] = 1$. Tức là $C_1^0 = 1$ và $C_1^1 = 1$. Với các giá trị i từ 2 đến n , ta thực hiện như sau:

- $V[0]$ được gán giá trị 1 tức là $C_i^0 = 1$. Tuy nhiên giá trị $V[0] = 1$ đã được gán ở trên, không cần phải gán lại.
- Với j từ 1 đến $i-1$, ta vẫn áp dụng công thức $C_i^j = C_{i-1}^{j-1} + C_{i-1}^j$. Nghĩa là để tính các giá trị trong dòng i ta phải dựa vào dòng $i-1$. Tuy nhiên do chỉ có một vectơ V và lúc này nó sẽ lưu trữ các giá trị của dòng i , tức là dòng $i-1$ sẽ không còn. Để khắc phục điều này ta dùng thêm hai biến trung gian $p1$ và $p2$. Trong đó $p1$ dùng để lưu trữ C_{i-1}^{j-1} và $p2$ dùng để lưu trữ C_{i-1}^j . Khởi đầu $p1$ được gán $V[0]$ tức là C_{i-1}^0 và $p2$ được gán $V[j]$ tức là C_{i-1}^j , $V[j]$ lưu trữ giá trị C_i^j sẽ được gán bởi $p1+p2$, sau đó $p1$ được gán bởi $p2$, nghĩa là khi j tăng lên 1 đơn vị thành $j+1$ thì $p1$ là C_{i-1}^j và nó được dùng để tính C_i^{j+1} .
- Cuối cùng với $j = i$ ta gán $V[i]$ giá trị 1 tức là $C_i^i = 1$.

Giải thuật cụ thể như sau:

```

FUNCTION Comb(n, k : Integer) : Integer
VAR  V: array[0..n] of integer;
      i, j : integer;
      p1, p2: integer;
BEGIN
{1}  V[0] := 1;

```



```

{2}  V[1] := 1;
{3}  FOR i := 2 TO n DO BEGIN
{4}      p1 := V[0];
{5}      FOR j := 1 TO i-1 DO BEGIN
{6}          p2 := V[j];
{7}          V[j] := p1+p2;
{8}          p1 := p2;
      END;
{9}      V[i] := 1;
      END;
{10} Comb := V[k];
END;

```

Dễ dàng tính được độ phức tạp của giải thuật vẫn là $O(n^2)$.

3.4.3 Bài toán cái ba lô

Sử dụng kỹ thuật quy hoạch động để giải bài toán cái ba lô đã trình bày trong mục 3.2.5 với một lưu ý là các số liệu đều cho dưới dạng **số nguyên**.

Giả sử $X[k,V]$ là số lượng đồ vật k được chọn, $F[k,V]$ là tổng giá trị của k đồ vật đã được chọn và V là trọng lượng còn lại của ba lô, $k = 1..n$, $V = 1..W$.

Trong trường hợp đơn giản nhất, khi chỉ có một đồ vật, ta tính được $X[1,V]$ và $F[1,V]$ với mọi V từ 1 đến W như sau:

$$X[1,V] = V \text{ DIV } g_1 \text{ và } F[1,V] = X[1,V] * v_1.$$

Giả sử ta đã tính được $F[k-1,V]$, khi có thêm đồ vật thứ k , ta sẽ tính được $F[k,V]$, với mọi V từ 1 đến W . Cách tính như sau: Nếu ta chọn x_k đồ vật loại k , thì trọng lượng còn lại của ba lô dành cho $k-1$ đồ vật từ 1 đến $k-1$ là $U = V - x_k * g_k$ và tổng giá trị của k loại đồ vật đã được chọn $F[k,V] = F[k-1,U] + x_k * v_k$, với x_k thay đổi từ 0 đến $y_k = V \text{ DIV } g_k$ và ta sẽ chọn x_k sao cho $F[k,V]$ lớn nhất.

Ta có công thức truy hồi như sau:

$$X[1,V] = V \text{ DIV } g_1 \text{ và } F[1,V] = X[1,V] * v_1.$$

$$F[k,V] = \text{Max}(F[k-1, V - x_k * g_k] + x_k * v_k) \text{ với } x_k \text{ chạy từ } 0 \text{ đến } V \text{ DIV } g_k.$$

Sau khi xác định được $F[k,V]$ thì $X[k,V]$ là x_k ứng với giá trị $F[k,V]$ được chọn trong công thức trên.

Để lưu các giá trị trung gian trong quá trình tính $F[k,V]$ theo công thức truy hồi trên, ta sử dụng một bảng gồm n dòng từ 1 đến n , dòng thứ k ứng với đồ vật loại k và $W+1$ cột từ 0 đến W , cột thứ V ứng với trọng lượng V . Mỗi cột V bao gồm hai cột nhỏ, cột bên trái lưu $F[k,V]$, cột bên phải lưu $X[k,V]$. Trong lập trình ta sẽ tổ chức hai bảng tách rời là F và X .

Ví dụ bài toán cái ba lô với trọng lượng $W=9$, và 5 loại đồ vật được cho trong bảng sau

Đồ vật	Trọng lượng (gi)	Giá trị (vi)
--------	------------------	--------------

1	3	4
2	4	5
3	5	6
4	2	3
5	1	1

Ta có bảng $F[k,V]$ và $X[k,V]$ như sau, trong đó mỗi cột V có hai cột con, cột bên trái ghi $F[k,V]$ và cột bên phải ghi $X[k,V]$.

k \ v	0		1		2		3		4		5		6		7		8		9	
1	0	0	0	0	0	0	4	1	4	1	4	1	8	2	8	2	8	2	12	3
2	0	0	0	0	0	0	4	0	5	1	5	1	8	0	9	1	10	2	12	0
3	0	0	0	0	0	0	4	0	5	0	6	1	8	0	9	0	10	0	12	0
4	0	0	0	0	3	1	4	0	6	2	7	1	9	3	10	2	12	4	13	3
5	0	0	1	1	3	0	4	0	6	0	7	0	9	0	10	0	12	0	13	0

Trong bảng trên, việc điền giá trị cho dòng 1 rất đơn giản bằng cách sử dụng công thức: $X[1,V] = V \text{ DIV } g_1$ và $F[1,V] = X[1,V] * v_1$.

Từ dòng 2 đến dòng 5, phải sử dụng công thức truy hồi:

$F[k,V] = \text{Max}(F[k-1, V-x_k * g_k] + x_k * v_k)$ với x_k chạy từ 0 đến $V \text{ DIV } g_k$.

Ví dụ để tính $F[2,7]$, ta có x_k chạy từ 0 đến $V \text{ DIV } g_k$, trong trường hợp này là x_k chạy từ 0 đến $7 \text{ DIV } 4$, tức x_k có hai giá trị 0 và 1.

Khi đó $F[2,7] = \text{Max}(F[2-1, 7-0*4] + 0*5, F[2-1, 7-1*4] + 1*5)$
 $= \text{Max}(F[1,7], F[1,3] + 5) = \text{Max}(8, 4+5) = 9$.

$F[2,7] = 9$ ứng với $x_k = 1$ do đó $X[2,7] = 1$.

Vấn đề bây giờ là cần phải tra trong bảng trên để xác định phương án.

Khởi đầu, trọng lượng còn lại của ba lô $V = W$.

Xét các đồ vật từ n đến 1, với mỗi đồ vật k , ứng với trọng lượng còn lại V của ba lô, nếu $X[k,V] > 0$ thì chọn $X[k,V]$ đồ vật loại k . Tính lại $V = V - X[k,V] * g_k$.

Ví dụ, trong bảng trên, ta sẽ xét các đồ vật từ 5 đến 1. Khởi đầu $V = W = 9$.

Với $k = 5$, vì $X[5,9] = 0$ nên ta không chọn đồ vật loại 5.

Với $k = 4$, vì $X[4,9] = 3$ nên ta chọn 3 đồ vật loại 4. Tính lại $V = 9 - 3 * 2 = 3$.

Với $k = 3$, vì $X[3,3] = 0$ nên ta không chọn đồ vật loại 3.

Với $k = 2$, vì $X[2,3] = 0$ nên ta không chọn đồ vật loại 2.

Với $k = 1$, vì $X[1,3] = 1$ nên ta chọn 1 đồ vật loại 1. Tính lại $V = 3 - 1 * 3 = 0$.

Vậy tổng trọng lượng các vật được chọn là $3 * 2 + 1 * 3 = 9$. Tổng giá trị các vật được chọn là $3 * 3 + 1 * 4 = 13$.

Giải thuật thô theo kĩ thuật quy hoạch động như sau:

Tổ chức dữ liệu:

- Mỗi đồ vật được biểu diễn bởi một mẫu tin có các trường:
 - Ten: Lưu trữ tên đồ vật.
 - Trong_luong: Lưu trữ trọng lượng của đồ vật.
 - Gia_tri: Lưu trữ giá trị của đồ vật
 - Phuong_an: Lưu trữ số lượng đồ vật được chọn theo phương án.
- Danh sách các đồ vật được biểu diễn bởi một mảng các đồ vật.
- Bảng được biểu diễn bởi một mảng hai chiều các số nguyên để lưu trữ các giá trị $F[k,v]$ và $X[k,v]$.

Khai báo bằng pascal:

```
Type
Do_vat = Record
    Ten: String[20]
    Trong_luong, Gia_tri : integer;
    Phuong_an : Integer;
End;
Danh_sach_vat = ARRAY[1..MAX] OF do_vat;
BANG = ARRAY[1..10, 0..100] of integer;
```

Thủ tục tạo bảng nhận vào ds_vat là danh sách các vật, n là số lượng các loại vật, W là trọng lượng của ba lô. F và X là hai tham số thuộc kiểu Bang và được truyền bằng tham chiếu để nhận lại hai bảng F và X do thủ tục tạo ra.

```
PROCEDURE Tao_Bang (ds_vat:Danh_sach_vat;n,W: integer; VAR F,X: Bang);
VAR   xk, yk, k: integer;
FMax, XMax, v : integer;
BEGIN
FOR v:= 0 To W DO BEGIN {Hàng đầu tiên của hai bảng}
    X[1, v] := v div ds_vat[1].trong_luong;
    F[1, v] := X[1, v] * ds_vat[1].gia_tri;
END;
FOR k:= 2 TO N DO BEGIN
    X[k, 0] := 0;
    F[1, 0] := 0;
    For v:= 1 TO W DO BEGIN
        FMax := F[k-1, v] ;
        XMax := 0;
        yk := v DIV ds_vat[k].trong_luong;
        FOR xk:= 1 TO yk DO
            If (F[k-1,v-xk*ds_vat[k].trong_luong]+xk*ds_vat[k].gia_tri>FMax)
            THEN BEGIN
                FMax:=F[k-1,v-xk*ds_vat[k].trong_luong]+xk*ds_vat[k].gia_tri;
                XMax:= xk;
            END ;
        F[k, v] := FMax;
        X[k, v] := XMax;
    END;
END;
END;
```

Thủ tục Tra_bang nhận vào hai bảng F và X; n là số lượng các loại đồ vật, W là trọng lượng của ba lô và trả ra ds_vat là một danh sách đồ vật đã được xác định phương án. Tham số ds_vat được truyền bằng tham chiếu.

```

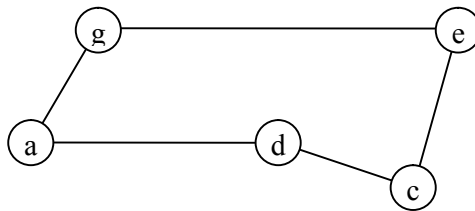
PROCEDURE Tra_Bang(VAR ds_vat:Danh_sach_vat;n,W:integer;F,X:
Bang);
VAR k, v: integer;
BEGIN
    v := W;
    FOR k:= n DOWNT0 1 DO
        IF X[k,v] > 0 THEN BEGIN
            ds_vat[k].Phuong_an := X[k,v];
            v := v - X[k, v] * ds_vat[k].trong_luong;
        END;
    END;
END;

```

3.4.4 Bài toán đường đi của người giao hàng

Chúng ta có thể áp dụng kĩ thuật quy hoạch động để giải bài toán TSP đã trình bày trong mục 3.2.4.

Đặt $S = \{x_1, x_2, \dots, x_k\}$ là tập hợp con các cạnh của đồ thị $G = (V, E)$. Ta nói rằng một đường đi P từ v đến w *phủ lên* S nếu $P = \{v, x_1, x_2, \dots, x_k, w\}$, trong đó x_i có thể xuất hiện ở một thứ tự bất kì, nhưng chỉ xuất hiện duy nhất một lần. Ví dụ đường cho trong hình sau, đi từ a đến a, phủ lên $\{c, d, e, g\}$.



Hình 3-6: Đường đi từ a đến a phủ lên $\{c, d, e, g\}$

Ta định nghĩa $d(v, w, S)$ là tổng độ dài của đường đi ngắn nhất từ v đến w, phủ lên S. Nếu không có một đường đi như vậy thì đặt $d(v, w, S) = \infty$. Một chu trình Hamilton nhỏ nhất $C_{\min}$ của G phải có tổng độ dài là $c(C_{\min}) = d(v, v, V - \{v\})$. Trong đó v là một đỉnh nào đó của V. Ta xác định $C_{\min}$ như sau:

Nếu $|V| = 1$ (G chỉ có một đỉnh) thì $c(C_{\min}) = 0$, ngược lại ta có công thức đệ qui để tính $d(v, w, S)$ là:

$$d(v, w, \{\}) = c(v, w)$$

$$d(v, w, S) = \min [c(v, x) + d(x, w, S - \{x\})], \text{ lấy với mọi } x \in S.$$

Trong đó $c(v, w)$ là độ dài của cạnh nối hai đỉnh v và w nếu nó tồn tại hoặc là ∞ nếu ngược lại. Dòng thứ hai trong công thức đệ qui trên ứng với tập S không rỗng, nó chỉ ra rằng đường đi ngắn nhất từ v đến w phủ lên S, trước hết phải đi đến một đỉnh x nào đó trong S và sau đó là đường đi ngắn nhất từ x đến w, phủ lên tập $S - \{x\}$.

Bằng cách lưu trữ các đỉnh x trong công thức đệ qui nói trên, chúng ta sẽ thu được một chu trình Hamilton tối tiểu.

3.5 KỸ THUẬT QUAY LUI

Kỹ thuật quay lui (backtracking) như tên gọi của nó, là một quá trình phân tích đi xuống và quay lui trở lại theo con đường đã đi qua. Tại mỗi bước phân tích chúng ta chưa giải quyết được vấn đề do còn thiếu dữ liệu nên cứ phải phân tích cho tới các điểm dừng, nơi chúng ta xác định được lời giải của chúng hoặc là xác định được là không thể (hoặc không nên) tiếp tục theo hướng này. Từ các điểm dừng này chúng ta quay ngược trở lại theo con đường mà chúng ta đã đi qua để giải quyết các vấn đề còn tồn đọng và cuối cùng ta sẽ giải quyết được vấn đề ban đầu.

Ở đây chúng ta sẽ xét 3 kỹ thuật quay lui: “vét cạn” là kỹ thuật phải đi tới tất cả các điểm dừng rồi mới quay lui. “Cắt tỉa Alpha-Beta” và “Nhánh-Cận” là hai kỹ thuật cho phép chúng ta không cần thiết phải đi tới tất cả các điểm dừng, mà chỉ cần đi đến một số điểm nào đó và dựa vào một số suy luận để có thể quay lui sớm. Các kỹ thuật này sẽ được trình bày thông qua một số bài toán cụ thể sau.

3.5.1 Định trị cây biểu thức số học

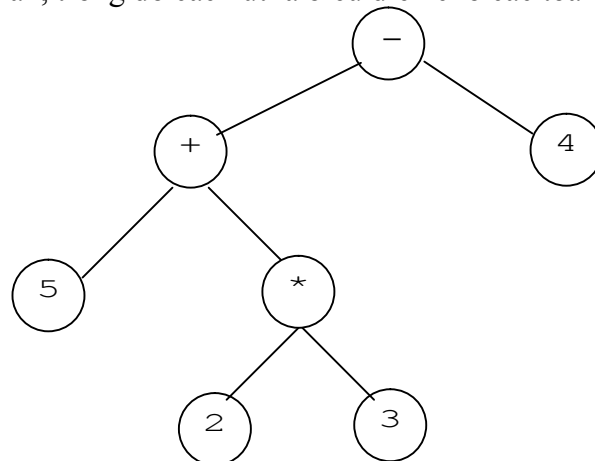
Trong các ngôn ngữ lập trình đều có các biểu thức số học, việc dịch các biểu thức này đòi hỏi phải đánh giá (định trị) chúng. Để làm được điều đó cần phải có một biểu diễn trung gian cho biểu thức. Một trong các biểu diễn trung gian cho biểu thức là cây biểu thức.

Cây biểu thức số học là một cây nhị phân, trong đó các nút lá biểu diễn cho các toán hạng, các nút trong biểu diễn cho các toán tử.

Ví dụ 3-3: Biểu thức $5 + 2 * 3 - 4$ sẽ được biểu diễn bởi cây trong hình 3-8

Trị của một nút lá chính là trị của toán hạng mà nút đó biểu diễn. Trị của một nút trong có được bằng cách lấy toán tử mà nút đó biểu diễn áp dụng vào các con của nó.

Trị của nút gốc chính là trị của biểu thức.



Hình 3-7: Một cây biểu thức số học

Như vậy để định trị cho nút gốc, chúng ta phải định trị cho hai con của nó, đối với mỗi con ta xem nó có phải là nút lá hay không, nếu không phải ta lại phải xét hai con của nút đó. Quá trình cứ tiếp tục như vậy cho tới khi gặp các nút lá mà giá trị của chúng đã được biết, quay lui để định trị cho các nút cha của các nút lá và cứ như thế mà định trị cho tổ tiên của chúng. Đó chính là kỹ thuật quay lui vét cạn, vì chúng ta phải lần đến tất cả các nút lá mới định trị được cho các nút trong và do thế mới định trị được cho nút gốc.

Ví dụ 3-4: Với cây biểu thức trong ví dụ 3-3. Để định trị cho nút - chúng ta phải định trị cho nút + và nút 4. Nút 4 là nút lá nên giá trị của nó là 4. Để định trị cho nút + ta phải định trị cho nút 5 và nút *. Nút 5 là nút lá nên giá trị của nó là 5. Để định trị cho nút *, ta phải định trị cho nút 2 và nút 3. Cả hai nút này đều là lá nên giá trị của chúng tương ứng là 2 và 3. Quay lui lại nút *, lấy toán tử * áp dụng cho hai con của nó là 2 và 3 ta được trị của nút * là 6. Quay lui về nút +, lại áp dụng toán tử + vào hai con của nó là 5 và 6 được trị của nút + là 11. Cuối cùng quay về nút -, áp dụng toán tử - vào hai con của nó là 11 và 4 ta được trị của nút - (nút gốc) là 7. Đó chính là trị của biểu thức. Trong hình 3-9i, mũi tên nét đứt minh họa quá trình đi tìm nút lá và mũi tên nét liền minh họa quá trình quay lui để định trị cho các nút, các số bên phải mỗi nút là trị của nút đó.

Giải thuật sơ bộ để định trị một nút bất kỳ như sau:

```
FUNCTION Eval(n : node): real;
BEGIN
    IF n là lá THEN RETURN (trị của toán hạng trong n)
    ELSE RETURN (Toán tử trong n (Eval (Con trái của n),
        Eval (Con phải của n)) );
END;
```

Muốn định trị cho cây biểu thức T, ta gọi Eval(ROOT(T)).

3.5.2 Kĩ thuật cắt tỉa Alpha-Beta

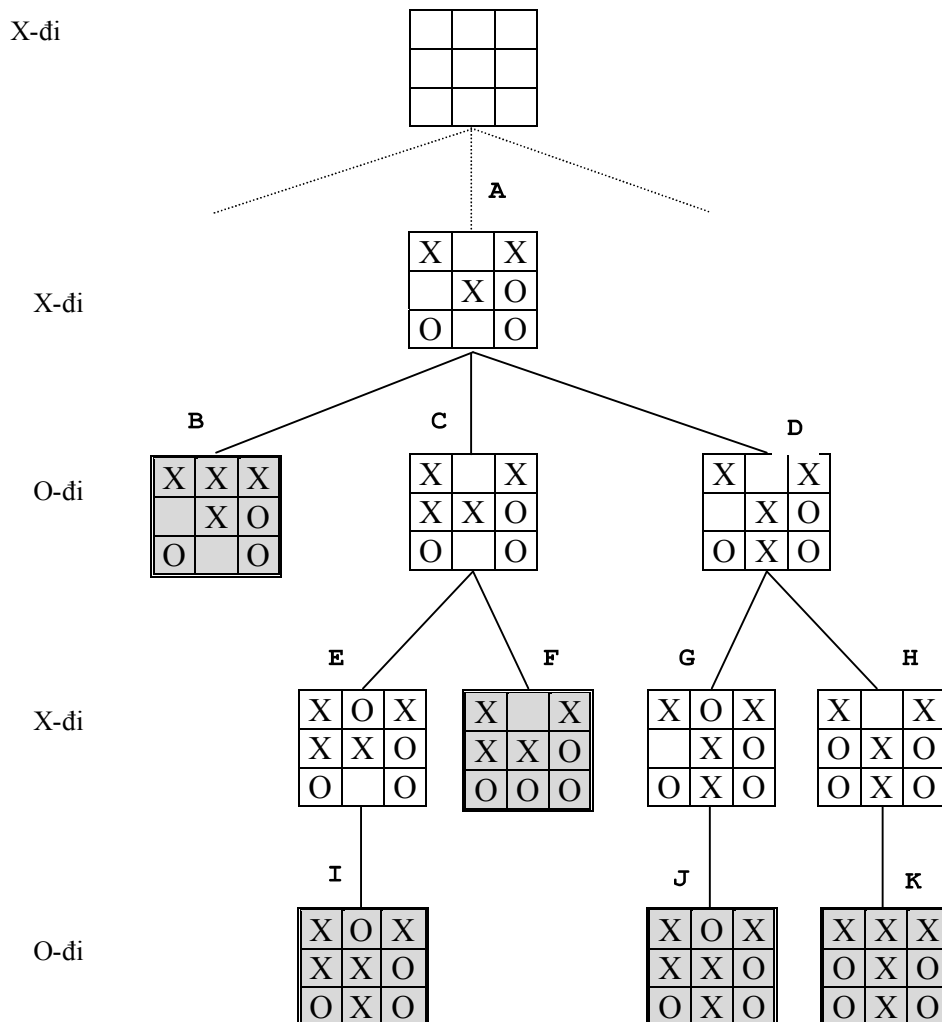
3.5.2.1 Cây trò chơi



Xét một trò chơi trong đó hai người thay phiên nhau đi nước của mình như cờ vua, cờ tướng, carô... Trò chơi có một trạng thái bắt đầu và mỗi nước đi sẽ biến đổi trạng thái hiện hành thành một trạng thái mới. Trò chơi sẽ kết thúc theo một quy định nào đó, theo đó thì cuộc chơi sẽ dẫn đến một trạng thái phản ánh có một người thắng cuộc hoặc một trạng thái mà cả hai đấu thủ không thể phát triển được nước đi của mình, ta gọi nó là trạng thái hòa cờ. Ta tìm cách phân tích xem từ một trạng thái nào đó sẽ dẫn đến đấu thủ nào sẽ thắng với điều kiện cả hai đấu thủ đều có trình độ như nhau.

Một trò chơi như vậy có thể được biểu diễn bởi một cây, gọi là cây trò chơi. Mỗi một nút của cây biểu diễn cho một trạng thái. Nút gốc biểu diễn cho trạng thái bắt đầu của cuộc chơi. Mỗi nút lá biểu diễn cho một trạng thái kết thúc của trò chơi (trạng thái thắng thua hoặc hòa). Nếu trạng thái x được biểu diễn bởi nút n thì các con của n biểu diễn cho tất cả các trạng thái kết quả của các nước đi có thể xuất phát từ trạng thái x.

Ví dụ 3-5: Xét trò chơi carô có 9 ô. Hai người thay phiên nhau đi X hoặc O. Người nào đi được 3 ô thẳng hàng (ngang, dọc, chéo) thì thắng cuộc. Nếu đã hết ô đi mà chưa phân thắng bại thì hai đấu thủ hòa nhau. Một phần của trò chơi này được biểu diễn bởi cây sau:



Hình 3-8: Một phần của cây trò chơi carô 9 ô

Trong cây trò chơi trên, các nút lá được tô nền và viền khung đôi để dễ phân biệt với các nút khác. Ta gán cho mỗi nút một chữ cái (A, B, C...) để tiện trong việc trình bày các giải thuật.

Ta có thể gán cho mỗi nút lá một giá trị để phản ánh trạng thái thắng thua hay hòa của các đấu thủ. Chẳng hạn ta gán cho nút lá các giá trị như sau:

- 1 nếu tại đó người đi X đã thắng,
- -1 nếu tại đó người đi X đã thua và
- 0 nếu hai đấu thủ đã hòa nhau.

Như vậy từ một trạng thái bất kỳ, đến lượt mình, người đi X sẽ chọn cho mình một nước đi sao cho dẫn đến trạng thái có giá trị lớn nhất (trong trường hợp này là 1). Ta nói X chọn nước đi MAX, nút mà từ đó X chọn nước đi của mình được gọi là nút MAX. Người đi O đến lượt mình sẽ chọn một nước đi sao cho dẫn đến trạng thái có giá trị nhỏ nhất (trong trường hợp này là -1, khi đó X sẽ thua và do đó O sẽ thắng). Ta nói O chọn nước đi MIN, nút mà từ đó O chọn nước đi của mình được

gọi là nút MIN. Do hai đấu thủ luân phiên nhau đi nước của mình nên các mức trên cây trò chơi cũng luân phiên nhau là MAX và MIN. Cây trò chơi vì thế còn có tên là cây MIN-MAX. Ta có thể đưa ra một quy tắc định trị cho các nút trên cây để phản ánh tình trạng thắng thua hay hòa và khả năng thắng cuộc của hai đấu thủ.

Nếu một nút là nút lá thì trị của nó là giá trị đã được gán cho nút đó. Ngược lại, nếu nút là nút MAX thì trị của nó bằng giá trị lớn nhất của tất cả các trị của các con của nó. Nếu nút là nút MIN thì trị của nó là giá trị nhỏ nhất của tất cả các trị của các con của nó.

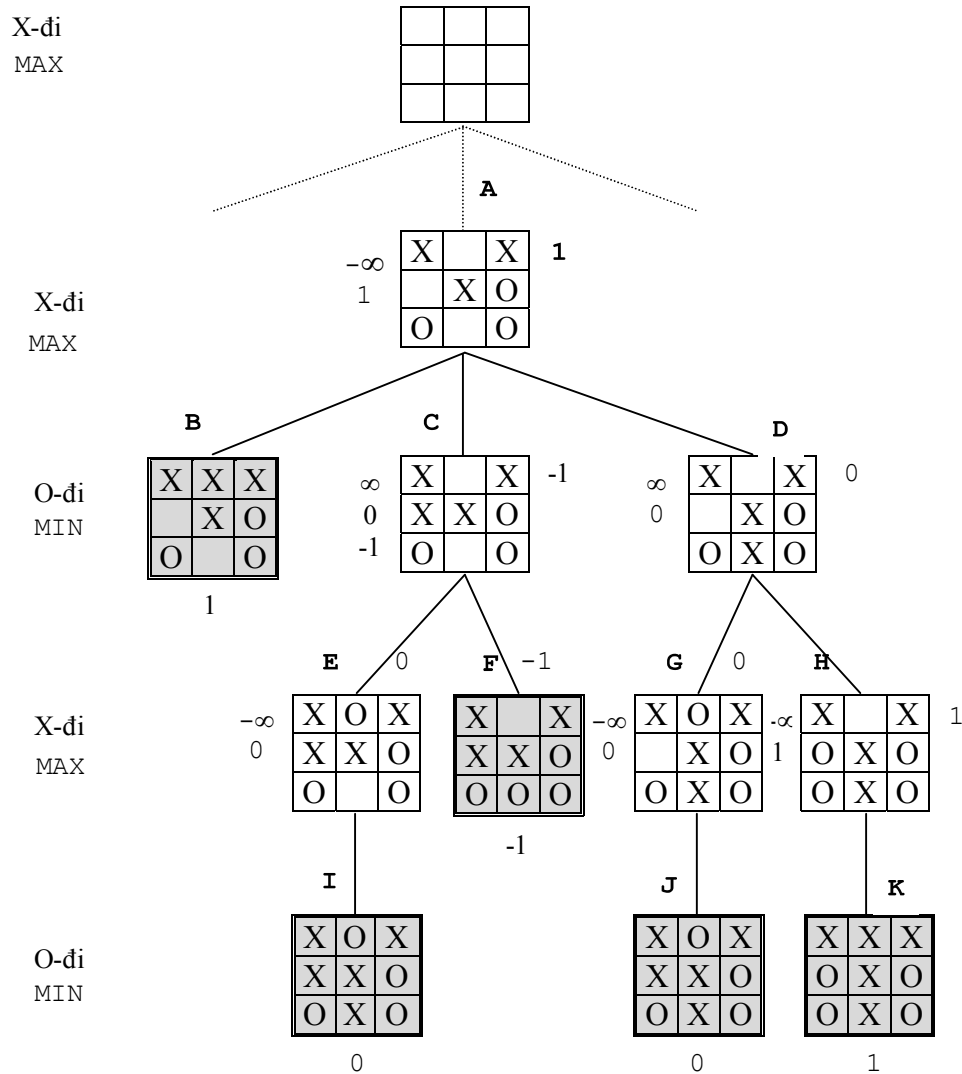
Quy tắc định trị này cũng gần giống với quy tắc định trị cho cây biểu thức số học, điểm khác biệt ở đây là các toán tử là các hàm lấy max hoặc min và mỗi nút có thể có nhiều con. Do vậy ta có thể dùng kĩ thuật quay lui để định trị cho các nút của cây trò chơi.

Ví dụ 3-6: Vận dụng quy tắc quay lui vét cạn để định trị cho nút A trong cây trò chơi trong ví dụ 3-5.

Trước hết ta gán trị cho các nút lá, theo qui định trên thì nút lá B được gán giá trị 1, vì tại đó người đánh X đã thắng. Nút F được gán giá trị -1 vì tại đó người đánh X đã thua (người đánh O đã thắng). Nút I được gán giá trị 0 vì tại đó hai người hòa nhau. Tương tự nút J được gán giá trị 0 và nút K được gán giá trị 1.

Vì người đánh X được gán giá trị 1 tại nút lá mà anh ta đã thắng (giá trị lớn nhất) nên ta nói X chọn nước đi MAX, ngược lại người đánh O sẽ chọn nước đi MIN.

Để định trị cho nút A, ta thấy A là nút MAX và không phải là nút lá nên ta gán giá trị tạm là $-\infty$, xét B là con của A, B là nút lá nên giá trị của nó là giá trị đã được gán 1, giá trị tạm của A bây giờ là $\max(-\infty, 1) = 1$. Xét con C của A, C là nút MIN, giá trị tạm lúc đầu của C là ∞ . Xét con E của C, E là nút MAX, giá trị tạm của E là $-\infty$. Xét con I của E, I là nút lá nên giá trị của nó là 0. Quay lui lại E, giá trị tạm của E bây giờ là $\max(-\infty, 0) = 0$. Vì E chỉ có một con là I đã xét nên giá trị tạm 0 trở thành giá trị của E. Quay lui lại C, giá trị tạm mới của C là $\min(\infty, 0) = 0$. Lại xét con F của C, vì F là nút lá, nên giá trị của F đã được gán là -1. Quay lui lại C, giá trị tạm mới của C là $\min(0, -1) = -1$. Nút C có hai con là E và F, cả hai con này đều đã được xét, vậy giá trị tạm -1 của C trở thành giá trị của nó. Sau khi có giá trị của C, ta phải quay lại A và đặt lại giá trị tạm của A là $\max(1, -1) = 1$. Tiếp tục xét nút D, D là nút MIN nên giá trị tạm là ∞ , xét nút con G của D, G là nút MAX nên giá trị tạm của nó là $-\infty$, xét nút con J của G. Vì J là nút lá nên có giá trị 0. Quay lui lại G, giá trị tạm của G bây giờ là $\max(-\infty, 0) = 0$ và giá trị tạm này trở thành giá trị của G vì G chỉ có một con J đã xét. Quay lui về D, giá trị tạm của D bây giờ là $\min(\infty, 0) = 0$. Lại xét con H của D, H là nút MAX nên gán giá trị tạm ban đầu là $-\infty$. Xét con K của H, nút K là nút lá nên giá trị của K đã được gán là 1. Quay lui về H và đặt lại giá trị tạm của H là $\max(-\infty, 1) = 1$. Giá trị tạm này chính là giá trị của H vì H chỉ có một con K đã được xét. Quay lui về D và đặt lại giá trị tạm của D là $\min(0, 1) = 0$. Cả hai con G và H của D đều đã được xét nên giá trị tạm 0 của D trở thành giá trị của nó. Quay lui về A, giá trị tạm của nó là $\max(1, 0) = 1$ vẫn không thay đổi, nhưng lúc này cả 3 con của A đều đã được xét nên giá trị tạm 1 trở thành giá trị của A. Kết quả được minh họa trong hình sau:



Hình 3-9: Định trị cây trò chơi bằng kỹ thuật quay lui vét cạn

Trong hình trên, các nút lá có giá trị được gán ghi phía dưới mỗi nút. Đối với các nút trong, bên trái ghi các giá trị tạm theo thứ tự trên xuống, các giá trị thực được ghi bên phải hoặc phía trên bên phải.

3.5.2.2 Giải thuật vét cạn định trị cây trò chơi

Để cài đặt ta có một số giả thiết sau:

- Ta có một hàm Payoff nhận vào một nút lá và cho ta giá trị của nút lá đó.
- Các hằng ∞ và $-\infty$ tương ứng là các trị Payoff lớn nhất và nhỏ nhất.
- Khai báo kiểu ModeType = (MIN, MAX) để xác định định trị cho nút là MIN hay MAX.

- Một kiểu NodeType được khai báo một cách thích hợp để biểu diễn cho một nút trên cây phản ánh một trạng thái của cuộc chơi.
- Ta có một hàm is_leaf để xác định xem một nút có phải là nút lá hay không?
- Hàm max và min tương ứng lấy giá trị lớn nhất và giá trị nhỏ nhất của hai giá trị.

Hàm Search nhận vào một nút n và kiểu mode của nút đó (MIN hay MAX) trả về giá trị của nút.

Nếu nút n là nút lá thì trả về giá trị đã được gán cho nút lá. Ngược lại ta cho n một giá trị tạm value là $-\infty$ hoặc ∞ tùy thuộc n là nút MAX hay MIN và xét con của n. Sau khi một con của n có giá trị V thì đặt lại value = max(value,V) nếu n là nút MAX và value = min(value,V) nếu n là nút MIN. Khi tất cả các con của n đã được xét thì giá trị tạm value của n trở thành giá trị của nó.

```

FUNCTION Search(n : NodeType; mode: ModeType): real;
VAR  C : NodeType ; { C là một nút con của nút n }
    Value : real;
{Lúc đầu ta cho value một giá trị tạm, sau khi đã xét hết tất
cả các con của nút n thì value là giá trị của nút n }
BEGIN
    IF is_leaf(n) THEN RETURN ( Payoff(n) )
    ELSE BEGIN
        {Khởi tạo giá trị tạm cho n }
        IF mode = MAX THEN value :=  $-\infty$  ELSE value :=  $\infty$ ;

        {Xét tất cả các con của n, mỗi lần xác định được giá trị của
một nút con, ta phải đặt lại giá trị tạm value. Khi đã xét
hết tất cả các con thì value là giá trị của n}

        FOR với mỗi con C của n DO
            IF mode = MAX THEN
                Value := max(Value, Search(C, MIN) )
            ELSE Value := min(Value, Search(C, MAX) );
        RETURN (value);
    END;
END;

```

3.5.2.3 Kĩ thuật cắt tỉa Alpha-Beta (Alpha-Beta Pruning)

Trong giải thuật vét cạn ở trên, ta thấy để định trị cho một nút nào đó, ta phải định trị cho tất cả các nút con cháu của nó, và muốn định trị cho nút gốc ta phải định trị cho tất cả các nút trên cây. Số lượng các nút trên cây trò chơi tuy hữu hạn nhưng không phải là ít. Chẳng hạn trong cây trò chơi ca rô nói trên, nếu ta có bàn cờ bao gồm n ô thì có thể có tới $n!$ nút trên cây (trong trường hợp trên là 9!). Đối với các loại cờ khác như cờ vua chẳng hạn, thì số lượng các nút còn lớn hơn nhiều. Ta gọi là một sự bùng nổ tổ hợp các nút.

Chúng ta cố gắng tìm một cách sao cho khi định trị một nút thì không nhất thiết phải định trị cho tất cả các nút con cháu của nó. Trước hết ta có nhận xét như sau:

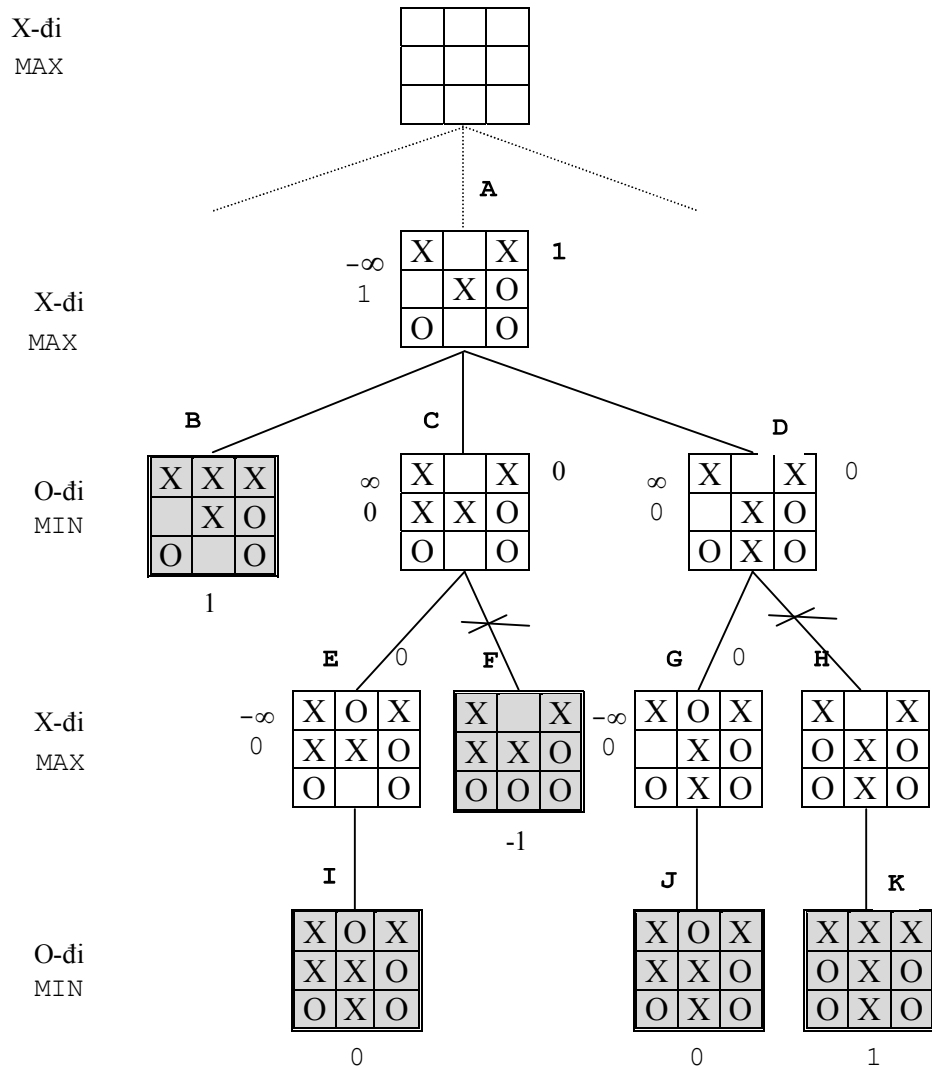
Nếu P là một nút MAX và ta đang xét một nút con Q của nó (dĩ nhiên Q là nút MIN). Giả sử V_p là một giá trị tạm của P, V_q là một giá trị tạm của Q và nếu ta có $V_p \geq V_q$ thì ta không cần xét các con chưa xét của Q nữa. Vì nếu có xét thì giá trị của Q cũng sẽ nhỏ hơn hoặc bằng V_q và do đó không ảnh hưởng gì đến V_p . Tương tự nếu P là nút MIN (tất nhiên Q là nút MAX) và $V_p \leq V_q$ thì ta cũng không cần xét đến các con chưa xét của Q nữa. Việc không xét tiếp các con chưa được xét của nút Q gọi là việc cắt tỉa Alpha-Beta các con của nút Q.

Trên cơ sở nhận xét đó, ta nêu ra quy tắc định trị cho một nút không phải là nút lá trên cây như sau:

1. Khởi đầu nút MAX có giá trị tạm là $-\infty$ và nút MIN có giá trị tạm là ∞ .
2. Nếu tất cả các nút con của một nút đã được xét hoặc bị cắt tỉa thì giá trị tạm của nút đó trở thành giá trị của nó.
3. Nếu một nút MAX n có giá trị tạm là V_1 và một nút con của nó có giá trị là V_2 thì đặt giá trị tạm mới của n là $\max(V_1, V_2)$. Nếu n là nút MIN thì đặt giá trị tạm mới của n là $\min(V_1, V_2)$.
4. Vận dụng quy tắc cắt tỉa Alpha-Beta nói trên để hạn chế số lượng nút phải xét.

Ví dụ 3-7: Vận dụng quy tắc trên để định trị cho nút A của cây trò chơi trong ví dụ 3-5.

A là nút MAX, vì A không phải là nút lá nên ta gán giá trị tạm là $-\infty$, xét B là con của A, B là nút lá nên giá trị của nó là giá trị đã được gán 1, giá trị tạm của A bây giờ là $\max(-\infty, 1) = 1$. Xét con C của A, C là nút MIN, giá trị tạm lúc đầu của C là ∞ . Xét con E của C, E là nút MAX, giá trị tạm của E là $-\infty$. Xét con I của E, I là nút lá nên giá trị của nó là 0. Quay lui lại E, giá trị tạm của E bây giờ là $\max(-\infty, 0) = 0$. Vì E chỉ có một con là I đã xét nên giá trị tạm 0 trở thành giá trị của E. Quay lui lại C, giá trị tạm mới của C là $\min(\infty, 0) = 0$. A là nút MAX có giá trị tạm là 1, C là con của A, có giá trị tạm là 0, $1 > 0$ nên ta không cần xét con F của C nữa. Nút C có hai con là E và F, trong đó E đã được xét, F đã bị cắt, vậy giá trị tạm 0 của C trở thành giá trị của nó. Sau khi có giá trị của C, ta phải đặt lại giá trị tạm của A, nhưng giá trị tạm này không thay đổi vì $\max(1, 0) = 1$. Tiếp tục xét nút D, D là nút MIN nên giá trị tạm là ∞ , xét nút con G của D, G là nút MAX nên giá trị tạm của nó là $-\infty$, xét nút con J của G. Vì J là nút lá nên có giá trị 0. Quay lui lại G, giá trị tạm của G bây giờ là $\max(-\infty, 0) = 0$ và giá trị tạm này trở thành giá trị của G vì G chỉ có một con J đã xét. Quay lui về D, giá trị tạm của D bây giờ là $\min(\infty, 0) = 0$. Giá trị tạm này của D nhỏ hơn giá trị tạm của nút A MAX là cha của nó nên ta cắt tỉa con H chưa được xét của D và lúc này D có giá trị là 0. Quay lui về A, giá trị tạm của nó vẫn không thay đổi, nhưng lúc này cả 3 con của A đều đã được xét nên giá trị tạm 1 trở thành giá trị của A. Kết quả được minh họa trong hình sau:



Hình

3-10: Định trị cây trò chơi bằng kĩ thuật cắt tỉa alpha-beta

Hàm `cat_tia` sau trình bày giải thuật thô để định trị một nút, áp dụng kĩ thuật cắt tỉa alpha-beta

```

FUNCTION cat_tia(Q:NodeType; mode:ModeType; Vp: real): real;
var C : NodeType ; { C là một nút con của nút Q }
    Vq : real;
{Vq là giá trị tạm của Q, sau khi tất cả các con của nút Q đã
xét hoặc bị cắt tỉa thì Vq là giá trị của nút Q}
BEGIN
    IF is_leaf(Q) THEN RETURN ( Payoff(Q) )
    ELSE BEGIN
        { Khởi tạo giá trị tạm cho Q }
        IF mode = MAX THEN Vq :=  $-\infty$  ELSE Vq :=  $\infty$ ;

```

{Xét các con của Q , mỗi lần xác định được giá trị của một nút con của Q , ta phải đặt lại giá trị tạm V_q và so sánh với V_p để có thể cắt tỉa hay không}

```

    Xét C là con trái nhất của Q;
    WHILE C là con của Q DO
        IF mode = MAX THEN BEGIN
             $V_q := \max(V_q, \text{Cat\_tia}(C, \text{MIN}, V_q));$ 
            IF  $V_p \leq V_q$  THEN RETURN( $V_q$ );
        END
        ELSE BEGIN
             $V_q := \min(V_q, \text{Cat\_tia}(C, \text{MAX}, V_q));$ 
            IF  $V_p \geq V_q$  THEN RETURN( $V_q$ );
        END;
    RETURN ( $V_q$ );
END;
END;
```

3.5.3 Kĩ thuật nhánh cận

Với các bài toán tìm phương án tối ưu, nếu chúng ta xét hết tất cả các phương án thì mất rất nhiều thời gian, nhưng nếu sử dụng phương pháp tham ăn thì phương án tìm được chưa hẳn đã là phương án tối ưu. Nhánh cận là kĩ thuật xây dựng cây tìm kiếm phương án tối ưu, nhưng không xây dựng toàn bộ cây mà sử dụng giá trị cận để hạn chế bớt các nhánh.

Cây tìm kiếm phương án có nút gốc biểu diễn cho tập tất cả các phương án có thể có, mỗi nút lá biểu diễn cho một phương án nào đó. Nút n có các nút con tương ứng với các khả năng có thể lựa chọn tập phương án xuất phát từ n . Kĩ thuật này gọi là phân nhánh.

Với mỗi nút trên cây ta sẽ xác định một giá trị cận. Giá trị cận là một giá trị gần với giá của các phương án. Với bài toán tìm min ta sẽ xác định cận dưới còn với bài toán tìm max ta sẽ xác định cận trên. Cận dưới là giá trị nhỏ hơn hoặc bằng giá của phương án, ngược lại cận trên là giá trị lớn hơn hoặc bằng giá của phương án.

Để dễ hình dung ta sẽ xét hai bài toán quen thuộc là bài toán TSP và bài toán cái ba lô.

3.5.3.1 Bài toán đường đi của người giao hàng

3.5.3.1.1 Phân nhánh

Cây tìm kiếm phương án là cây nhị phân trong đó:

- Nút gốc là nút biểu diễn cho cấu hình bao gồm tất cả các phương án.
- Mỗi nút sẽ có hai con, con trái biểu diễn cho cấu hình bao gồm tất cả các phương án chứa một cạnh nào đó, con phải biểu diễn cho cấu hình bao gồm tất cả các phương án không chứa cạnh đó (các cạnh để xét phân nhánh được thành lập tuân theo một thứ tự nào đó, chẳng hạn thứ tự từ điển).
- Mỗi nút sẽ kế thừa các thuộc tính của tổ tiên của nó và có thêm một thuộc tính mới (chứa hay không chứa một cạnh nào đó).

- Nút lá biểu diễn cho một cấu hình chỉ bao gồm một phương án.
- Để quá trình phân nhánh mau chóng tới nút lá, tại mỗi nút ta cần có một quyết định bổ sung dựa trên nguyên tắc là mọi đỉnh trong chu trình đều có cấp 2 và không tạo ra một chu trình thiếu.

Ví dụ 3-7: Xét bài toán TSP có 5 đỉnh với độ dài các cạnh được cho trong hình 3-11.

Các cạnh theo thứ tự từ điển để xét là:

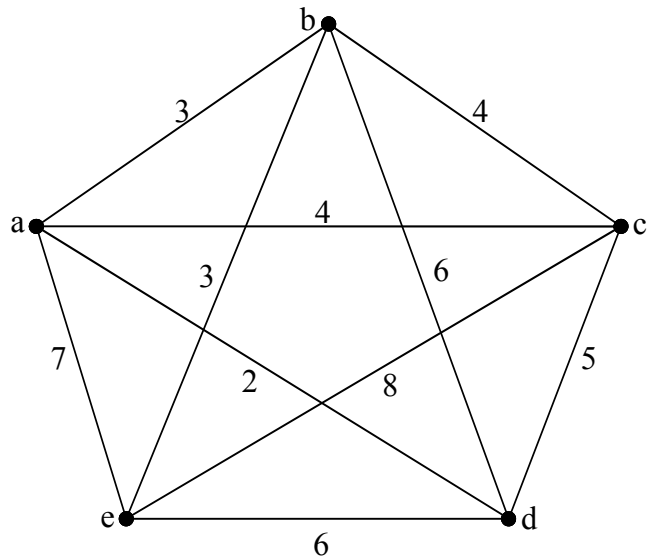
ab, ac, ad, ae, bc, bd, be, cd, ce và de.

Nút gốc A của cây bao gồm tất cả các phương án.

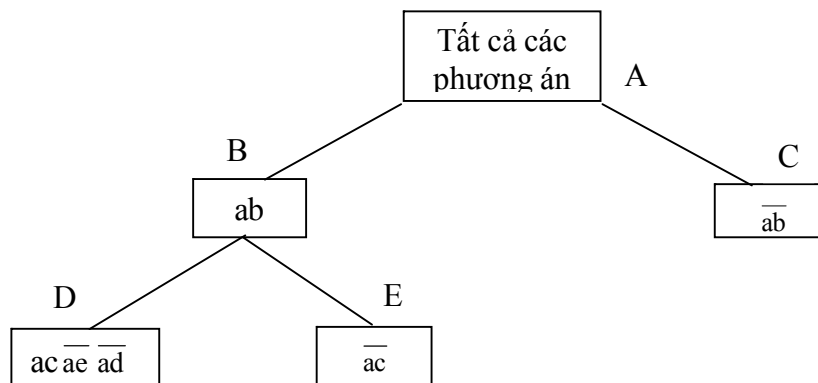
Hai con của A là B và C, trong đó B bao gồm tất cả các phương án chứa cạnh ab, C bao gồm tất cả các phương án không chứa ab, kí hiệu là $\overline{ab}$

Hai con của B là D và E. Nút D bao gồm tất cả các phương án chứa ac. Vì các phương án này vừa chứa ab (kế thừa của B) vừa chứa ac nên đỉnh a đã đủ cấp hai nên D không thể chứa ad và ae. Nút E bao gồm tất cả các phương án không chứa ac...

Ta được cây (chưa đầy đủ) trong hình 3-12.



Hình 3-11: Bài toán TSP có 5 đỉnh



Hình 3-12: Phân nhánh

3.5.3.1.2 Tính cận dưới

Đây là bài toán tìm min nên ta sử dụng cận dưới. Cận dưới tại mỗi nút là một số nhỏ hơn hoặc bằng giá của tất cả các phương án được biểu diễn bởi nút đó. Giá của một phương án ở đây là tổng độ dài của một chu trình.

Để tính cận dưới của nút gốc, mỗi đỉnh ta chọn hai cạnh có độ dài nhỏ nhất. Cận dưới của nút gốc bằng tổng độ dài tất cả các cạnh được chọn chia cho 2.

Ví dụ 3-8: Với số liệu cho trong ví dụ 3-7 nói trên, ta tính cận dưới của nút gốc A (hình 3-12) như sau:

- Đỉnh a chọn $ad = 2$, $ab = 3$
- Đỉnh b chọn $ba = 3$, $be = 3$
- Đỉnh c chọn $ca = 4$, $cb = 4$
- Đỉnh d chọn $da = 2$, $dc = 5$
- Đỉnh e chọn $eb = 3$, $ed = 6$

Tổng độ dài các cạnh được chọn là 35, cận dưới của nút gốc A là $35/2 = 17.5$

Đối với các nút khác, chúng ta phải lựa chọn hai cạnh có độ dài nhỏ nhất thỏa điều kiện ràng buộc (phải chứa cạnh này, không chứa cạnh kia).

Ví dụ 3-9: Tính cận dưới cho nút D trong hình 3-13. Điều kiện ràng buộc là phải chứa ab , ac và không chứa ad , ae .

- Đỉnh a chọn $ab = 3$, $ac = 4$, do hai cạnh này buộc phải chọn.
- Đỉnh b chọn $ba = 3$, $be = 3$
- Đỉnh c chọn $ca = 4$, $cb = 4$
- Đỉnh d chọn $de = 6$, $dc = 5$, do không được chọn da nên ta phải chọn de .
- Đỉnh e chọn $eb = 3$, $ed = 6$

Tổng độ dài các cạnh được chọn là 41, cận dưới của nút D là $41/2 = 20.5$

3.5.3.1.3 Kĩ thuật nhánh cận

Bây giờ ta sẽ kết hợp hai kĩ thuật trên để xây dựng cây tìm kiếm phương án. Quy tắc như sau:

- Xây dựng nút gốc, bao gồm tất cả các phương án, tính cận dưới cho nút gốc.
- Sau khi phân nhánh cho mỗi nút, ta tính cận dưới cho cả hai con.
- Nếu cận dưới của một nút con lớn hơn hoặc bằng giá nhỏ nhất tạm thời của một phương án đã được tìm thấy thì ta không cần xây dựng các cây con cho nút này nữa (Ta gọi là cắt tỉa các cây con của nút đó).
- Nếu cả hai con đều có cận dưới nhỏ hơn giá nhỏ nhất tạm thời của một phương án đã được tìm thấy thì nút con nào có cận dưới nhỏ hơn sẽ được ưu tiên phân nhánh trước.
- Mỗi lần quay lui để xét nút con chưa được xét của một nút ta phải xem xét lại nút con đó để có thể cắt tỉa các cây của nó hay không vì có thể một phương án có giá nhỏ nhất tạm thời vừa được tìm thấy.

- Sau khi tất cả các con đã được phân nhánh hoặc bị cắt tia thì phương án có giá nhỏ nhất trong các phương án tìm được là phương án cần tìm.

Trong quá trình xây dựng cây có thể ta đã xây dựng được một số nút lá, như ta biết mỗi nút lá biểu diễn cho một phương án. Giá nhỏ nhất trong số các giá của các phương án này được gọi là giá nhỏ nhất tạm thời.

Ví dụ 3-10: Xét bài toán TSP trong ví dụ 3-7 nói trên.

Tập hợp các cạnh để xét phân nhánh là $ab, ac, ad, ae, bc, bd, be, cd, ce$ và de . Điều kiện bổ sung ở đây là mỗi đỉnh phải được chọn hai cạnh, bị loại hai cạnh và không được tạo ra chu trình thiếu.

Nút gốc A bao gồm tất cả các phương án, có cận dưới là 17.5. Phân nhánh cho A, xây dựng hai con là B và C. Tính cận dưới cho hai nút này được cận dưới của B là 17.5 và C là 18.5. Nút B có cận dưới nhỏ hơn nên được phân nhánh trước. Hai con của B là D và E. Các ràng buộc của D và E giống như ta đã nói trong ví dụ của phần phân nhánh. Tính cận cho D và E, được cận dưới của D là 20.5 và của E là 18. Nút E được xét trước. Phân nhánh cho nút E theo cạnh ad , hai con của E là F và G. F chứa ad và G không chứa ad . Do F kế thừa các thuộc tính của E và B, nên F là tập hợp các phương án chứa ab, ad và không chứa ac , đỉnh a đã đủ cấp 2 vậy F không chứa ae . Tương tự G chứa ab , không chứa ac , không chứa ad nên phải chứa ae . Tính cận dưới cho F và G được cận dưới của F là 18 và của G là 23. Tiếp tục xây dựng hai con cho F theo cạnh bc là H và I. H chứa bc và I không chứa bc . Do H kế thừa các thuộc tính của B, E và F nên H là các phương án chứa ab, ad , không chứa ac và chứa bc . Như vậy đỉnh a đã thỏa điều kiện là được chọn hai cạnh (ab và ad) và bị loại hai cạnh (ac và ae), Đỉnh b đã được chọn 2 cạnh (ba và bc) nên bd và be bị loại. Đỉnh c đã được chọn cb , bị loại ca , ta có thể chọn cd hoặc ce . Nếu chọn cd thì sẽ có một chu trình thiếu $a b c d a$, như vậy cd bị loại nên phải chọn ce . Đỉnh d có db và dc đã bị loại, da đã được chọn nên phải chọn thêm de . Lúc đó đỉnh e cũng đã có hai cạnh được chọn là ec và ed , hai cạnh bị loại là eb và ea . Tóm lại H là tập chỉ bao gồm một phương án $a b c e d a$ có giá là 23. Đối với I ta đã có I chứa ab , không chứa ac , chứa ad , không chứa ae và không chứa bc . Bằng lý luận tương tự ta có I không chứa bd , chứa be, cd, ce và không chứa de . Một phương án mới là $a b e c d a$ với giá 21. Đây là giá nhỏ nhất tạm thời mới được tìm thấy.

Bây giờ ta quay lui về E và xét nút con của nó là G. Vì G có cận dưới là 23 lớn hơn giá thấp nhất tạm thời 21 nên cắt tia các con của G.

Quay lui về B và xét nút con D của nó. Cận dưới của D là 20.5 không lớn hơn 21. Nhưng vì độ dài các cạnh trong bài toán đã cho là số nguyên nên nếu ta triển khai các con của D tới nút lá gồm một phương án. Giá của phương án này phải là một số nguyên lớn hơn 20.5 hay lớn hơn hoặc bằng 21. Vậy ta cũng không cần xây dựng các con của D nữa.

Tiếp tục quay lui đến A và xét con C của nó. Phân nhánh C theo cạnh ac thành hai con J và K. J chứa ac có cận dưới là 18.5. K không chứa ac nên phải chứa ad và ae , cận dưới của K là 21 bằng giá nhỏ nhất tạm thời nên cắt tia các con của K.

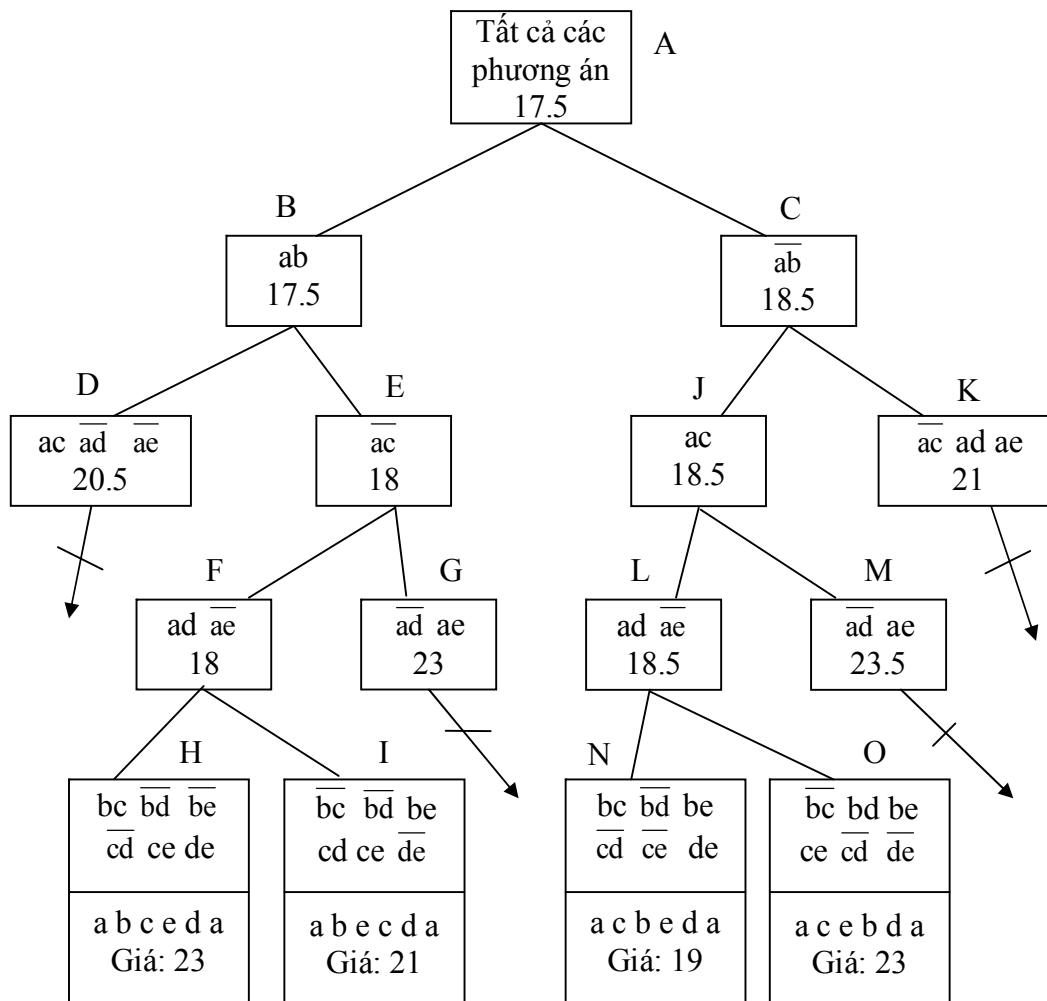
Hai con của J là L và M. M không chứa ad, ab , chứa ac và ae có cận dưới 23.5 nên bị cắt tia các con. Hai con của L là N và O, N chứa bc và O không chứa bc .

Xét nút N ta có: Đỉnh a được chọn hai cạnh ac và ad, bị loại hai cạnh ab và ae. Đỉnh b đã được chọn bc, bị loại ba, ta có thể chọn bd hoặc be. Nếu chọn bd thì sẽ có một chu trình thiếu là a c b d a, vậy phải loại bd và chọn be. Đỉnh c đã được chọn ca, cb nên phải loại cd và ce. Đỉnh d đã được chọn da, bị loại db và dc nên phải chọn de. Khi đó đỉnh e có đủ hai cạnh được chọn là eb, ed và hai cạnh bị loại là ea và ec. Vậy N bao gồm chỉ một phương án là a c b e d a với giá 19.

Tương tự nút O bao gồm chỉ một phương án a c e b d a có giá là 23.

Tất cả các nút con của cây đã được xét hoặc bị cắt tía nên phương án cần tìm là a c b e d a với giá 19.

Hình 3-13 minh họa cho những điều ta vừa nói.



Hình 3-13: Kỹ thuật nhánh cận giải bài toán TSP

3.5.3.2 Bài toán cái ba lô

Ta thấy đây là một bài toán tìm max. Danh sách các đồ vật được sắp xếp theo thứ tự giảm của đơn giá để xét phân nhánh.

1. Nút gốc biểu diễn cho trạng thái ban đầu của ba lô, ở đó ta chưa chọn một vật nào. Tổng giá trị được chọn $TGT = 0$. Cận trên của nút gốc $CT = W * \text{Đơn giá lớn nhất}$.

2. Nút gốc sẽ có các nút con tương ứng với các khả năng chọn đồ vật có đơn giá lớn nhất. Với mỗi nút con ta tính lại các thông số:

- $TGT = TGT \text{ (của nút cha)} + \text{số đồ vật được chọn} * \text{giá trị mỗi vật}$.
- $W = W \text{ (của nút cha)} - \text{số đồ vật được chọn} * \text{trọng lượng mỗi vật}$.
- $CT = TGT + W * \text{Đơn giá của vật sẽ xét kế tiếp}$.

3. Trong các nút con, ta sẽ ưu tiên phân nhánh cho nút con nào có cận trên lớn hơn trước. Các con của nút này tương ứng với các khả năng chọn đồ vật có đơn giá lớn tiếp theo. Với mỗi nút ta lại phải xác định lại các thông số TGT , W , CT theo công thức đã nói trong bước 2.

4. Lặp lại bước 3 với chú ý: đối với những nút có cận trên nhỏ hơn hoặc bằng giá lớn nhất tạm thời của một phương án đã được tìm thấy thì ta không cần phân nhánh cho nút đó nữa (cắt bỏ).

5. Nếu tất cả các nút đều đã được phân nhánh hoặc bị cắt bỏ thì phương án có giá lớn nhất là phương án cần tìm.

Ví dụ 3-11: Với bài toán cái ba lô đã cho trong ví dụ 3-2, sau khi tính đơn giá cho các đồ vật và sắp xếp các đồ vật theo thứ tự giảm dần của đơn giá ta được bảng sau.

Loại đồ vật	Trọng lượng	Giá trị	Đơn giá
b	10	25	2.5
a	15	30	2.0
d	4	6	1.5
c	2	2	1

Gọi X_A, X_B, X_C, X_D là số lượng cần chọn tương ứng của các đồ vật a, b, c, d.

Nút gốc A biểu diễn cho trạng thái ta chưa chọn bất cứ một đồ vật nào. Khi đó tổng giá trị $TGT=0$, trọng lượng của ba lô $W=37$ (theo đề ra) và cận trên $CT = 37*2.5 = 92.5$, trong đó 37 là W , 2.5 là đơn giá của đồ vật b.

Với đồ vật b, ta có 4 khả năng: chọn 3 đồ vật b ($X_B=3$), chọn 2 đồ vật b ($X_B=2$), chọn 1 đồ vật b ($X_B=1$) và không chọn đồ vật b ($X_B=0$). Ứng với 4 khả năng này, ta phân nhánh cho nút gốc A thành 4 con B, C, D và E.

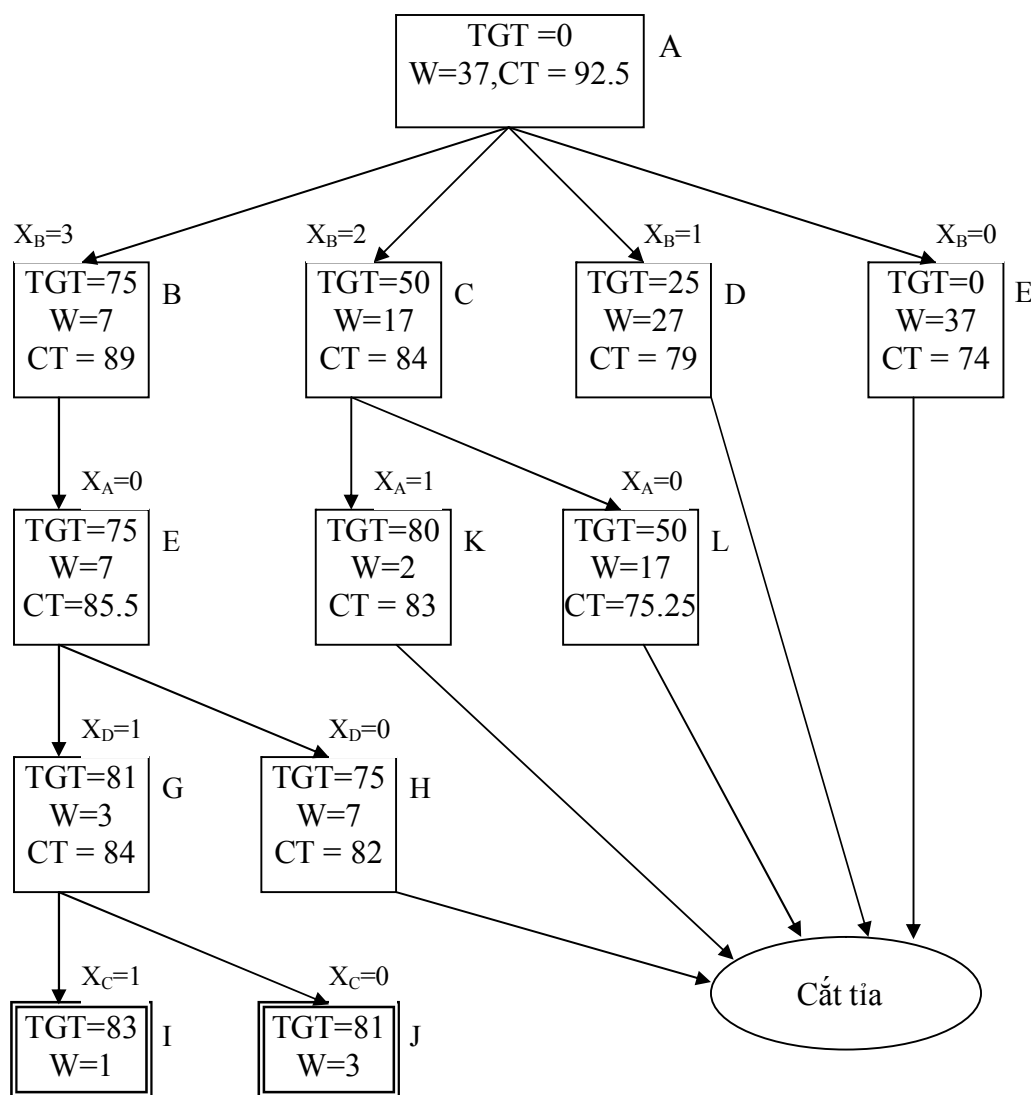
Với nút con B, ta có $TGT = 0 + 3*25 = 75$, trong đó 3 là số vật b được chọn, 25 là giá trị của mỗi đồ vật b. $W = 37 - 3*10 = 7$, trong đó 37 là trọng lượng ban đầu của ba lô, 3 là số vật b được, 10 là trọng lượng mỗi đồ vật b. $CT = 75 + 7*2 = 89$, trong đó 75 là TGT , 7 là trọng lượng còn lại của ba lô và 2 là đơn giá của đồ vật a. Tương

tự ta tính được các thông số cho các nút C, D và E, trong đó cận trên tương ứng là 84, 79 và 74.

Trong các nút B, C, D và E thì nút B có cận trên lớn nhất nên ta sẽ phân nhánh cho nút B trước với hy vọng sẽ có được phương án tốt từ hướng này. Từ nút B ta chỉ có một nút con F duy nhất ứng với $X_A=0$ (do trọng lượng còn lại của ba lô là 7, trong khi trọng lượng của mỗi đồ vật a là 15). Sau khi xác định các thông số cho nút F ta có cận trên của F là 85.5. Ta tiếp tục phân nhánh cho nút F. Nút F có 2 con G và H tương ứng với $X_D=1$ và $X_D=0$. Sau khi xác định các thông số cho hai nút này ta thấy cận trên của G là 84 và của H là 82 nên ta tiếp tục phân nhánh cho nút G. Nút G có hai con là I và J tương ứng với $X_C=1$ và $X_C=0$. Đây là hai nút lá (biểu diễn cho phương án) vì với mỗi nút thì số các đồ vật đã được chọn xong. Trong đó nút I biểu diễn cho phương án chọn $X_B=3$, $X_A=0$, $X_D=1$ và $X_C=1$ với giá 83, trong khi nút J biểu diễn cho phương án chọn $X_B=3$, $X_A=0$, $X_D=1$ và $X_C=0$ với giá 81. Như vậy giá lớn nhất tạm thời ở đây là 83.

Quay lui lên nút H, ta thấy cận trên của H là $82 < 83$ nên cắt tia nút H.

Quay lui lên nút C, ta thấy cận trên của C là $84 > 83$ nên tiếp tục phân nhánh cho nút C. Nút C có hai con là K và L ứng với $X_A=1$ và $X_A=0$. Sau khi tính các thông số cho K và L ta thấy cận trên của K là 83 và của L là 75.25. Cả hai giá trị này đều không lớn hơn 83 nên cả hai nút này đều bị cắt tia. Cuối cùng các nút D và E cũng bị cắt tia. Như vậy tất cả các nút trên cây đều đã được phân nhánh hoặc bị cắt tia nên phương án tốt nhất tạm thời là phương án cần tìm. Theo đó ta cần chọn 3 đồ vật loại b, 1 đồ vật loại d và một đồ vật loại c với tổng giá trị là 83, tổng trọng lượng là 36. Xem minh họa trong hình 3-14.



Hình 3-14: Kĩ thuật nhánh cận áp dụng cho bài toán cái ba lô

3.6 KỸ THUẬT TÌM KIẾM ĐỊA PHƯƠNG

3.6.1 Nội dung kĩ thuật

Kĩ thuật tìm kiếm địa phương (local search) thường được áp dụng để giải các bài toán tìm lời giải tối ưu. Phương pháp như sau:

- Xuất phát từ một phương án nào đó.
- Áp dụng một phép biến đổi lên phương án hiện hành để được một phương án mới tốt hơn phương án đã có.
- Lặp lại việc áp dụng phép biến đổi lên phương án hiện hành cho đến khi không còn có thể cải thiện được phương án nữa.

Thông thường một phép biến đổi chỉ thay đổi một bộ phận nào đó của phương án hiện hành để được một phương án mới nên phép biến đổi được gọi là phép biến đổi địa phương và do đó ta có tên kĩ thuật tìm kiếm địa phương. Sau đây ta sẽ trình bày một số ví dụ áp dụng kĩ thuật tìm kiếm địa phương.

3.6.2 Bài toán cây phủ tối thiểu

Cho $G = (V, E)$ là một đồ thị vô hướng liên thông, trong đó V là tập các đỉnh và E là tập các cạnh. Các cạnh của đồ thị G đều có trọng số. Cây T có tập hợp các nút là V được gọi là cây phủ (spanning tree) của đồ thị G .

Cây phủ tối thiểu là một cây phủ của G mà tổng độ dài (trọng số) các cạnh nhỏ nhất.

Bài toán cây phủ tối thiểu thường được áp dụng trong việc thiết kế một mạng lưới giao thông giữa các thành phố hay thiết kế một mạng máy tính.

Kĩ thuật tìm kiếm địa phương áp dụng vào bài toán này như sau:

- Phương án ban đầu là một cây phủ nào đó.
- Thành lập tập tất cả các cạnh theo thứ tăng dần của độ dài (có $\frac{n(n-1)}{2}$ cạnh đối với đồ thị có n đỉnh).
- Phép biến đổi địa phương ở đây là: Chọn một cạnh có độ dài nhỏ nhất trong tập các cạnh chưa sử dụng để thêm vào cây. Trong cây sẽ có một chu trình, loại khỏi chu trình cạnh có độ dài lớn nhất trong chu trình đó. Ta được một cây phủ mới. Lặp lại bước này cho đến khi không còn cải thiện được phương án nữa.

Ví dụ 3-12: Cho đồ thị G bao gồm 5 đỉnh a, b, c, d, e và độ dài các cạnh được cho trong hình 3-15.

Tập hợp các cạnh để xét được thành lập theo thứ tự từ nhỏ đến lớn là $ad, ab, be, bc, ac, cd, bd, de, ae$ và ce .

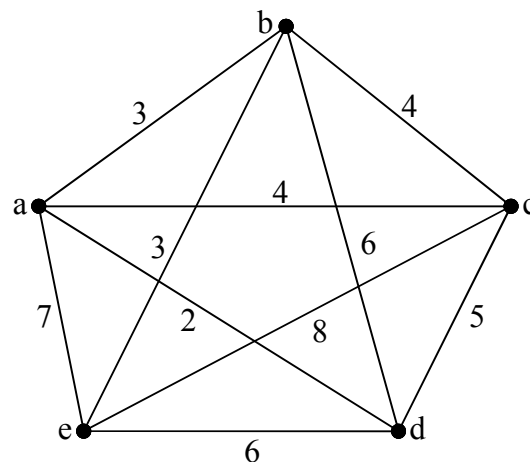
Cây xuất phát với giá là 20 (Hình 3-16). Thêm cạnh $ad = 2$, bỏ cạnh $cd = 5$ ta được cây mới có giá là 17 (Hình 3-17).

Lại thêm cạnh $ab = 3$, bỏ cạnh $bc = 4$ ta được cây có giá là 16 (Hình 3-18).

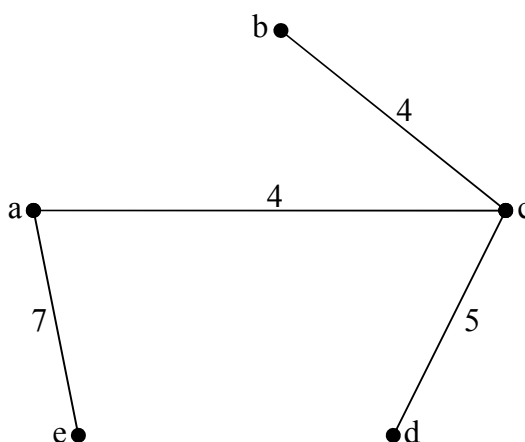
Thêm cạnh $be = 3$, bỏ cạnh $ae = 7$ ta được cây có giá là 12. (Hình 3-19).

Việc áp dụng các phép biến đổi đến đây dừng lại vì nếu tiếp tục nữa thì cũng không cải thiện được phương án.

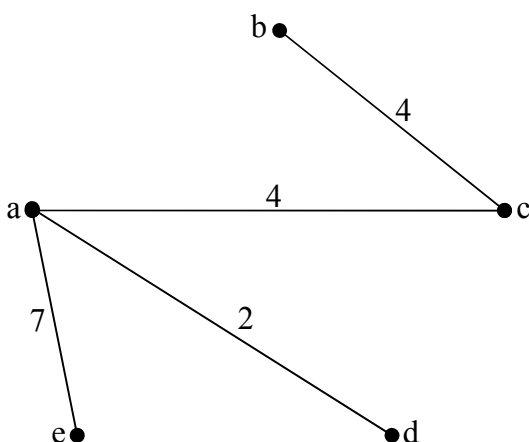
Vậy cây phủ tối thiểu cần tìm là cây trong hình 3-19



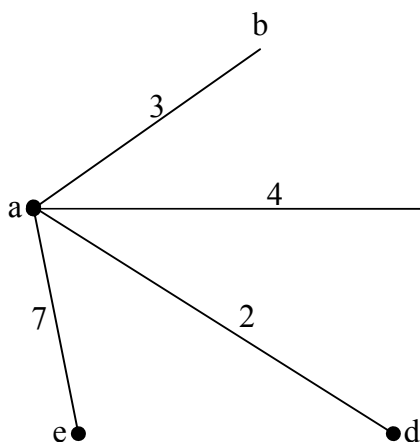
Hình 3-15: Bài toán cây phủ tối thiểu



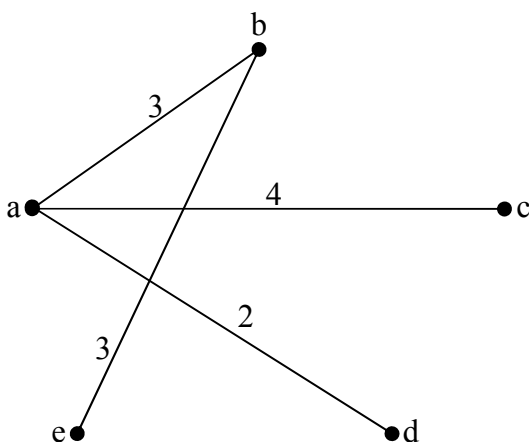
Hình 3-16: Cây xuất phát, giá 20



Hình 3-17: Giá 17



Hình 3-18: Giá 16



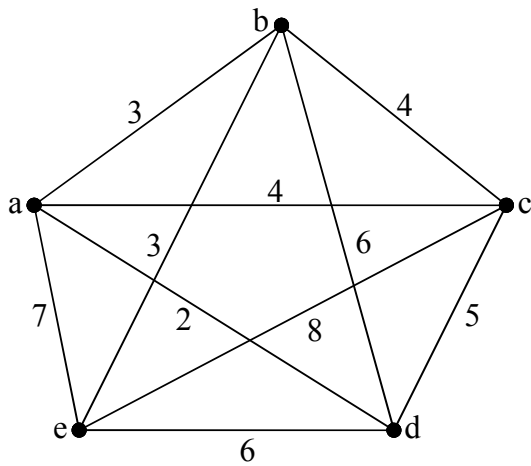
Hình 3-19: Giá 12

3.6.3 Bài toán đường đi của người giao hàng.

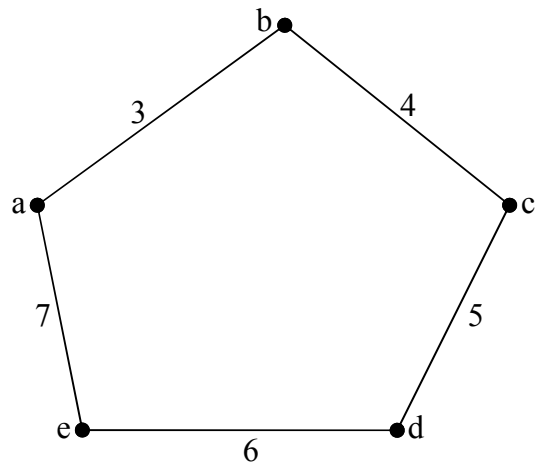
Ta có thể vận dụng kỹ thuật tìm kiếm địa phương để giải bài toán tìm đường đi ngắn nhất của người giao hàng (TSP).

- Xuất phát từ một chu trình nào đó.
- Bỏ đi hai cạnh có độ dài lớn nhất không kề nhau, nối các đỉnh lại với nhau sao cho vẫn tạo ra một chu trình đủ.
- Tiếp tục quá trình biến đổi trên cho đến khi nào không còn cải thiện được phương án nữa.

Ví dụ 3-13: Bài toán TSP có 5 đỉnh và các cạnh có độ dài được cho trong hình 3-20. Phương án ban đầu là chu trình (a b c d e a) có giá (tổng độ dài) là 25. (Hình 3-21).

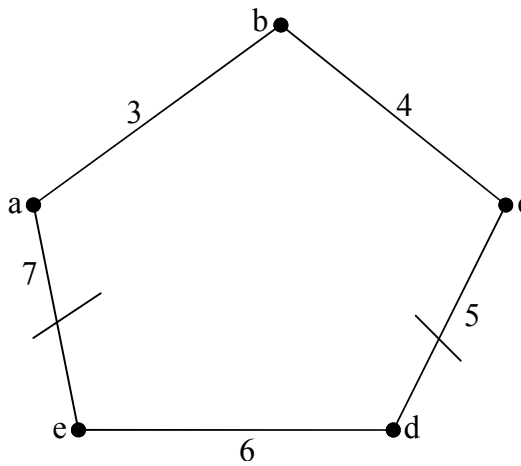


Hình 3-20: Bài toán TSP với 5 đỉnh

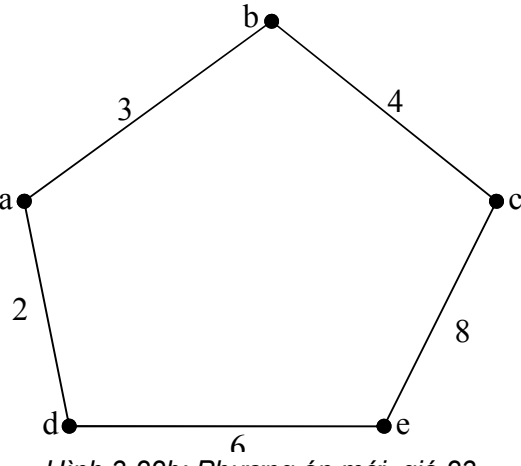


Hình 3-21: Phương án ban đầu, giá 25

Bỏ hai cạnh có độ dài lớn nhất không kề nhau là ae và cd (hình 3-22a), nối a với d và e với c, ta được chu trình mới (a b c e d a) với giá = 23 (Hình 3-22b).

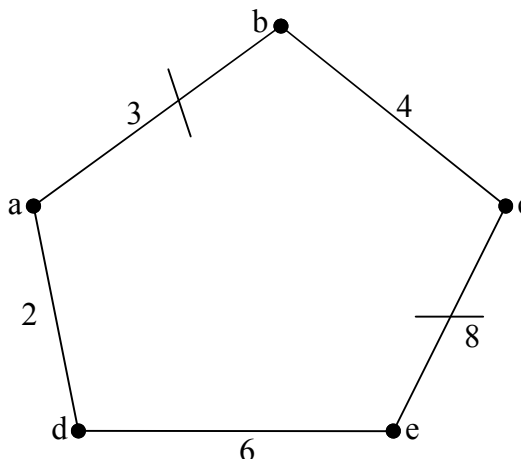


Hình 3-22a: Bỏ hai cạnh ae và cd

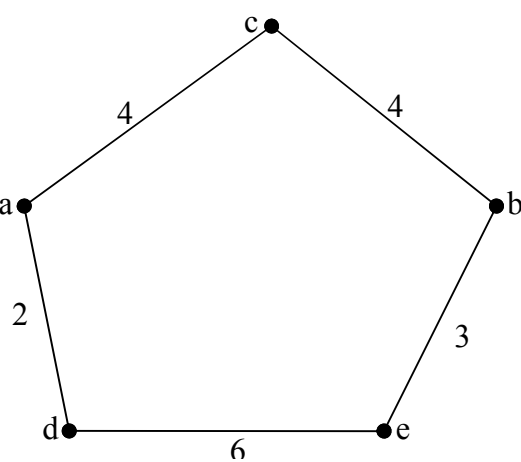


Hình 3-22b: Phương án mới, giá 23.

Bỏ hai cạnh có độ dài lớn nhất, không kề nhau là ce và ab (hình 3-23a), nối a với c và b với e, ta được chu trình mới (a c b e d a) có giá = 19. (Hình 3-23b). Quá trình kết thúc vì nếu tiếp tục thì giá sẽ tăng lên.



Hình 3-23a: Bỏ hai cạnh ce và ab.



Hình 3-23b: Phương án mới, giá 19

3.7 TỔNG KẾT CHƯƠNG 3

Trong các kĩ thuật được trình bày trong chương, kĩ thuật chia để trị là kĩ thuật cơ bản nhất. Hãy chia nhỏ các bài toán để giải quyết nó!

Với các bài toán tìm phương án tối ưu, kĩ thuật “tham ăn” giúp chúng ta nhanh chóng xây dựng được một phương án, dầu rằng chưa hẳn tối ưu nhưng chấp nhận được. Kĩ thuật nhánh cận cho phép chúng ta tìm được phương án tối ưu. Trong kĩ thuật nhánh cận, việc phân nhánh không khó nhưng việc xác định giá trị cận là điều quan trọng. Cần phải xác định giá trị cận sao cho càng sát với giá của phương án càng tốt vì như thế thì có thể cắt tia được nhiều nút trên cây và do đó sẽ giảm được thời gian thực hiện chương trình.

Vận dụng phương pháp quy hoạch động có thể giải được rất nhiều bài toán. Điều quan trọng nhất để áp dụng phương pháp quy hoạch động là phải xây dựng được công thức đệ quy để xác định kết quả bài toán thông qua kết quả các bài toán con.

BÀI TẬP CHƯƠNG 3

Bài 1: Giả sử có hai đội A và B tham gia một trận thi đấu thể thao, đội nào thắng trước n hiệp thì sẽ thắng cuộc. Chẳng hạn một trận thi đấu bóng chuyền 5 hiệp, đội nào thắng trước 3 hiệp thì sẽ thắng cuộc. Giả sử hai đội ngang tài ngang sức. Đội A cần thắng thêm i hiệp để thắng cuộc còn đội B thì cần thắng thêm j hiệp nữa. Gọi $P(i,j)$ là xác suất để đội A cần i hiệp nữa để chiến thắng, B cần j hiệp. Dĩ nhiên i, j đều là các số nguyên không âm.

Để tính $P(i,j)$ ta thấy rằng nếu $i=0$, tức là đội A đã thắng nên $P(0,j) = 1$. Tương tự nếu $j=0$, tức là đội B đã thắng nên $P(i,0) = 0$. Nếu i và j đều lớn hơn không thì ít nhất còn một hiệp nữa phải đấu và hai đội có khả năng 5 ăn, 5 thua trong hiệp này. Như vậy $P(i,j)$ là trung bình cộng của $P(i-1,j)$ và $P(i,j-1)$. Trong đó $P(i-1,j)$ là xác suất để đội A thắng cuộc nếu nó thắng hiệp đó và $P(i,j-1)$ là xác suất để A thắng cuộc nếu nó thua hiệp đó. Tóm lại ta có công thức tính $P(i,j)$ như sau:

$$\begin{aligned} P(i,j) &= 1 && \text{Nếu } i = 0 \\ P(i,j) &= 0 && \text{Nếu } j = 0 \\ P(i,j) &= (P(i-1,j) + P(i,j-1))/2 && \text{Nếu } i > 0 \text{ và } j > 0 \end{aligned}$$

- Viết một hàm đệ quy để tính $P(i,j)$. Tính độ phức tạp của hàm đó.
- Dùng kĩ thuật quy hoạch động để viết hàm tính $P(i,j)$. Tính độ phức tạp của hàm đó.
- Viết hàm $P(i,j)$ bằng kĩ thuật quy hoạch động nhưng chỉ dùng mảng một chiều (để tiết kiệm bộ nhớ).

Bài 2: Bài toán phân công lao động: Có n công nhân có thể làm n công việc. Công nhân i làm công việc j trong một khoảng thời gian t_{ij} . Phải tìm một phương án phân công như thế nào để các công việc đều được hoàn thành, các công nhân đều có việc làm, mỗi công nhân chỉ làm một công việc và mỗi công việc chỉ do một công nhân thực hiện đồng thời tổng thời gian là nhỏ nhất.

- Mô tả kĩ thuật “tham ăn” (greedy) cho bài toán phân công lao động.
- Tìm phương án theo giải thuật “háu ăn” cho bài toán phân công lao động được cho trong bảng sau. Trong đó mỗi dòng là một công nhân, mỗi cột là một công

việc, ô (i,j) ghi thời gian t_{ij} mà công nhân i cần để hoàn thành công việc j . (Cần chỉ rõ công nhân nào làm công việc gì và tổng thời gian là bao nhiêu)

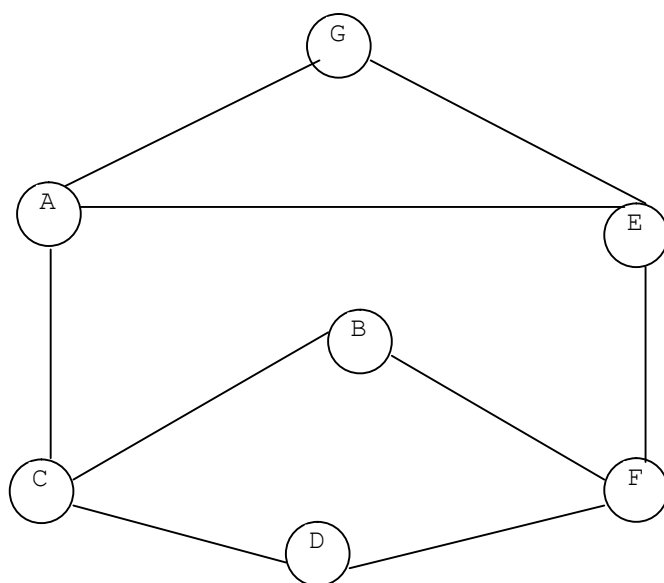
Công nhân \ Công việc	1	2	3	4	5
1	5	6	4	7	2
2	5	2	4	5	1
3	4	5	4	6	3
4	5	5	3	4	2
5	3	3	5	2	5

Bài 3: Bài toán tô màu bản đồ thế giới

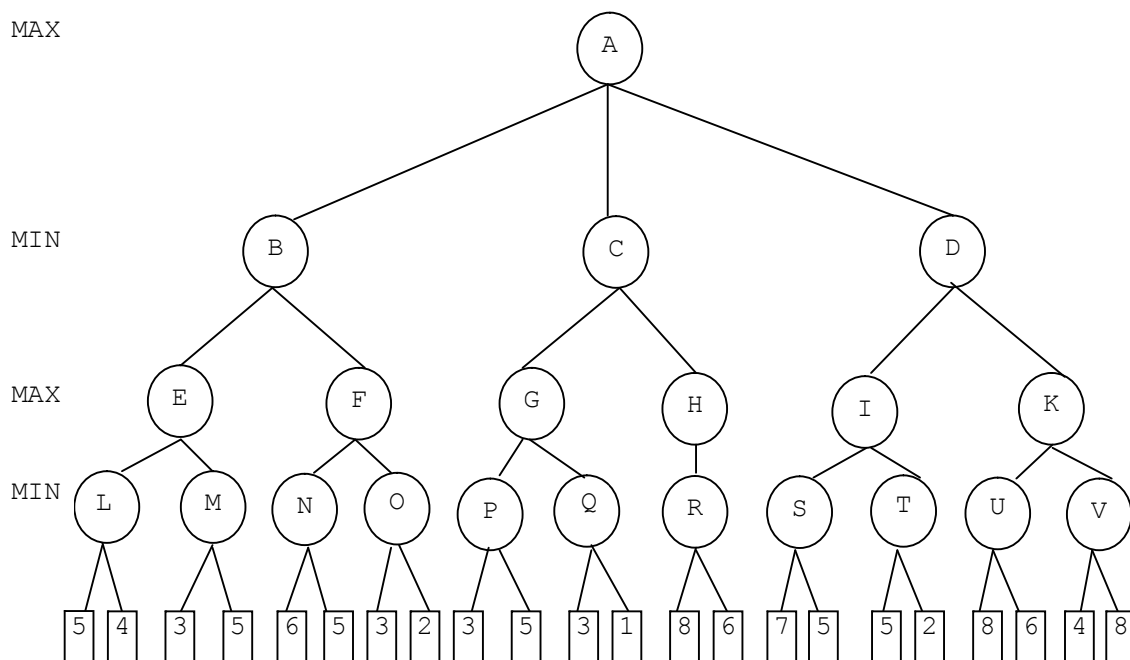
Người ta muốn tô màu bản đồ các nước trên thế giới, mỗi nước đều được tô màu và hai nước có biên giới chung nhau thì không được có màu giống nhau (các nước không chung biên giới có thể được tô màu giống nhau). Tìm một phương án tô màu sao cho số loại màu phải dùng ít nhất.

Người ta có thể mô hình hóa bản đồ thế giới bằng một đồ thị không có hướng, trong đó mỗi đỉnh biểu diễn cho một nước, biên giới của hai nước được biểu diễn bằng cạnh nối hai đỉnh. Bài toán tô màu bản đồ thế giới trở thành bài toán tô màu các đỉnh của đồ thị: Mỗi đỉnh của đồ thị phải được tô màu và hai đỉnh có chung một cạnh thì không được tô cùng một màu (các đỉnh không chung cạnh có thể được tô cùng một màu). Tìm một phương án tô màu sao cho số loại màu phải dùng là ít nhất.

1. Hãy mô tả kỹ thuật “tham ăn” (Greedy) để giải bài toán tô màu cho đồ thị.
2. Áp dụng kỹ thuật hấu ăn để tô màu cho các đỉnh của đồ thị sau (các màu có thể sử dụng để tô là: ĐỎ, CAM, VÀNG, XANH, ĐEN, NÂU, TÍM)



Bài 4: Dùng kỹ thuật cắt tỉa alpha-beta để định trị cho nút gốc của cây trò chơi sau (các số trong các nút lá là các giá trị đã được gán cho chúng)



Bài 5: Xét một trò chơi có 6 viên bi, hai người thay phiên nhau nhặt từ 1 đến 3 viên. Người phải nhặt chỉ một viên bi cuối cùng thì bị thua.

1. Vẽ toán bộ cây trò chơi
2. Sử dụng kĩ thuật cắt tỉa alpha-beta định trị cho nút gốc
3. Ai sẽ thắng trong trò chơi này nếu hai người đều đi những nước tốt nhất. Hãy cho một nhận xét về trường hợp tổng quát khi ban đầu có n viên bi và mỗi lần có thể nhặt từ 1 đến m viên.

Bài 6: Xét một trò chơi có 7 cái đĩa. Người chơi 1 chia thành 2 chồng có số đĩa không bằng nhau. Người chơi 2 chọn một chồng trong số các chồng có thể chia và tiếp tục chia thành hai chồng không bằng nhau. Hai người luân phiên nhau chia đĩa như vậy cho đến khi không thể chia được nữa thì thua.

1. Vẽ toàn bộ cây trò chơi.
2. Sử dụng kĩ thuật cắt tỉa alpha-beta định trị cho nút gốc
3. Ai sẽ thắng trong trò chơi này nếu hai người đều đi những nước tốt nhất.

Bài 7: Cho bài toán cái ba lô với trọng lượng của ba lô $W = 30$ và 5 loại đồ vật được cho trong bảng bên. Tất cả các loại đồ vật đều **chỉ có một cái**.

1. Giải bài toán bằng kỹ thuật “Tham ăn” (Greedy).
2. Giải bài toán bằng kỹ thuật nhánh cận.

Loại đồ vật	Trọng lượng	Giá trị
A	15	30
B	10	25
C	2	2
D	4	6
E	8	24

CHƯƠNG 4: CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT LƯU TRỮ NGOÀI

4.1 TỔNG QUAN

4.1.1 Mục tiêu

Sau khi học chương này, sinh viên cần nắm được các vấn đề sau:

- Tiêu chuẩn để đánh giá giải thuật xử lý ngoài.
- Giải thuật sắp xếp trộn để sắp xếp ngoài và phương pháp cải tiến tốc độ sắp xếp trộn.
- Cách thức tổ chức lưu trữ và các giải thuật tìm kiếm, xen, xoá thông tin trên các tập tin tuần tự, tập tin chỉ mục, tập tin bảng băm và đặc biệt là tập tin B-cây.

4.1.2 Kiến thức cơ bản cần thiết

- Cấu trúc dữ liệu danh sách liên kết.
- Các cấu trúc dữ liệu cây và bảng băm.
- Vấn đề tìm kiếm tuần tự và tìm kiếm nhị phân.
- Các thao tác trên kiểu dữ liệu tập tin.

4.1.3 Tài liệu tham khảo

A.V. Aho, J.E. Hopcroft, J.D. Ullman; *Data Structures and Algorithms*; Addison-Wesley; 1983. (Chapter 10).

Đinh Mạnh Tường; *Cấu trúc dữ liệu & Thuật toán*; Nhà xuất bản khoa học và kỹ thuật; Hà Nội-2001. (Chương 7).

4.1.4 Nội dung cốt lõi

Trong chương này chúng ta sẽ nghiên cứu hai vấn đề chính là sắp xếp dữ liệu được lưu trong bộ nhớ ngoài và kỹ thuật lưu trữ tập tin. Trong kỹ thuật lưu trữ tập tin chúng ta sẽ sử dụng các cấu trúc dữ liệu tuần tự, bảng băm, tập tin chỉ mục và cấu trúc B-cây.

4.2 MÔ HÌNH XỬ LÝ NGOÀI

Trong các giải thuật mà chúng ta đã đề cập từ trước tới nay, chúng ta đã giả sử rằng số lượng các dữ liệu vào là khá nhỏ để có thể chứa hết ở bộ nhớ trong (main memory). Nhưng điều gì sẽ xảy ra nếu ta muốn xử lý phiếu điều tra dân số toàn quốc hay thông tin về quản lý đất đai cả nước chẳng hạn? Trong các bài toán như vậy, số lượng dữ liệu vượt quá khả năng lưu trữ của bộ nhớ trong. Để có thể giải quyết các bài toán đó chúng ta phải dùng bộ nhớ ngoài để lưu trữ và xử lý. Các thiết

bị lưu trữ ngoài như băng từ, đĩa từ đều có khả năng lưu trữ lớn nhưng đặc điểm truy nhập hoàn toàn khác với bộ nhớ trong. Chúng ta cần tìm các cấu trúc dữ liệu và giải thuật thích hợp cho việc xử lý dữ liệu lưu trữ trên bộ nhớ ngoài.

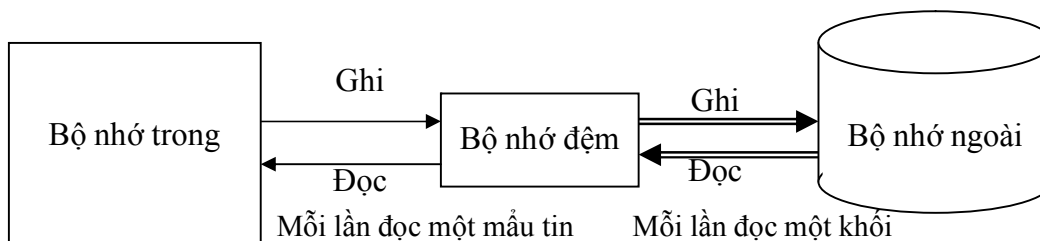
Kiểu dữ liệu tập tin là kiểu thích hợp nhất cho việc biểu diễn dữ liệu được lưu trong bộ nhớ ngoài. Hệ điều hành chia bộ nhớ ngoài thành các khối (block) có kích thước bằng nhau, kích thước này thay đổi tùy thuộc vào hệ điều hành nhưng nói chung là từ 512 bytes đến 4096 bytes.

Trong quá trình xử lý, việc chuyển giao dữ liệu giữa bộ nhớ trong và bộ nhớ ngoài được tiến hành thông qua vùng nhớ đệm (buffer). Bộ đệm là một vùng dành riêng của bộ nhớ trong mà kích thước bằng với kích thước của một khối của bộ nhớ ngoài.

Có thể xem một tập tin bao gồm nhiều mẫu tin được lưu trong các khối. Mỗi khối lưu một số nguyên vẹn các mẫu tin, không có mẫu tin nào bị chia cắt để lưu trên hai khối khác nhau.

Trong thao tác đọc, nguyên một khối của tập tin được chuyển vào trong bộ đệm và lần lượt đọc các mẫu tin có trong bộ đệm cho tới khi bộ đệm rỗng thì lại chuyển một khối từ bộ nhớ ngoài vào bộ đệm.

Để ghi thông tin ra bộ nhớ ngoài, các mẫu tin lần lượt được xếp vào trong bộ đệm cho đến khi đầy bộ đệm thì nguyên một khối được chuyển ra bộ nhớ ngoài. Khi đó bộ đệm trở nên rỗng và lại có thể xếp tiếp các mẫu tin vào trong đó.



Hình 4-1: Mô hình giao tiếp giữa bộ nhớ trong, bộ nhớ ngoài và vùng nhớ đệm

Như vậy đơn vị giao tiếp giữa bộ nhớ trong và bộ đệm là mẫu tin còn giữa bộ đệm và bộ nhớ ngoài là khối.

Hình 4-1 mô tả hoạt động của bộ nhớ trong, bộ đệm và bộ nhớ ngoài trong thao tác đọc và ghi tập tin

4.3 ĐÁNH GIÁ CÁC GIẢI THUẬT XỬ LÝ NGOÀI

Đối với bộ nhớ ngoài thì thời gian tìm một khối để đọc vào bộ nhớ trong là rất lớn so với thời gian thao tác trên dữ liệu trong khối đó. Ví dụ giả sử ta có một khối có thể lưu 1000 số nguyên được lưu trên đĩa quay với vận tốc 1000 vòng/ phút thì thời gian để đưa đầu từ vào rãnh chứa khối và quay đĩa để đưa khối đến chỗ đầu từ hết khoảng 100 mili giây. Với thời gian này máy có thể thực hiện 100000 lệnh, tức là đủ để sắp xếp các số nguyên này theo giải thuật QuickSort. Vì vậy khi đánh giá các

giải thuật thao tác trên bộ nhớ ngoài, chúng ta tập trung vào việc xét số lần đọc khối vào bộ nhớ trong và số lần ghi khối ra bộ nhớ ngoài ta gọi chung là phép truy xuất khối (block access). Vì kích thước các khối là cố định nên ta không thể tìm cách tăng kích thước một khối mà chúng ta phải tìm cách giảm số lần truy xuất khối.

4.4 SẮP XẾP NGOÀI

Sắp xếp dữ liệu được tổ chức như một tập tin hoặc tổng quát hơn, sắp xếp dữ liệu được lưu trên bộ nhớ ngoài gọi là sắp xếp ngoài.

4.4.1 Sắp xếp trộn (merge sorting)

4.4.1.1 Khái niệm về đường

Đường độ dài k là một tập hợp k mẫu tin đã được sắp thứ tự theo khoá tức là, nếu các mẫu tin $r_1, r_2, \dots, r_k$ có khoá lần lượt là $k_1, k_2, \dots, k_k$ tạo thành một đường thì $k_1 \leq k_2 \leq \dots \leq k_k$.

Cho tập tin chứa các mẫu tin $r_1, r_2, \dots, r_n$, ta nói tập tin được tổ chức thành đường có độ dài k nếu ta chia tập tin thành các đoạn k mẫu tin liên tiếp và mỗi đoạn là một đường, đoạn cuối có thể không có đủ k mẫu tin, trong trường hợp này ta gọi đoạn ấy là đuôi (tail).

Ví dụ 4-1: Tập tin gồm 14 mẫu tin có khóa là các số nguyên được tổ chức thành 4 đường độ dài 3 và một đuôi có độ dài 2

5	6	9	13	26	27	1	5	8	12	14	17	23	25
---	---	---	----	----	----	---	---	---	----	----	----	----	----

4.4.1.2 Giải thuật

Để sắp xếp tập tin F có n mẫu tin ta sử dụng 4 tập tin $F1, F2, G1$ và $G2$.

Khởi đầu ta phân phối các mẫu tin của tập tin đã cho F luân phiên vào trong hai tập tin $F1, F2$. Như vậy hai tập tin này được xem như được tổ chức thành các đường độ dài 1.

Bước 1: Đọc 2 đường, mỗi đường độ dài 1 từ hai tập tin $F1, F2$ và trộn hai đường này thành đường độ dài 2 và ghi luân phiên vào trong hai tập tin $G1, G2$. Đổi vai trò của $F1$ cho $G1, F2$ cho $G2$.

Bước 2: Đọc 2 đường, mỗi đường độ dài 2 từ hai tập tin $F1, F2$ và trộn hai đường này thành đường độ dài 4 và ghi luân phiên vào trong hai tập tin $G1, G2$. Đổi vai trò của $F1$ cho $G1, F2$ cho $G2$.

Quá trình trên cứ tiếp tục và sau i bước thì độ dài của một đường là 2^i . Nếu $2^i \geq n$ thì giải thuật kết thúc, lúc đó tập tin $G2$ sẽ rỗng và tập tin $G1$ chứa các mẫu tin đã được sắp.

4.4.1.3 Đánh giá giải thuật sắp xếp trộn

Ta thấy giải thuật kết thúc sau i bước với $i \geq \log n$. Mỗi bước phải đọc từ 2 tập tin và ghi vào 2 tập tin, mỗi tập tin có trung bình $n/2$ mẫu tin. Giả sử mỗi một khối lưu trữ

được b mẫu tin thì mỗi bước cần đọc và ghi $\frac{2 \cdot 2 \cdot n}{2 \cdot b} = \frac{2n}{b}$ khối mà chúng ta cần $\log n$ bước vậy tổng cộng chúng ta cần $\frac{2n}{b} \log n$ phép truy xuất khối.

Ví dụ 4-2: Cho tập tin F có 23 mẫu tin với khóa là các số nguyên như sau:

2 31 13 5 98 96 10 40 54 85 65 9 30 39 90 13 10 8 69 77 8 10 22.

Để bắt đầu ta phân phối các mẫu tin của F luân phiên vào hai tập tin $F1$ và $F2$ được tổ chức thành các đường có độ dài 1

2	13	98	10	54	65	30	90	10	69	8	22
---	----	----	----	----	----	----	----	----	----	---	----

F1

31	5	96	40	85	9	39	13	8	77	10
----	---	----	----	----	---	----	----	---	----	----

F2

Bước 1: Trộn các đường độ dài 1 của $F1$ và $F2$ được các đường độ dài 2 và ghi luân phiên vào trong hai tập tin $G1, G2$:

2	31	96	98	54	85	30	39	8	10	8	10
---	----	----	----	----	----	----	----	---	----	---	----

F1

5	13	10	40	9	65	13	90	69	77	22
---	----	----	----	---	----	----	----	----	----	----

F2

Bước 2: Đổi vai trò của $F1$ và $G1, F2$ và $G2$ cho nhau. Trộn các đường độ dài 2 trong hai tập tin $F1$ và $F2$ được các đường độ dài 4 rồi ghi luân phiên vào trong hai tập tin $G1$ và $G2$:

2	5	13	31	9	54	65	85	8	10	69	77
---	---	----	----	---	----	----	----	---	----	----	----

F1

10	40	96	98	13	30	39	90	8	10	22
----	----	----	----	----	----	----	----	---	----	----

F2

Bước 3: Đổi vai trò của $F1$ và $G1, F2$ và $G2$ cho nhau. Trộn các đường độ dài 4 trong hai tập tin $F1$ và $F2$ được các đường độ dài 8 rồi ghi luân phiên vào trong hai tập tin $G1$ và $G2$:

2	5	10	13	31	40	96	98	8	8	10	10	22	69	77
---	---	----	----	----	----	----	----	---	---	----	----	----	----	----

F1

9	13	30	39	54	65	85	90
---	----	----	----	----	----	----	----

F2

Bước 4: Đổi vai trò của $F1$ và $G1, F2$ và $G2$ cho nhau. Trộn các đường độ dài 8 trong hai tập tin $F1$ và $F2$ được các đường độ dài 16 rồi ghi luân phiên vào trong 2 tập tin $G1$ và $G2$.

2	5	9	10	13	13	30	31	39	40	54	65	85	90	96	98
---	---	---	----	----	----	----	----	----	----	----	----	----	----	----	----

F1

8	8	10	10	22	69	77
---	---	----	----	----	----	----

F2

Bước 5: Đổi vai trò của F1 và G1, F2 và G2 cho nhau. Trộn các đường độ dài 16 trong hai tập tin F1 và F2 được 1 đường độ dài 23 rồi ghi vào trong tập tin G1.

G1

2	5	8	8	9	10	10	10	13	13	22	30	31	39	40	54	65	69	77	85	90	96	98
---	---	---	---	---	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----

Tập tin G1 chứa các mẫu tin đã được sắp xếp còn tập tin G2 rỗng.

4.4.1.4 Chương trình

```
procedure Merge(k:integer; f1,f2,g1,g2: File of RecordType);
{Thủ tục này trộn các đường độ dài k và trong hai tập tin f1
và f2 thành các đường độ dài 2k và ghi luân phiên vào trong
hai tập tin g1 và g2}
```

```
var
```

```
OutSwitchh : boolean; {Nếu OutSwitch = TRUE thì ghi vào tập
tin g1, ngược lại ghi vào g2}
```

```
Winner: integer; {Để chỉ định mẫu tin hiện hành nào trong hai
tập tin f1 và f2 sẽ được ghi ra tập tin g1 hoặc g2}
```

```
Used: array[1..2] of integer; { Used[ij] ghi số mẫu tin đã
được đọc trong đường hiện tại của tập tin fj }
```

```
Fin : array[1..2] Of boolean; {Fin[j] sẽ có giá trị TRUE nếu
đã đọc hết các mẫu tin trong đường hiện hành của fj hoặc đx
đến cuối tập tin fj }
```

```
Current: array[1..2] Of RecordType; { Current[j] để lưu mẫu
tin hiện hành của tập tin f[j]}
```

```
procedure GetRecord(i:integer);
{Nếu đã đọc hết các mẫu tin trong đường hiện hành của tập tin
fi hoặc đã đến cuối tập tin fi thì đặt fin[i] = TRUE nếu
không thì đọc một mẫu tin của tập tin fi vào trong
current[i]}
```

```
begin
```

```
    Used[i] := Used[i] + 1;
```

```
    if (Used[i] = k+1 ) or (i = 1) and ( eof(f1)) or (i = 2
and ( eof(f2)) then fin[i] := TRUE
```

```
    else if i=1 then Read(f1, current[1])
```

```
        else read(f2, current[2]);
```

```
end;
```

```
begin
```

```
    { Khởi tạo }
```

```
    OutSwitch := TRUE;
```

```
ReSet(f1);
```

```
ReSet(f2);
```

```

ReWrite(g1);
ReWrite(g2);
while (not eof(f1)) or (not eof(f2)) do begin
    {Bắt đầu đọc các mẫu tin từ trong hai đường hiển
    hành của hai tập tin f1,f2 }
    Used[1] := 0; Used[2] := 0;
    Fin[1] := FALSE ; Fin[2] := FALSE ;
    GetRecord(1) ; GetRecord(2);
    while ( not fin[1] ) or (not fin[2]) do begin
        {Trộn hai đường }
        { Chọn Winner }
        if Fin[1] then Winner := 2
        else if Fin[2] then Winner := 1
            else if current[1].key < Current[2].key
            then
                Winner := 1
            else Winner := 2;
        if OutSwitch then Write(g1, Current[winner] )
        else Write(g2, current[winner] );
        GetRecord(Winner);
    end;
    OutSwitch := Not OutSwitch;
end;
end;

```

4.4.2 Cải tiến sắp xếp trộn

Ta thấy quá trình sắp xếp trộn nói trên bắt đầu từ các đường độ dài 1 cho nên phải sau $\log n$ bước giải thuật mới kết thúc. Chúng ta có thể tiết kiệm thời gian bằng cách chọn một số k thích hợp sao cho k mẫu tin có thể đủ chứa trong bộ nhớ trong. Mỗi lần đọc vào bộ nhớ trong k mẫu tin, dùng sắp xếp trong (chẳng hạn dùng QuickSort) để sắp xếp k mẫu tin này và ghi luân phiên vào hai tập tin F1 và F2. Như vậy chúng ta bắt đầu sắp xếp trộn với các tập tin được tổ chức thành các đường độ dài k .

Sau i bước thì độ dài mỗi đường là $k \cdot 2^i$. Giải thuật sẽ kết thúc khi $k \cdot 2^i \geq n$ hay $i \geq \log \frac{n}{k}$. Do đó số phép truy xuất khối sẽ là $\frac{2n}{b} \log \frac{n}{k}$. Dễ thấy $\frac{2n}{b} \log \frac{n}{k} < \frac{2n}{b} \log n$ tức là ta tăng được tốc độ sắp xếp trộn.

Ví dụ 4-3: Lấy tập tin F có 23 mẫu tin với khóa là các số nguyên như trong ví dụ 4-2:

2 31 13 5 98 96 10 40 54 85 65 9 30 39 90 13 10 8 69 77 8 10 22.

Ta giả sử bộ nhớ trong có thể chứa được 3 mẫu tin, ta đọc lần lượt 3 mẫu tin của F vào bộ nhớ trong, dùng một sắp xếp trong để sắp xếp chúng và ghi phiên vào 2 tập tin F1 và F2.

2 13 31	10 40 54	30 39 90	8 69 77
---------	----------	----------	---------

F1

5 96 98	9 65 85	8 10 13	10 22	F2
---------	---------	---------	-------	----

Bước 1: Trộn các đường độ dài 3 của F1 và F2 được các đường độ dài 6 và ghi luân phiên vào trong hai tập tin G1, G2:

G1	2 5 13 31 96 98	8 10 13 30 39 90	F1
----	-----------------	------------------	----

G2	9 10 40 54 65 85	8 10 22 69 77	F2
----	------------------	---------------	----

Bước 2: Đổi vai trò của F1 và G1, F2 và G2 cho nhau. Trộn các đường độ dài 6 trong 2 tập tin F1 và F2 được các đường độ dài 12 rồi ghi luân phiên vào trong 2 tập tin G1 và G2:

G1	2 5 9 10 13 31 40 54 65 85 96 98	F1
----	----------------------------------	----

G2	8 8 10 10 13 22 30 39 69 77 90	F2
----	--------------------------------	----

Bước 3: Đổi vai trò của F1 và G1, F2 và G2 cho nhau. Trộn các đường độ dài 12 trong 2 tập tin F1 và F2 được 1 đường ghi vào trong tập tin G1, còn G2 rỗng

G1	2 5 8 8 9 10 10 10 13 13 22 30 31 39 40 54 65 77 85 90 96 98
----	--

Tập tin G1 chứa các mẫu tin đã được sắp còn tập tin G2 rỗng.

4.4.3 Trộn nhiều đường (multiway merge)

4.4.3.1 Giải thuật

Để sắp xếp tập tin F có n mẫu tin ta sử dụng m tập tin (m là một số chẵn) F[1], F[2], ..., F[m]. Trong trường hợp m=4 ta có giải thuật sắp xếp trộn bình thường.

Gọi $h = m/2$, ta có nội dung của phương pháp như sau (ta vẫn giả sử bộ nhớ trong có thể chứa k mẫu tin).

Khởi đầu: Mỗi lần đọc từ tập tin F vào bộ nhớ trong k mẫu tin, sử dụng một sắp xếp trong để sắp xếp k mẫu tin này thành một đường rồi ghi luân phiên vào các tập tin F[1], F[2], ..., F[h].

Bước 1: Trộn các đường độ dài k của h tập tin F[1], F[2], ..., F[h] thành một đường độ dài $k.h$ và ghi luân phiên vào trong h tập tin F[h+1], F[h+2], ..., F[m]. Đổi vai trò của F[i] và F[h+i] cho nhau (với $1 \leq i \leq h$).

Bước 2: Trộn các đường độ dài kh của h tập tin F[1], F[2], ..., F[h] thành một đường độ dài $k.h^2$ và ghi luân phiên vào trong h tập tin F[h+1], F[h+2], ..., F[m]. Đổi vai trò của F[i] và F[h+i] cho nhau (với $1 \leq i \leq h$).

Sau i bước thì độ dài mỗi đường là $k.h^i$ và giải thuật kết thúc khi $k.h^i \geq n$ và khi đó tập tin đã được sắp chính là một đường ghi trong F[h+1].

4.4.3.2 Đánh giá giải thuật sắp xếp trộn nhiều đường

Theo trên thì giải thuật kết thúc sau i bước, với $kh^i \geq n$ hay $i \geq \log_h \frac{n}{k}$. Mỗi bước ta phải đọc từ h tập tin và ghi vào trong h tập tin, trung bình mỗi tập tin có $\frac{n}{h}$ mẫu tin. Ta vẫn giả sử mỗi khối lưu được b mẫu tin thì mỗi bước phải truy xuất $\frac{2 * h * n}{h * b} = \frac{2n}{b}$ khối. Do chúng ta cần $\log_h \frac{n}{k}$ bước nên tổng cộng ta chỉ cần $\frac{2n}{b} \log_h \frac{n}{k}$ phép truy xuất khối. Ta thấy rõ ràng $\frac{2n}{b} \log_h \frac{n}{k} < \frac{2n}{b} \log \frac{n}{k}$ và thủ tục mergeSort nói trên là một trường hợp đặc biệt khi $h = 2$.

Ví dụ 4-4: Lấy tập tin F có 23 mẫu tin với khóa là các số nguyên như trong ví dụ 4-2

2 31 13 5 98 96 10 40 54 85 65 9 30 39 90 13 10 8 69 77 8 10 22.

Sử dụng 6 tập tin để sắp xếp tập tin F . Ta giả sử bộ nhớ trong có thể chứa được 3 mẫu tin, ta đọc lần lượt 3 mẫu tin của F vào bộ nhớ trong, dùng một sắp xếp trong để sắp xếp chúng và ghi phiên vào 3 tập tin $F[1]$, $F[2]$ và $F[3]$ như sau:

$F[1]$

2	13	31	9	65	85	8	69	77
---	----	----	---	----	----	---	----	----

$F[2]$

5	96	98	30	39	90	10	22
---	----	----	----	----	----	----	----

$F[3]$

10	40	54	8	10	13
----	----	----	---	----	----

Bước 1: Trộn các đường độ dài 3 trong các tập tin $F[1]$, $F[2]$, $F[3]$ thành các đường độ dài 9 và ghi vào trong các tập tin $F[4]$, $F[5]$ và $F[6]$.

$F[4]$

2	5	10	13	31	40	54	96	98
---	---	----	----	----	----	----	----	----

 $F[1]$

$F[5]$

8	9	10	13	30	39	65	85	90
---	---	----	----	----	----	----	----	----

 $F[2]$

$F[6]$

8	10	22	69	77
---	----	----	----	----

 $F[3]$

Bước 2: Đổi vai trò của $F[1]$ cho $F[4]$, $F[2]$ cho $F[5]$ và $F[3]$ cho $F[6]$. Trộn các đường độ dài 9 trong các tập tin $F[1]$, $F[2]$, $F[3]$ thành 1 đường độ dài 23 và ghi vào trong tập tin $F[4]$.

$F[4]$

2	5	8	8	9	10	10	10	13	13	22	30	31	39	40	54	65	69	77	85	90	96	98
---	---	---	---	---	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----

Tập tin $F[4]$ chứa các mẫu tin đã được sắp còn $F[5]$ và $F[6]$ rỗng.

4.5 LƯU TRỮ THÔNG TIN TRONG TẬP TIN

Trong phần này ta sẽ nghiên cứu các cấu trúc dữ liệu và giải thuật cho lưu trữ (storing) và lấy thông tin (retrieving) trong các tập tin được lưu trữ ngoài. Chúng ta sẽ coi một tập tin như là một chuỗi tuần tự các mẫu tin, mỗi mẫu tin bao gồm nhiều trường (field). Một trường có thể có độ dài cố định hoặc độ dài thay đổi. Ở đây ta sẽ xét các mẫu tin có độ dài cố định và khảo sát các thao tác trên tập tin là:

- Insert: Thêm một mẫu tin vào trong một tập tin,
- Delete: Xoá một mẫu tin từ trong tập tin,
- Modify: Sửa đổi thông tin trong các mẫu tin của tập tin, và
- Retrieve: Tìm lại thông tin được lưu trong tập tin.

Sau đây ta sẽ nghiên cứu một số cấu trúc dữ liệu dùng để lưu trữ tập tin. Với mỗi cấu trúc chúng ta sẽ trình bày tổ chức, cách thức tiến hành các thao tác tìm, thêm, xoá mẫu tin và có đánh giá về cách tổ chức đó. Sự đánh giá ở đây chủ yếu là đánh giá xem để tìm một mẫu tin thì phải đọc bao nhiêu khối vì các thao tác khác đều phải sử dụng thao tác tìm.

4.5.1 Tập tin tuần tự

4.5.1.1 Tổ chức

Tập tin tuần tự là một danh sách liên kết của các khối, các mẫu tin được lưu trữ trong các khối theo một thứ tự bất kỳ.

4.5.1.2 Tìm mẫu tin

Việc tìm kiếm một mẫu tin có giá trị xác định được thực hiện bằng cách đọc từng khối, với mỗi khối ta tìm mẫu tin cần tìm trong khối, nếu không tìm thấy ta lại đọc tiếp một khối khác. Quá trình cứ tiếp tục cho đến khi tìm thấy mẫu tin hoặc duyệt qua toàn bộ các khối của tập tin và trong trường hợp đó thì mẫu tin không tồn tại trong tập tin.

4.5.1.3 Thêm mẫu tin mới

Việc thêm một mẫu tin có thể thực hiện đơn giản bằng cách đưa mẫu tin này vào khối cuối cùng của tập tin nếu như khối đó còn chỗ trống. Ngược lại nếu khối cuối cùng đã hết chỗ thì xin cấp thêm một khối mới, thêm mẫu tin vào khối mới và nối khối mới vào cuối danh sách.

4.5.1.4 Sửa đổi mẫu tin

Để sửa đổi một mẫu tin có giá trị cho trước, ta tìm mẫu tin cần sửa đổi rồi thực hiện các sửa đổi cần thiết sau đó ghi lại mẫu tin vào vị trí cũ trong tập tin.

4.5.1.5 Xoá mẫu tin

Để xoá một mẫu tin, trước hết ta cũng cần tìm mẫu tin đó, nếu tìm thấy ta có thể thực hiện một trong các cách xoá sau đây:

Một là xoá mẫu tin cần xoá trong khối lưu trữ nó, nếu sau khi xoá, khối trở nên rỗng thì xoá khối khỏi danh sách (giải phóng bộ nhớ).

Hai là đánh dấu xoá mẫu tin bằng một cách nào đó. Nghĩa là chỉ xoá mẫu tin một cách logic, vùng không gian nhớ vẫn còn dành cho mẫu tin. Việc đánh dấu có thể được thực hiện bằng một trong hai cách:

- Thay thế mẫu tin bằng một giá trị nào đó mà giá trị này không bao giờ là giá trị thật của bất kỳ một mẫu tin nào.
- Mỗi một mẫu tin có một bit xoá, bình thường bit xoá của mẫu tin có giá trị 0, muốn xoá mẫu tin ta đặt cho bit xoá giá trị 1. Với phương pháp này thì một mẫu tin sau khi bị đánh dấu xoá cũng có thể phục hồi được bằng cách đặt bit xoá của mẫu tin giá trị 0.

4.5.1.6 Đánh giá

Đây là một phương pháp tổ chức tập tin đơn giản nhất nhưng kém hiệu quả nhất. Ta thấy tập tin là một danh sách liên kết của các khối nên các thao tác trên tập tin đều đòi hỏi phải truy xuất hầu như tất cả các khối, từ khối đầu tiên đến khối cuối cùng.

Giả sử tập tin có n mẫu tin và mỗi khối lưu trữ được k mẫu tin thì toàn bộ tập tin được lưu trữ trong $\frac{n}{k}$ khối, do đó mỗi lần tìm (hoặc thêm hoặc sửa hoặc xoá) một mẫu tin thì phải truy xuất $\frac{n}{k}$ khối.

4.5.2 Tăng tốc độ cho các thao tác tập tin

Nhược điểm của cách tổ chức tập tin tuần tự ở trên là các thao tác trên tập tin rất chậm. Để cải thiện tốc độ thao tác trên tập tin, chúng ta phải tìm cách giảm số lần truy xuất khối. Muốn vậy phải tìm các cấu trúc sao cho khi tìm một mẫu tin chỉ cần phép truy xuất một số nhỏ các khối của tập tin.

Để tạo ra các tổ chức tập tin như vậy chúng ta phải giả sử rằng mỗi mẫu tin có một khoá (key), đó là một tập hợp các trường mà căn cứ vào đó ta có thể phân biệt các mẫu tin với nhau. Hai mẫu tin khác nhau thì khoá của chúng phải khác nhau. Chẳng hạn mã sinh viên trong mẫu tin về sinh viên, biển số xe trong quản lý các phương tiện vận tải đường bộ.

Sau đây ta sẽ xét một số cấu trúc như thế.

4.5.3 Tập tin băm (hash files)

4.5.3.1 Tổ chức

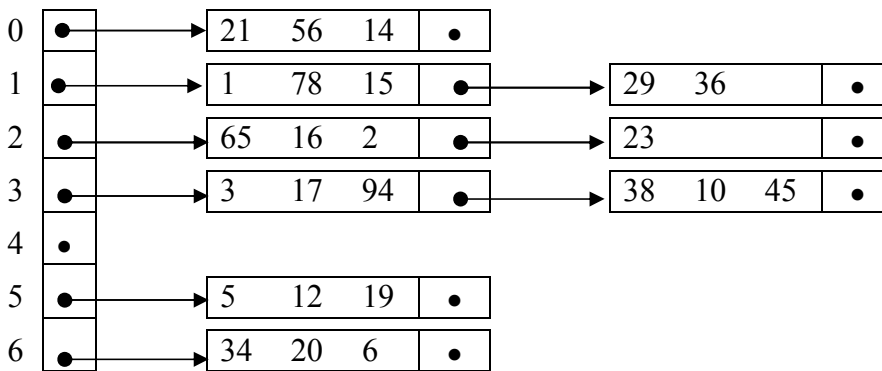
Ta sẽ sử dụng bảng băm mở để lưu trữ tập tin. Bảng băm là một bảng có m phần tử, mỗi phần tử được đánh số từ 0 đến $m-1$ (đơn giản nhất là mảng một chiều B gồm m phần tử $B[0], B[1], \dots, B[m-1]$). Mỗi phần tử là một con trỏ, trỏ tới phần tử đầu tiên của danh sách liên kết các khối.

Để phân phối các mẫu tin có khóa x vào trong các danh sách liên kết, ta dùng hàm băm (hash function). Hàm băm $h(x)$ ánh xạ mỗi giá trị khóa x với một số nguyên từ 0 đến $m-1$. Nếu $h(x) = i$ thì mẫu tin r có khóa x sẽ được đưa vào một khối nào đó trong danh sách liên kết được trỏ bởi $B[i]$.

Có nhiều phương pháp để xác định hàm băm. Cách đơn giản nhất là “nguyên hóa” giá trị khóa x (nếu x không phải là một số nguyên) sau đó ta cho $h(x) = x \text{ MOD } m$.

Ví dụ 4-5: Một tập tin có 24 mẫu tin với giá trị khóa là các số nguyên: 3, 5, 12, 65, 34, 20, 21, 17, 56, 1, 16, 2, 78, 94, 38, 15, 23, 14, 10, 29, 19, 6, 45, 36

Giả sử chúng ta có thể tổ chức tập tin này vào trong bảng băm gồm 7 phần tử và giả sử mỗi khối có thể chứa được tối đa 3 mẫu tin. Với mỗi mẫu tin r có khóa là x ta xác định $h(x) = x \text{ MOD } 7$ và đưa mẫu tin r vào trong một khối của danh sách liên kết được trỏ bởi $B[h(x)]$.



Mảng B Các lô được tổ chức bởi các danh sách liên kết.

Hình 4-2: Tập tin được tổ chức bởi bảng băm

4.5.3.2 Tìm mẫu tin

Để tìm một mẫu tin r có khóa là x , chúng ta xác định $h(x)$ chẳng hạn $h(x) = i$, khi đó ta chỉ cần tìm r trong danh sách liên kết được trỏ bởi $B[i]$. Chẳng hạn để tìm mẫu tin r có khóa là 36, ta tính $h(36) = 36 \text{ MOD } 7 = 1$. Như vậy nếu mẫu tin r tồn tại trong tập tin thì nó phải thuộc một khối nào đó được trỏ bởi $B[1]$.

4.5.3.3 Thêm mẫu tin

Để thêm mẫu tin r có khóa x , trước hết ta phải tìm xem đã có mẫu tin nào trong tập tin có khóa x chưa. Nếu có ta cho một thông báo “mẫu tin đã tồn tại” vì theo giả thiết các mẫu tin không có khóa trùng nhau. Ngược lại ta sẽ tìm một khối (trong danh sách các khối của lô được trỏ bởi $B[h(x)]$) còn chỗ trống và thêm r vào khối này. Nếu không còn khối nào đủ chỗ cho mẫu tin mới ta yêu cầu hệ thống cấp phát một khối mới và đặt mẫu tin r vào khối này rồi nối khối mới này vào cuối danh sách liên kết của lô.

4.5.3.4 Xoá mẫu tin

Để xoá mẫu tin r có khoá x , trước hết ta phải tìm mẫu tin này. Nếu không tìm thấy thì thông báo “Mẫu tin không tồn tại”. Nếu tìm thấy thì đặt bút xoá cho nó. Ta cũng có thể xoá hẳn mẫu tin r và nếu việc xoá này làm khối trở nên rỗng thì ta giải phóng khối này (xoá khối khỏi danh sách liên kết các khối).

4.5.3.5 Đánh giá

Giả sử tập tin có n mẫu tin và mỗi khối lưu trữ được k mẫu tin thì tập tin cần $\frac{n}{k}$ khối. Trung bình mỗi danh sách liên kết (mỗi lô) của bảng băm có $\frac{n}{m.k}$ khối (do bảng băm có m lô), mà chúng ta chỉ tìm trong một danh sách liên kết nên ta chỉ phải truy xuất $\frac{n}{m.k}$ khối. Số này nhỏ hơn m lần so với cách tổ chức tập tin tuần tự (trong tập tin tuần tự ta cần truy xuất tất cả các khối, tức là $\frac{n}{k}$ khối). Chẳng hạn với 24 mẫu tin như trong ví dụ trên, với cách tổ chức tập tin tuần tự ta cần đúng 8 khối để lưu trữ (vì mỗi khối chứa tối đa 3 mẫu tin). Như vậy để tìm một mẫu tin, chẳng hạn mẫu tin có khoá 36 chúng ta phải đọc đúng 8 khối (do mẫu tin có khoá 36 nằm trong khối cuối cùng). Nhưng với cách tổ chức tập tin bảng băm chúng ta chỉ cần trung bình $\frac{8}{7}$ lần đọc khối. Trong thực tế ta chỉ cần 2 lần đọc khối (vì mẫu tin có khoá 36 nằm trong khối thứ 2 của lô được trỏ bởi $B[1]$).

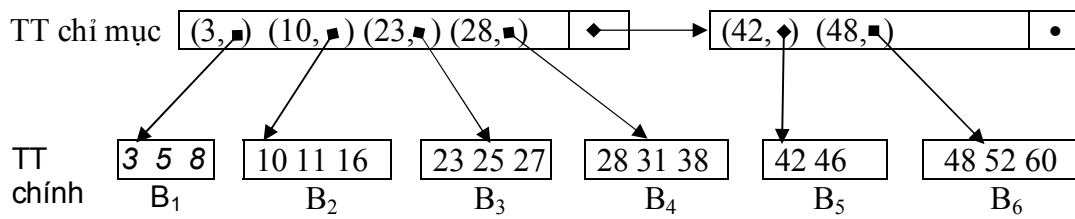
4.5.4 Tập tin chỉ mục (index file)

4.5.4.1 Tổ chức

Một cách khác thường gặp là tập tin được sắp xếp theo khoá, rồi chúng ta tiến hành tìm kiếm như là tìm một từ trong từ điển, tức là tìm kiếm theo từ đầu tiên trên mỗi trang.

Để thực hiện được điều đó ta sử dụng hai tập tin: Tập tin chính và tập tin chỉ mục thưa (sparse index). Tập tin chính bao gồm các khối lưu các mẫu tin sắp thứ tự theo giá trị khoá. Tập tin chỉ mục thưa bao gồm các khối chứa các cặp (x, p) trong đó x là khoá của mẫu tin đầu tiên trong một khối của tập tin chính, còn p là con trỏ, trỏ đến khối đó.

Ví dụ 4-6: Ta có tập tin được tổ chức thành tập tin chỉ mục với mỗi khối trong tập tin chính lưu trữ được tối đa 3 mẫu tin, mỗi khối trong tập tin chỉ mục lưu trữ được tối đa 4 cặp khoá – con trỏ. Hình sau minh hoạ tập tin chỉ mục này.



Hình 4-3: Tập tin chỉ mục

4.5.4.2 Tìm kiếm

Để tìm mẫu tin r có khoá x , ta phải tìm cặp (z, p) với z là giá trị lớn nhất và $z \leq x$. Mẫu tin r có khoá x nếu tồn tại thì sẽ nằm trong khối được trỏ bởi p .

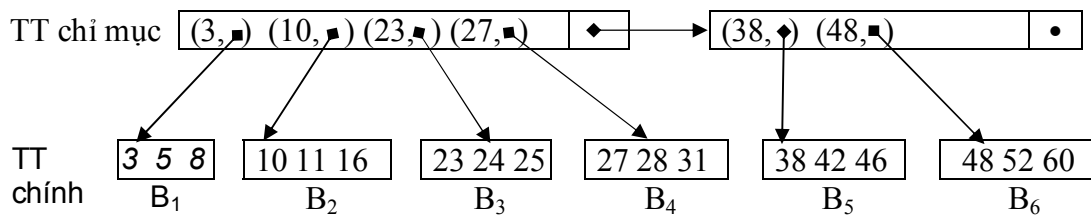
Chẳng hạn để tìm mẫu tin r có khoá 46 trong tập tin của ví dụ 4-6, ta tìm trong tập tin chỉ mục được cặp $(42, p)$, trong đó 42 là giá trị khoá lớn nhất trong tập tin chỉ mục mà $42 \leq 46$ và p là con trỏ, trỏ tới khối B_5 của tập tin chính. Trong khối B_5 , ta tìm thấy mẫu tin có khoá 46.

Việc tìm một mẫu tin trong một khối của tập tin chính có thể tiến hành bằng tìm kiếm tuần tự hoặc bằng tìm kiếm nhị phân bởi lẽ các mẫu tin trong một khối đã được sắp thứ tự.

4.5.4.3 Thêm mẫu tin

Giả sử tập tin chính được lưu trong các khối $B_1, B_2, \dots, B_m$. Để xen một mẫu tin r với khoá x vào trong tập tin, ta phải dùng thủ tục tìm kiếm để xác định một khối B_i nào đó. Nếu tìm thấy thì thông báo “mẫu tin đã tồn tại”, ngược lại, B_i là nơi có thể chứa mẫu tin r . Nếu B_i còn chỗ trống thì xen r vào đúng vị trí của nó trong B_i . Ta phải chỉnh lại tập tin chỉ mục nếu mẫu tin mới trở thành mẫu tin đầu tiên trong khối B_i . Nếu B_i không còn chỗ trống để xen thì ta phải xét khối B_{i+1} để có thể chuyển mẫu tin cuối cùng trong khối B_i thành mẫu tin đầu tiên của khối B_{i+1} và xen mẫu tin r vào đúng vị trí của nó trong khối B_i . Điều chỉnh lại tập tin chỉ mục cho phù hợp với trạng thái mới của B_{i+1} . Quá trình này có thể dẫn đến việc ta phải xét khối B_m , nếu B_m đã hết chỗ thì yêu cầu hệ thống cấp thêm một khối mới B_{m+1} , chuyển mẫu tin cuối cùng của B_m sang B_{m+1} , mẫu tin cuối cùng của B_{m-1} sang B_m ... Xen mẫu tin r vào khối B_i và cập nhật lại tập tin chỉ mục. Việc cấp phát thêm khối mới B_{m+1} đòi hỏi phải xen thêm một cặp khoá-con trỏ vào khối cuối cùng của tập tin chỉ mục, nếu khối này hết chỗ thì xin cấp thêm một khối mới để xen cặp khoá-con trỏ này.

Ví dụ 4-7: Chẳng hạn ta cần xen mẫu tin r với khoá $x = 24$ vào trong tập tin được biểu diễn trong hình 4-3. Thủ tục tìm x trong tập tin chỉ mục xác định được khối cần xen r là khối B_3 . Vì khối B_3 đã có đủ 3 mẫu tin nên phải xét khối B_4 . Khối B_4 cũng đã có đủ 3 mẫu tin nên ta lại xét khối B_5 . Vì B_5 còn chỗ trống nên ta chuyển mẫu tin có khoá 38 từ B_4 sang B_5 và chuyển mẫu tin có khoá 27 từ B_3 sang B_4 và xen r vào khối B_3 . Vì mẫu tin đầu tiên của khối B_4 bây giờ có khoá 27 nên ta phải sửa lại giá trị này trong cặp của tập tin chỉ mục tương ứng với khối B_4 . Ta cũng phải làm tương tự đối với khối B_5 . Cấu trúc của tập tin sau khi thêm mẫu tin r có khoá 24 như sau:



Hình 4-4: Xen mẫu tin vào tập tin chỉ mục

4.5.4.4 Xoá mẫu tin

Để xoá mẫu tin r có khoá x , trước hết ta cần tìm r , nếu không tìm thấy thì thông báo “Mẫu tin không tồn tại”, ngược lại ta xoá mẫu tin r trong khối chứa nó, nếu mẫu tin bị xoá là mẫu tin đầu tiên của khối thì phải cập nhật lại giá trị khoá trong tập tin chỉ mục. Trong trường hợp khối trở nên rỗng sau khi xoá mẫu tin thì giải phóng khối đó và xoá cặp (khoá, con trỏ) của khối trong tập tin chỉ mục. Việc xoá trong tập tin chỉ mục cũng có thể dẫn đến việc giải phóng khối trong tập tin này.

4.5.4.5 Đánh giá

Ta thấy việc tìm một mẫu tin chỉ đòi hỏi phải đọc chỉ một số nhỏ các khối (một khối trong tập tin chính và một số khối trong tập tin chỉ mục). Tuy nhiên trong việc xen thêm mẫu tin, như trên đã nói, có thể phải đọc và ghi tất cả các khối trong tập tin chính. Đây chính là nhược điểm của tập tin chỉ mục.

4.5.5 Tập tin B-cây

4.5.5.1 Cây tìm kiếm m-phân

Cây tìm kiếm m-phân (m-ary tree) là sự tổng quát hoá của cây tìm kiếm nhị phân trong đó mỗi nút có thể có m nút con. Giả sử n_1 và n_2 là hai con của một nút nào đó, n_1 bên trái n_2 thì tất cả các con của n_1 có giá trị nhỏ hơn giá trị của các nút con của n_2 .

Chúng ta có thể sử dụng cây m-phân để lưu trữ các mẫu tin trong tập tin trên bộ nhớ ngoài. Mỗi một nút biểu diễn cho một khối vật lý trong bộ nhớ ngoài. Trong đó các nút lá lưu trữ các mẫu tin của tập tin. Các nút trong lưu trữ m con trỏ, trỏ tới m nút con.

Nếu ta dùng cây tìm kiếm nhị phân n nút để lưu trữ một tập tin thì cần trung bình $\log_2 n$ phép truy xuất khối để tìm kiếm một mẫu tin. Nếu ta dùng cây tìm kiếm m-phân để lưu trữ một tập tin thì chỉ cần $\log_m n$ phép truy xuất khối để tìm kiếm một mẫu tin. Sau đây chúng ta sẽ nghiên cứu một trường hợp đặc biệt của cây tìm kiếm m-phân là B-cây.

4.5.5.2 B-cây (B-tree)

B-cây bậc m là cây tìm kiếm m-phân cân bằng có các tính chất sau:

- Nút gốc hoặc là lá hoặc có ít nhất hai nút con,
- Mỗi nút, trừ nút gốc và nút lá, có từ $\lceil m/2 \rceil$ đến m nút con và
- Các đường đi từ gốc tới lá có cùng độ dài.

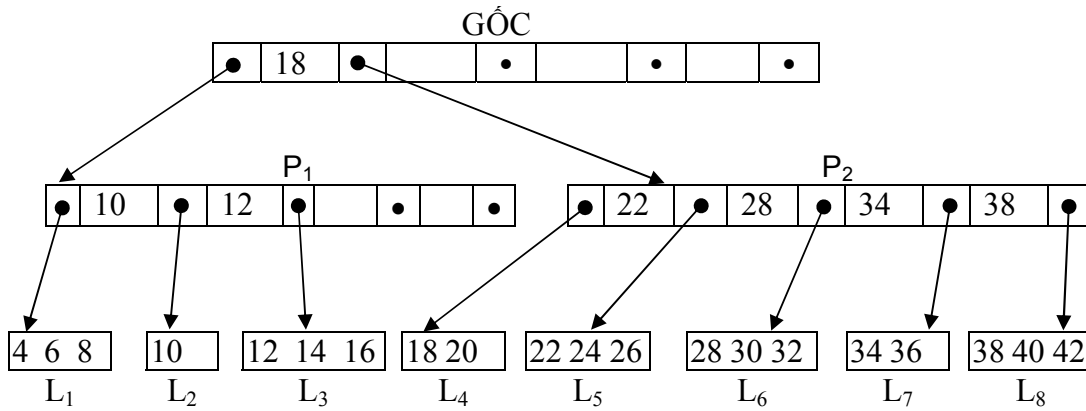
Tổ chức: Ta có thể sử dụng B-cây bậc m để lưu trữ tập tin như sau:

Mỗi nút trên cây là một khối trên đĩa, các mẫu tin của tập tin được lưu trữ trong các nút lá trên B-cây và lưu theo thứ tự của khoá. Giả sử mỗi nút lá lưu trữ được nhiều nhất b mẫu tin.

Mỗi nút không phải là nút lá có dạng $(p_0, k_1, p_1, k_2, p_2, \dots, k_n, p_n)$, với p_i ($0 \leq i \leq n$) là con trỏ, trỏ tới nút con thứ i của nút đó và k_i là các giá trị khoá. Các khoá trong một nút được sắp thứ tự, tức là $k_1 < k_2 < \dots < k_n$.

Tất cả các khoá trong cây con được trỏ bởi p_0 đều nhỏ hơn k_1 . Tất cả các khoá nằm trong cây con được trỏ bởi p_i ($0 < i < n$) đều lớn hơn hoặc bằng k_i và nhỏ hơn k_{i+1} . Tất cả các khoá nằm trong cây con được trỏ bởi p_n đều lớn hơn hoặc bằng k_n .

Ví dụ 4-8: Cho tập tin bao gồm 20 mẫu tin với giá trị khoá là các số nguyên được tổ chức thành B-cây bậc 5 với các nút lá chứa được nhiều nhất 3 mẫu tin.



Hình 4-5: Tập tin B-cây bậc 5

4.5.5.3 Tìm kiếm

Để tìm kiếm một mẫu tin r có khoá là x chúng ta sẽ lần từ nút gốc đến nút lá chứa r (nếu r tồn tại trong tập tin). Tại mỗi bước ta đưa nút trong $(p_0, k_1, p_1, \dots, k_n, p_n)$ vào bộ nhớ trong và xác định mối quan hệ giữa x với các giá trị khoá k_i .

- Nếu $k_i \leq x < k_{i+1}$ ($0 < i < n$) chúng ta sẽ xét tiếp nút được trỏ bởi p_i ,
- Nếu $x < k_1$ ta sẽ xét tiếp nút được trỏ bởi p_0 và
- Nếu $x \geq k_n$ ta sẽ xét tiếp nút được trỏ bởi p_n .

Quá trình trên sẽ dẫn đến việc xét một nút lá. Trong nút lá này ta sẽ tìm mẫu tin r với khoá x bằng tìm kiếm tuần tự hoặc tìm kiếm nhị phân.

4.5.5.4 Thêm mẫu tin

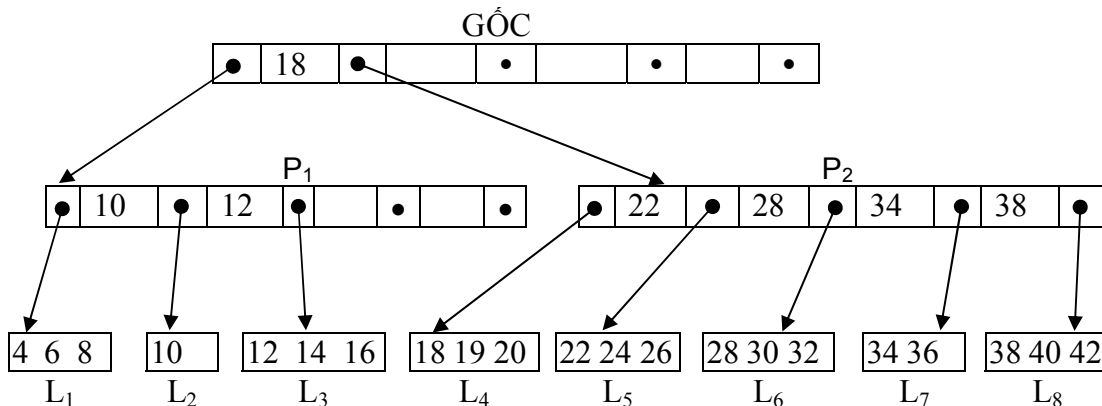
Để thêm một mẫu tin r có khoá là x vào trong B-cây, trước hết ta áp dụng thủ tục tìm kiếm nói trên để tìm r . Việc tìm kiếm này sẽ dẫn đến nút lá L . Nếu tìm thấy thì thông báo “Mẫu tin đã tồn tại”, ngược lại thì L là nút lá mà ta có thể xen r vào trong đó. Nếu khối L này còn đủ chỗ cho r thì ta thêm r vào sao cho đúng thứ tự của nó trong khối L và giải thuật kết thúc. Nếu L không còn chỗ cho r thì ta yêu cầu hệ thống cấp phát một khối mới L' . Dời $\lceil b/2 \rceil$ (b là số mẫu tin nhiều nhất có thể lưu

trong một khối) mẫu tin nằm ở phân nửa cuối khối L sang L' rồi xen r vào L hoặc L' sao cho việc xen đảm bảo thứ tự các khoá trong khối. Giả sử nút P là cha của L (P phải được biết vì thủ tục tìm đi từ gốc đến L phải thông qua P). Bây giờ ta áp dụng thủ tục xen đệ quy để xen vào P một khóa k' và con trỏ p' tương ứng của nút lá L' (k' là khóa của mẫu tin đầu tiên trong L'). Trong trường hợp trước khi xen k' và p', P đã có đủ m con thì ta phải cấp thêm một khối mới P' và chuyển một số con của P sang P' và xen con mới vào P hoặc P' sao cho cả P và P' đều có ít nhất $\lceil m/2 \rceil$ con. Việc chia cắt P này đòi hỏi phải xen một khóa và một con trỏ vào nút cha của P... Quá trình này có thể sẽ dẫn tới nút gốc và cũng có thể phải chia cắt nút gốc, trong trường hợp này phải tạo ra một nút gốc mới mà hai con của nó là hai nửa của nút gốc cũ. Khi đó chiều cao của B-cây sẽ tăng lên 1.

Ví dụ 4-9: Thêm mẫu tin r có khoá 19 vào tập tin được biểu diễn bởi B-cây trong ví dụ 4-8 (hình 4-5)

- Quá trình tìm kiếm sẽ xuất phát từ GỐC đi qua P₂ và dẫn tới nút lá L₄
- Trong nút lá L₄ còn đủ chỗ để xen r vào đúng vị trí và giải thuật kết thúc.

Kết quả việc xen ta được B-cây trong hình 4-6:

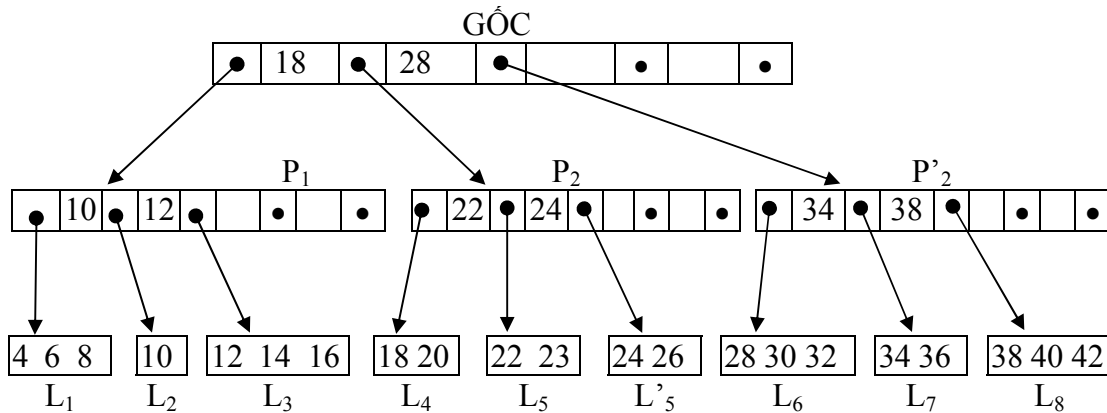


Hình 4-6: Xen thêm mẫu tin r có khoá 19 vào trong B-cây hình 4-5

Ví dụ 4-10: Thêm mẫu tin r có khoá 23 vào trong tập tin biểu diễn bởi B-cây trong ví dụ 4-8 (hình 4-5)

- Quá trình tìm kiếm đi từ nút GỐC, qua P₂ và tới nút lá L₅.
- Vì L₅ đã đủ 3 mẫu tin nên phải tạo ra một nút lá mới L'₅ và chuyển 2 mẫu tin có khóa 24, 26 sang L'₅ sau đó xen r vào L₅.
- Giá trị khóa của mẫu tin đầu tiên trong L'₅ là 24, ta phải xen 24 và con trỏ của L'₅ vào P₂, nhưng P₂ đã có đủ 5 con, vậy cần tạo ra một nút mới P'₂, chuyển các cặp khóa, con trỏ tương ứng với 34 và 38 sang P'₂ và xen cặp con trỏ, khóa 24 vào P₂.
- Do có một nút mới P'₂ nên phải xen vào cha của P₂ (Ở đây là nút GỐC) một cặp khóa, con trỏ tới P'₂. Con trỏ p₀ của nút P'₂ trở tới nút lá L₆,

giá trị khóa đầu tiên của L_6 là 28. Giá trị này phải được xen vào nút GỐC cùng với con trỏ của P'_2 .



Hình 4-7: Xen thêm mẫu tin r có khoá 23 vào trong B-cây hình 4-5

4.5.5.5 Xóa một mẫu tin

Để xóa mẫu tin r có khóa x , trước hết ta tìm mẫu tin r . Nếu không tìm thấy thì thông báo « Mẫu tin không tồn tại », ngược lại ta sẽ xác định được mẫu tin r nằm trong nút lá L và xóa r khỏi L . Nếu r là mẫu tin đầu tiên của L , thì ta phải quay lui lên nút P là cha của L để đặt lại giá trị khóa của L trong P , giá trị mới này bằng giá trị khóa của mẫu tin mới đầu tiên của L . Trong trường hợp L là con đầu tiên của P , thì khóa của L không nằm trong P mà nằm trong tổ tiên của P , chúng ta phải quay lui lên mà sửa đổi.

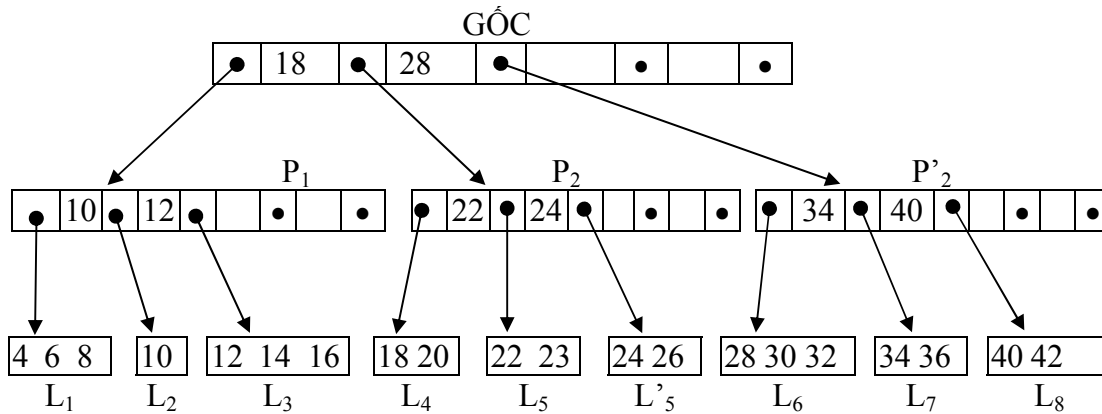
Nếu sau khi xóa mẫu tin r mà L trở nên rỗng thì giải phóng L và quay lui lên nút P là cha của L để xóa cặp khóa-con trỏ của L trong P . Nếu số con của P bây giờ (sau khi xóa khóa-con trỏ của L) nhỏ hơn $\lceil m/2 \rceil$ thì kiểm tra nút P' ngay bên trái hoặc bên phải và cùng mức với P . Nếu P' có ít nhất $\lceil m/2 \rceil + 1$ con, chúng ta chuyển một con của P' sang P . Lúc này cả P và P' có ít nhất $\lceil m/2 \rceil$. Sau đó ta phải cập nhật lại giá trị khóa của P hoặc P' trong cha của chúng, và nếu cần chúng ta phải sửa cả trong tổ tiên của chúng.

Nếu P' có đúng $\lceil m/2 \rceil$ con, ta nối P và P' thành một nút có đúng m con. Sau đó ta phải xóa khóa và con trỏ của P' trong nút cha của P' . Việc xóa này có thể phải quay lui lên tổ tiên của P' . Kết quả của quá trình xóa đệ quy này có thể dẫn tới việc nối hai con của nút gốc, tạo nên một gốc mới và giải phóng nút gốc cũ, độ cao của cây khi đó sẽ giảm đi 1.

Ví dụ 4-11: Xóa mẫu tin r có khóa 38 trong tập tin biểu diễn bởi B-cây kết quả của ví dụ 4-10 (hình 4-7).

- Quá trình tìm kiếm, xuất phát từ nút GỐC, đi qua P'_2 và đến nút lá L_8 ,
- Xóa mẫu tin r khỏi L_8 .
- Mẫu tin đầu tiên của L_8 bây giờ có khóa 40,

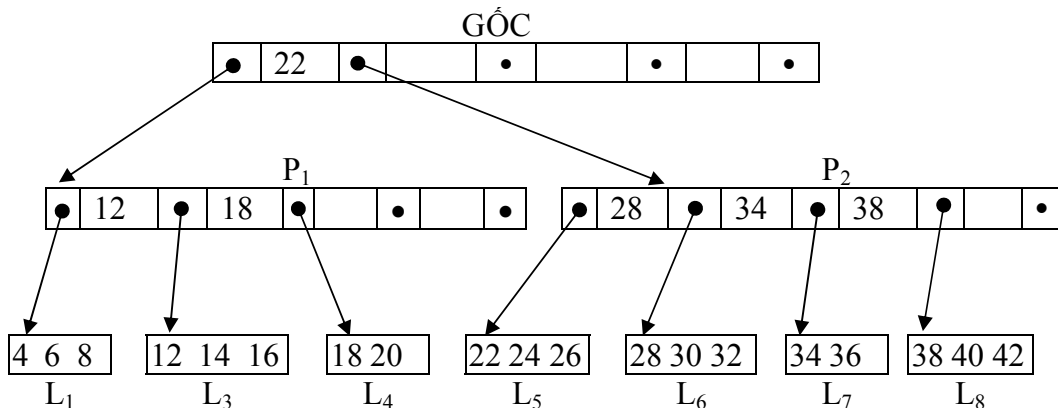
- Sửa lại giá trị khóa của L_8 trong P'_2 (thay 38 bởi 40) ta được kết quả là B-cây sau:



Hình 4-8: Xóa mẫu tin r có khóa 38 vào trong B-cây hình 4-7

Ví dụ 4-12 : Xóa mẫu tin r có khóa 10 trong tập tin biểu diễn bởi B-cây trong ví dụ 4-8 (hình 4-5).

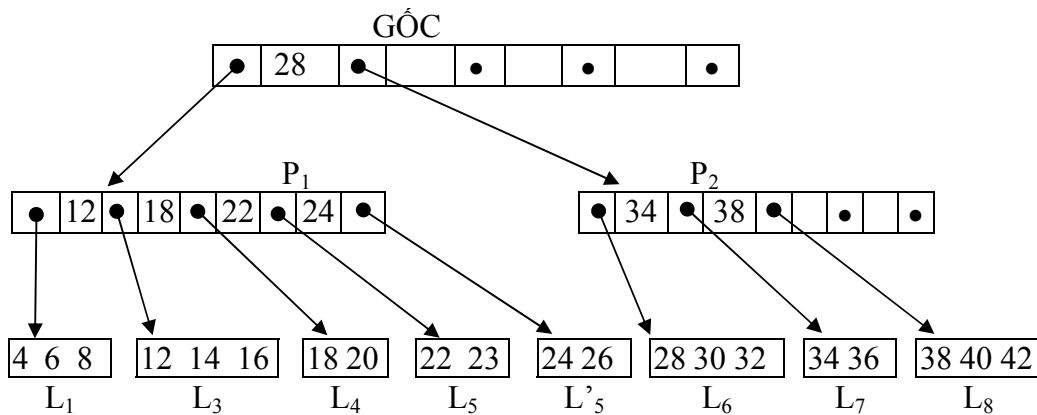
- Quá trình tìm kiếm, xuất phát từ nút GỐC, đi qua P_1 và đến nút lá L_2 .
- Xóa mẫu tin r khỏi L_2 .
- L_2 bây giờ trở nên rỗng, giải phóng L_2 .
- Xóa giá trị khóa 10 và con trỏ của L_2 trong P_1 , P_1 bây giờ chỉ có 2 con (Thiếu con do $2 < \lceil 5/2 \rceil$).
- Xét nút P_2 , bên phải và cùng cấp với P_1 , P_2 có 5 con nên ta chuyển một con từ P_2 sang P_1 .
- Cập nhật lại khóa của P_2 trong nút GỐC, ta được B-cây kết quả như sau:



Hình 4-9: Xóa mẫu tin có khóa 10 trong B-cây hình 4-5

Ví dụ 4-13: Xóa mẫu tin r có khóa 10 trong tập tin biểu diễn bởi B-cây kết quả của ví dụ 4-10 (hình 4-7).

- Quá trình tìm kiếm, xuất phát từ nút GỐC, đi qua P_1 và lần đến nút lá L_2 .
- Xóa mẫu tin r khỏi L_2 .
- L_2 bây giờ trở nên rỗng, giải phóng L_2 .
- Xóa giá trị khóa 10 và con trỏ của L_2 trong P_1 , P_1 bây giờ chỉ có 2 con (Thiếu con do $2 < \lceil 5/2 \rceil$).
- Xét nút P_2 , bên phải và cùng cấp với P_1 , P_2 có đúng $\lceil 5/2 \rceil = 3$ con nên ta nối P_1 và P_2 để P_1 có đúng 5 con, giải phóng P_2 .
- Xóa khóa và con trỏ của P_2 trong nút GỐC, ta được B-cây kết quả như sau:



Hình 4-10: Xóa mẫu tin r có khóa 10 trong B-cây hình 4-7

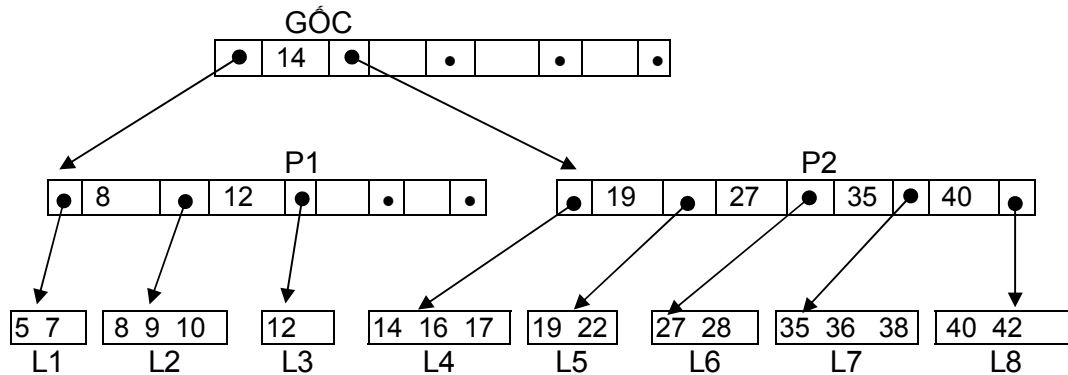
4.6 TỔNG KẾT CHƯƠNG 4

Để đánh giá các giải thuật xử lý ngoài, cần phải xác định số phép truy xuất khối (đọc và ghi khối) mà giải thuật đó thực hiện. Theo đó, một giải thuật được xem là tốt nếu số lượng phép truy xuất khối nhỏ và để cải tiến giải thuật, ta cần tìm cách giảm số phép truy xuất khối. Các giải thuật sắp xếp trộn minh họa khá rõ ràng cho việc cải tiến giải thuật xử lý ngoài.

Đối với việc tổ chức lưu trữ thông tin trong tập tin, chúng ta cần chú ý đến các loại tập tin bảng băm và tập tin B-cây, đây là hai loại tập tin rất hiệu quả.

BÀI TẬP CHƯƠNG 4

Bài 1: Cho tập tin bao gồm các mẫu tin với giá trị khóa là các số nguyên được tổ chức thành B-cây **bậc 5** với các nút lá chứa được nhiều nhất **3 mẫu tin** như sau.



- Xen mẫu tin R có giá trị khóa là 37 vào tập tin được biểu diễn bởi B-cây nói trên.
- Xóa mẫu tin R có giá trị khóa là 12 của tập tin được biểu diễn bởi B-cây nói trên.
- Xóa mẫu tin R có giá trị khóa là 12 của tập tin được biểu diễn bởi B-cây là kết quả của câu a).

Bài 2: Giả sử ta dùng B-cây **bậc 3** với các nút lá chứa được nhiều nhất **2 mẫu tin** để tổ chức tập tin. Khởi đầu tập tin rỗng, hãy mô tả quá trình hình thành tập tin B-cây (bằng hình vẽ) khi thực hiện tuần tự các thao tác sau:

- Xen mẫu tin R có khóa 8
- Xen mẫu tin R có khóa 2
- Xen mẫu tin R có khóa 10
- Xen mẫu tin R có khóa 1
- Xen mẫu tin R có khóa 12
- Xen mẫu tin R có khóa 3
- Xen mẫu tin R có khóa 5
- Xóa mẫu tin R có khóa 8
- Xóa mẫu tin R có khóa 1